



Walmart is an American multinational retail corporation that operates a chain of supercenters, discount departmental stores, and grocery stores in the United States. Walmart has more than 100 million customers worldwide.

```
1 #Importing libraries -
2 import pandas as pd
3 import numpy as np
4 import matplotlib.pyplot as plt
5 import seaborn as sns
6 from scipy.stats import norm
```

Problem Statement & Analyzing Basic Metrics:

The Management team at Walmart Inc. wants to analyze the customer purchase behavior (precisely, purchase amount) against the customer’s gender and the various other factors to help the business make better decisions. They want to understand if the spending habits differ between male and female customers: Do women spend more on Black Friday than men?

Basic Analysis Of The Data - EDA

1. Import the dataset and do usual data analysis steps like checking the structure & characteristics of the dataset.

a. The data type of all columns in the “customers” table.

Hint: We want you to display the data type of each column present in the dataset.

b. You can find the number of rows and columns given in the dataset.

Hint: You can find the shape of the dataset.

c. Check for the missing values and find the number of missing values in each column.

Importing the dataset

```
1 import gdown
2 !gdown 1AwVi5QXwazInLcw_FzxkZ6CXnsFcZTrT
```

Downloading...

From: https://drive.google.com/uc?id=1AwVi5QXwazInLcw_FzxkZ6CXnsFcZTrT

To: /content/walmart_data.csv

100% 23.0M/23.0M [00:00<00:00, 85.7MB/s]

```
1 data= pd.read_csv('walmart_data.csv')
2 pd.options.display.max_columns=999
3 pd.options.display.max_rows=999
4 data.head(5)
```

	User_ID	Product_ID	Gender	Age	Occupation	City_Category	Stay_In_Current_City_Years	Marital_Status	Product_Category	Purchase
0	1000001	P00069042	F	0-17	10	A	2	0	3	8370
1	1000001	P00248942	F	0-17	10	A	2	0	1	15200
2	1000001	P00087842	F	0-17	10	A	2	0	12	1422
3	1000001	P00085442	F	0-17	10	A	2	0	12	1057
4	1000002	P00285442	M	55+	16	C	4+	0	8	7969

Getting Basic Info About The Dataset

```
1 data.info()
```

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 550068 entries, 0 to 550067

Data columns (total 10 columns):

#	Column	Non-Null Count	Dtype
0	User_ID	550068 non-null	int64

```
1 Product_ID          550068 non-null object
2 Gender              550068 non-null object
3 Age                 550068 non-null object
4 Occupation          550068 non-null int64
5 City_Category       550068 non-null object
6 Stay_In_Current_City_Years  550068 non-null object
7 Marital_Status      550068 non-null int64
8 Product_Category    550068 non-null int64
9 Purchase            550068 non-null int64
dtypes: int64(5), object(5)
memory usage: 42.0+ MB
```

>**Insight:** The dataset consists of 5 columns containing object datatype, while 5 column contains integer datatype

```
1 data['City_Category']=data['City_Category'].astype('category')
2 data['Gender']=data['Gender'].astype('category')
3 data['Marital_Status']=data['Marital_Status'].astype('category')
4 data['Age']=data['Age'].astype('category')
5 data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 550068 entries, 0 to 550067
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  -
0   User_ID               550068 non-null  int64
1   Product_ID           550068 non-null  object
2   Gender               550068 non-null  category
3   Age                  550068 non-null  category
4   Occupation           550068 non-null  int64
5   City_Category        550068 non-null  category
6   Stay_In_Current_City_Years  550068 non-null  object
7   Marital_Status       550068 non-null  category
8   Product_Category     550068 non-null  int64
9   Purchase             550068 non-null  int64
dtypes: category(4), int64(4), object(2)
memory usage: 27.3+ MB
```

>**Insight:** Converted City_Category, Marital_Status, Gender, Age into category datatype

Shape Of Dataset

```
1 data.shape
```

```
(550068, 10)
```

>**Insight:** There are 550068 rows and 10 columns in the dataset

Duplicate Values

```
1 data.duplicated().any()
```

```
False
```

```
1 data.duplicated().sum()
```

```
0
```

>**Insight:** There are no duplicate rows in dataset.

Statistical Summary

```
1 data.describe()
```

	User_ID	Occupation	Product_Category	Purchase
count	5.500680e+05	550068.000000	550068.000000	550068.000000
mean	1.003029e+06	8.076707	5.404270	9263.968713
std	1.727592e+03	6.522660	3.936211	5023.065394
min	1.000001e+06	0.000000	1.000000	12.000000
25%	1.001516e+06	2.000000	1.000000	5823.000000
50%	1.003077e+06	7.000000	5.000000	8047.000000
75%	1.004478e+06	14.000000	8.000000	12054.000000
max	1.006040e+06	20.000000	20.000000	23961.000000

>**Insight:** This function calculates summary statistics, including count, mean, standard deviation, minimum, 25th percentile, median (50th percentile), 75th percentile, and maximum, for numerical columns within the DataFrame. These statistics aid in comprehending the central tendency, spread, and distribution of the data.

```
1 data.describe(include= 'object').T
```

	count	unique	top	freq
Product_ID	550068	3631	P00265242	1880
Stay_In_Current_City_Years	550068	5	1	193821

>**Insight:** This command retrieves information about categorical data within the DataFrame, including the count of observations, the number of unique categories, the most frequent category, and its frequency for each categorical column.

2. Non-Graphical Analysis: Value counts and unique attributes

Unique Values

```
1 data.nunique()
```

	0
User_ID	5891
Product_ID	3631
Gender	2
Age	7
Occupation	21
City_Category	3
Stay_In_Current_City_Years	5
Marital_Status	2
Product_Category	20
Purchase	18105

dtype: int64

Value Counts Of Categorical Data

Univariate

Gender

```
1 data['Gender'].value_counts()
```

	count
Gender	
M	414259
F	135809

dtype: int64

```
1 data['Gender'].value_counts(normalize= True)
```

	proportion
Gender	
M	0.753105
F	0.246895

dtype: float64

```
1 value= data['Gender'].value_counts()
2 type_data= data['Gender'].value_counts().index
3 plt.pie(value, labels= type_data, autopct= '%2.2f%')
4 plt.title('Gender Distribution')
5 plt.show()
```



>**Insight:** Dataset has 75.31% of Male and 24.69% of Female people

Marital Status

```
1 data['Marital_Status']= data['Marital_Status'].replace([0, 1], ['Unmarried', 'Married'])
```

```
<ipython-input-15-4a490eb8b2d7>:1: FutureWarning: The behavior of Series.replace (and DataFrame.replace) with CategoricalDtype is deprecated. In a futu
data['Marital_Status']= data['Marital_Status'].replace([0, 1], ['Unmarried', 'Married'])
```

```
1 data['Marital_Status'].value_counts()
```

count	
Marital_Status	
Unmarried	324731
Married	225337

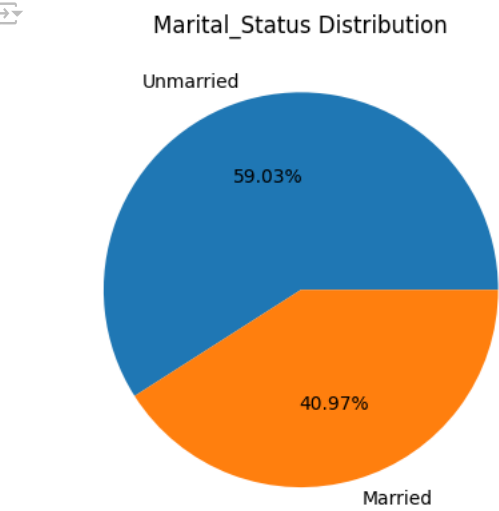
dtype: int64

```
1 data['Marital_Status'].value_counts(normalize= True)
```

proportion	
Marital_Status	
Unmarried	0.590347
Married	0.409653

dtype: float64

```
1 value= data['Marital_Status'].value_counts()
2 type_data= data['Marital_Status'].value_counts().index
3 plt.pie(value, labels= type_data, autopct= '%2.2f%%')
4 plt.title('Marital_Status Distribution')
5 plt.show()
```



>**Insight:** Dataset has 59.03% of Unmarried and 40.97% of Married people

Age Category

```
1 data['Age'].value_counts()
```

count	
Age	
26-35	219587
36-45	110013
18-25	99660
46-50	45701
51-55	38501
55+	21504
0-17	15102

dtype: int64

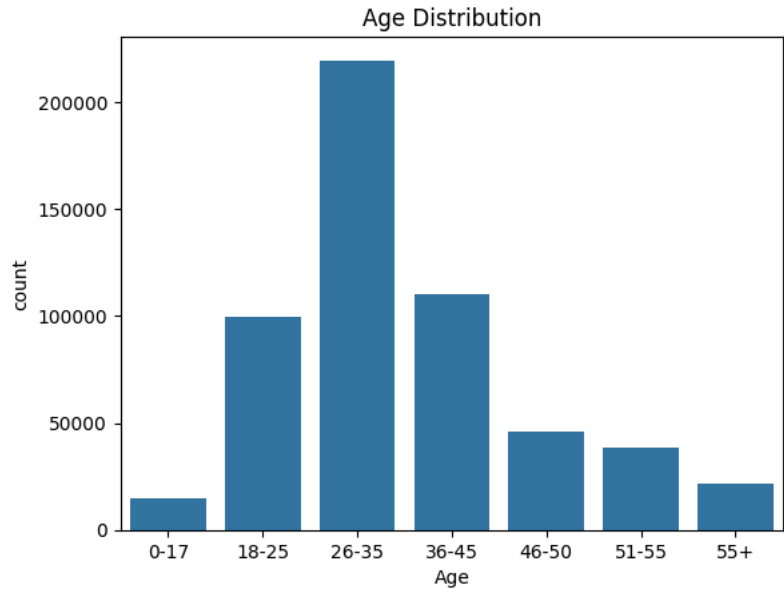
```
1 data['Age'].value_counts(normalize= True)
```



proportion	
Age	
26-35	0.399200
36-45	0.199999
18-25	0.181178
46-50	0.083082
51-55	0.069993
55+	0.039093
0-17	0.027455

dtype: float64

```
1 sns.countplot(data, x= 'Age')
2 plt.title('Age Distribution')
3 plt.show()
```



City_Category

```
1 data['City_Category'].value_counts()
```



count	
City_Category	
B	231173
C	171175
A	147720

dtype: int64

```
1 data['City_Category'].value_counts(normalize= True)
```



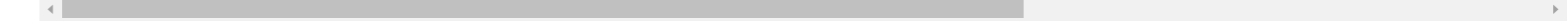
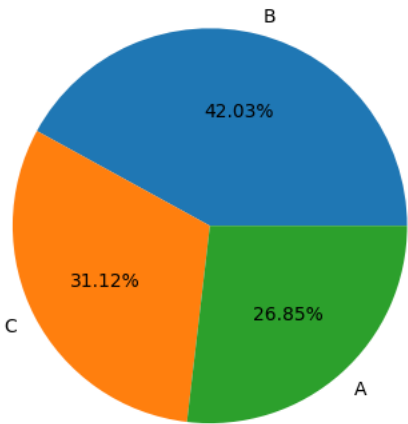
proportion	
City_Category	
B	0.420263
C	0.311189
A	0.268549

dtype: float64

```
1 value= data['City_Category'].value_counts()
2 type_data= data['City_Category'].value_counts().index
3 plt.pie(value, labels= type_data, autopct= '%2.2f%')
4 plt.title('City_Category Distribution')
5 plt.show()
```



City_Category Distribution



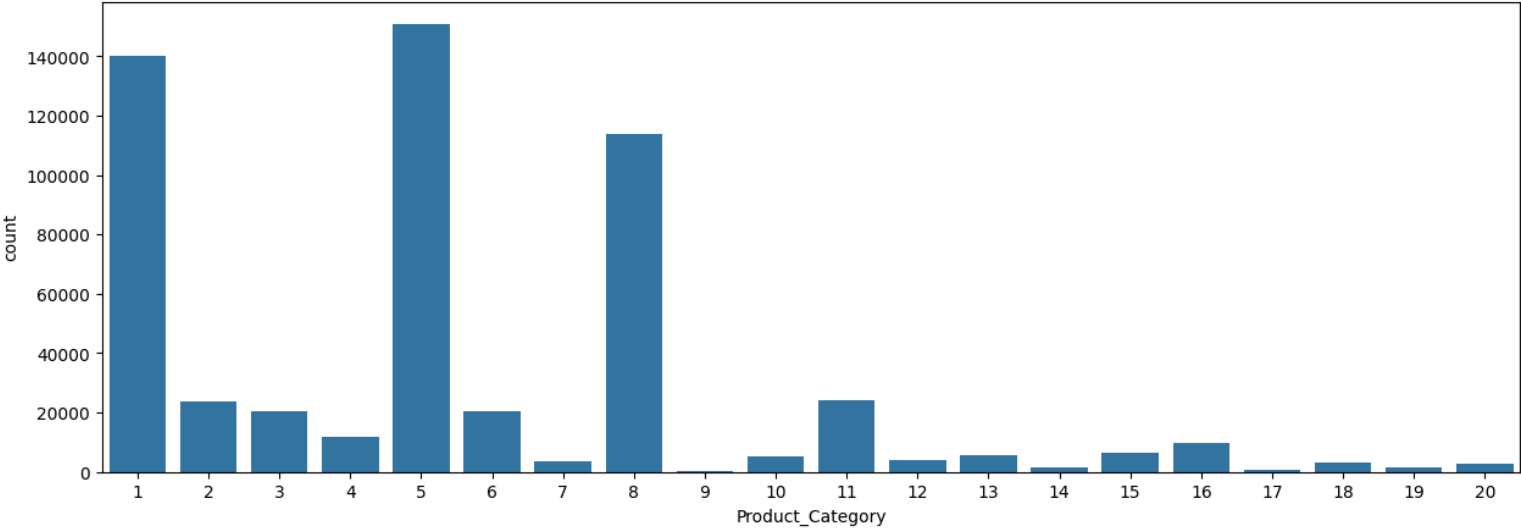
>**Insight:** In City Category, 26.85% belongs to 'A', 42.03% belongs to 'B' and 31.12% belongs to 'C'

Product_Category

```
1 plt.figure(figsize=(15,5))
2 sns.countplot(data, x= 'Product_Category')
3 plt.title('Product_Category Distribution')
4 plt.show()
```



Product_Category Distribution



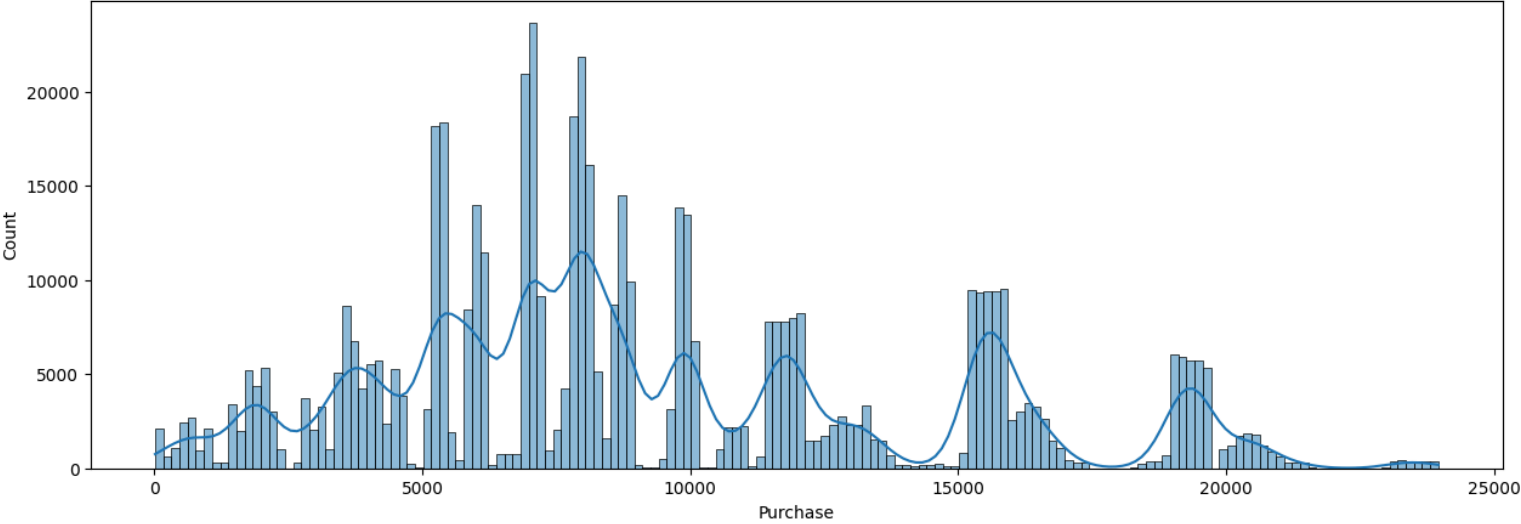
>**Insight:** Product Category 1, 5 and 8 are more famous as thier count is high. Category 5 is the most famous one.

Purchase

```
1 plt.figure(figsize=(15,5))
2 sns.histplot(data, x= 'Purchase', kde= True)
3 plt.title('Purchase Distribution')
4 plt.show()
```



Purchase Distribution



2. Data Exploration

a. What products are different age groups buying?

Hint: You can use histplot to find the relationship between products and age groups

b. Is there a relationship between age, marital status, and the amount spent?

Hint: You can do multivariate analysis to find the relationship between age, marital status, and the amount spent

c. Are there preferred product categories for different genders?

Hint: You can apply different hist plots for different genders

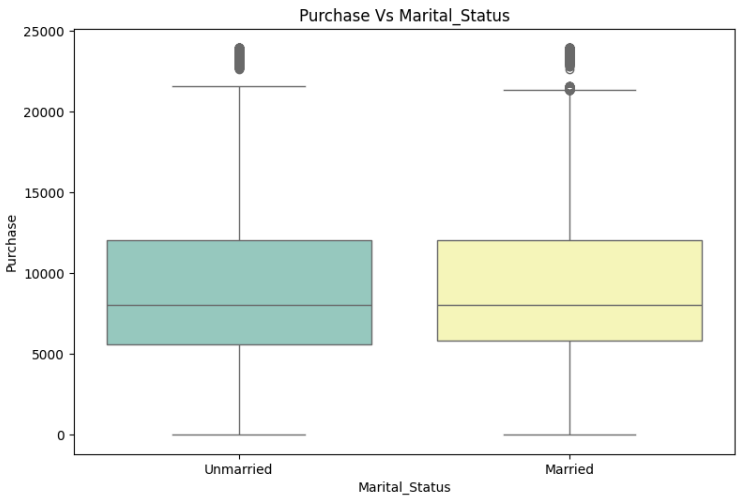
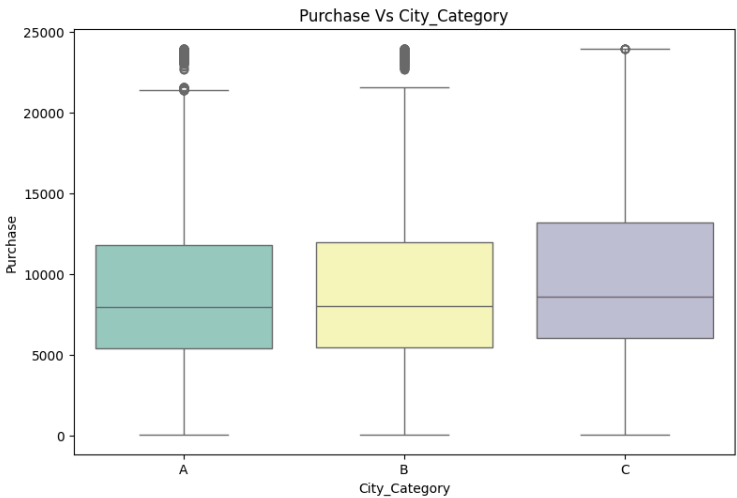
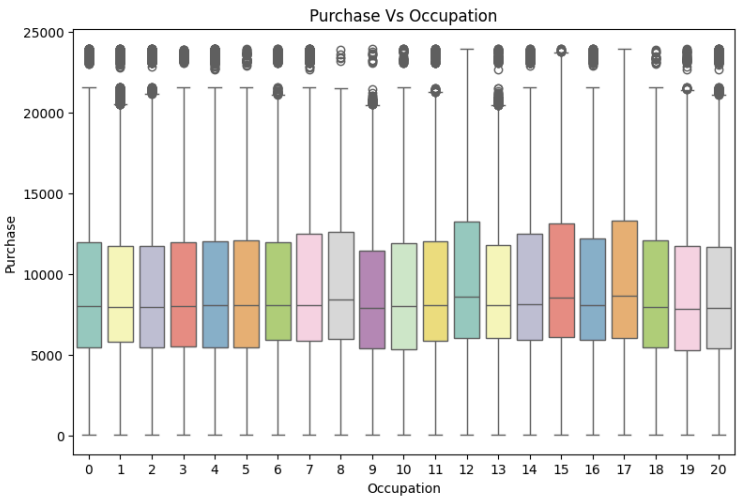
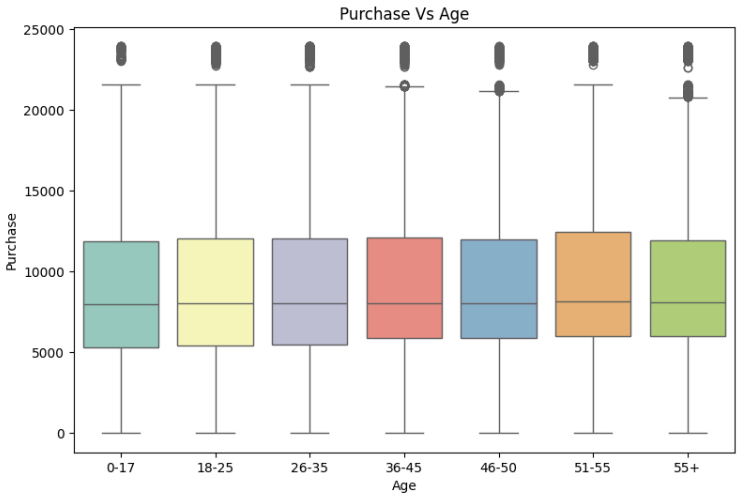
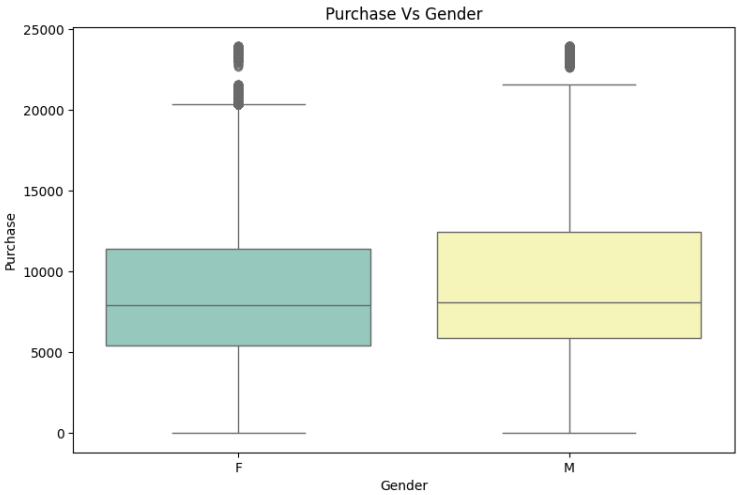
Bivariate

Purchase Vs All

```
1 plt.figure(figsize= (20, 20))
2 plt.suptitle('Purchase Vs All')
3
4 plt.subplot(3, 2, 1)
5 sns.boxplot(data, y= 'Purchase', x= 'Gender', hue= 'Gender', palette='Set3', legend=False)
6 plt.title('Purchase Vs Gender')
7
8 plt.subplot(3, 2, 2)
9 sns.boxplot(data, y= 'Purchase', x= 'Age', hue= 'Age', palette='Set3', legend=False)
10 plt.title('Purchase Vs Age')
11
12 plt.subplot(3, 2, 3)
13 sns.boxplot(data, y= 'Purchase', x= 'Occupation', hue= 'Occupation', palette='Set3', legend=False)
14 plt.title('Purchase Vs Occupation')
15
16 plt.subplot(3, 2, 4)
17 sns.boxplot(data, y= 'Purchase', x= 'City_Category', hue= 'City_Category', palette='Set3', legend=False)
18 plt.title('Purchase Vs City_Category')
19
20 plt.subplot(3, 2, 5)
21 sns.boxplot(data, y= 'Purchase', x= 'Stay_In_Current_City_Years', hue= 'Stay_In_Current_City_Years', palette='Set3', legend=False)
22 plt.title('Purchase Vs Stay_In_Current_City_Years')
23
24 plt.subplot(3, 2, 6)
25 sns.boxplot(data, y= 'Purchase', x= 'Marital_Status', hue= 'Marital_Status', palette='Set3', legend=False)
26 plt.title('Purchase Vs Marital_Status')
27
28 plt.show()
```

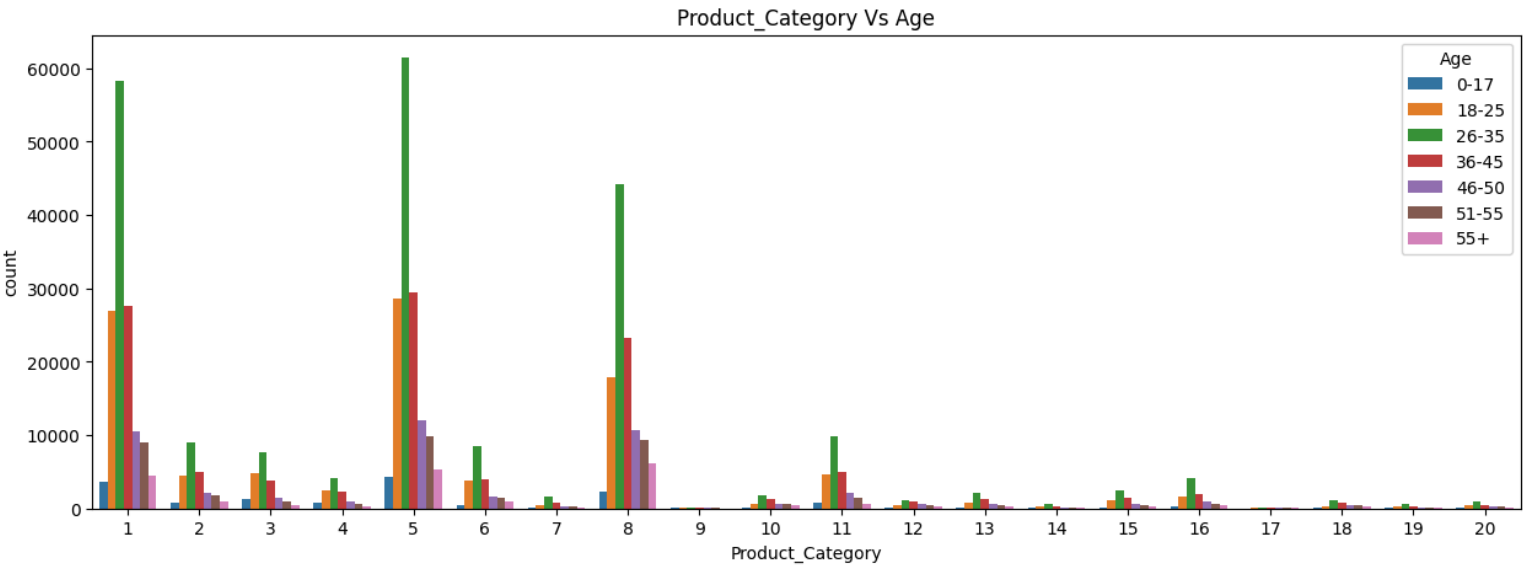


Purchase Vs All



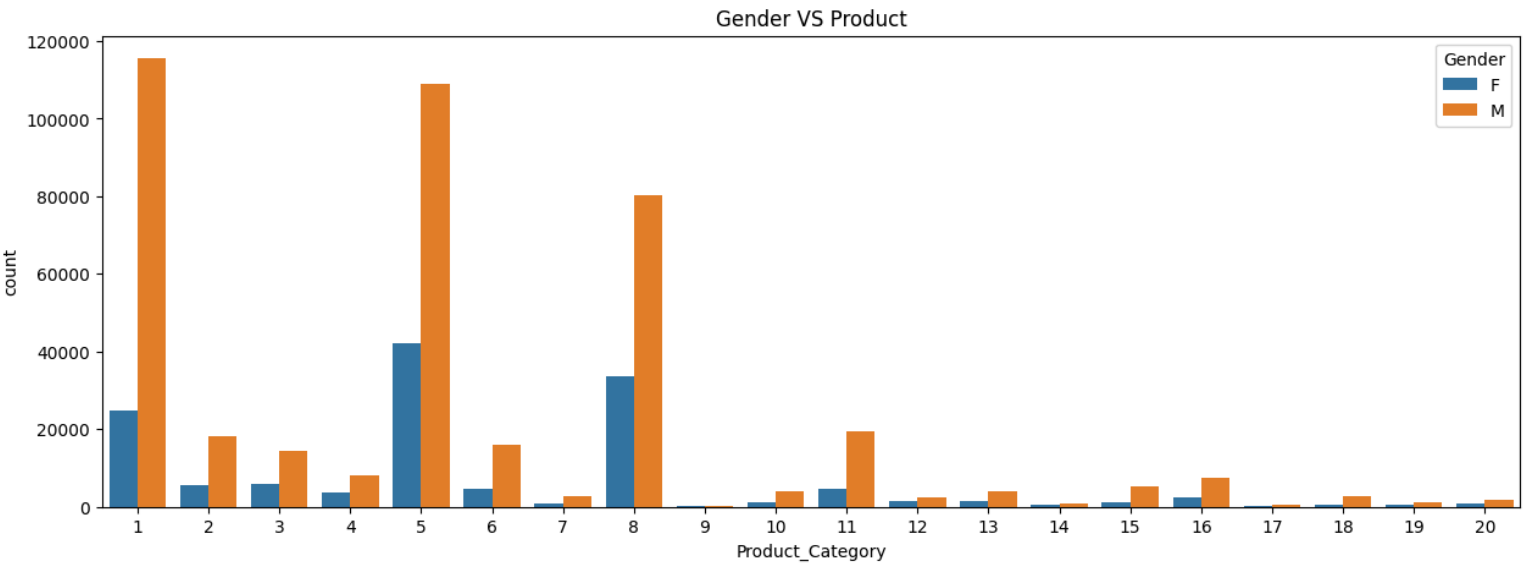
Age VS Product

```
1 plt.figure(figsize=(15,5))
2 plt.title('Product_Category Vs Age')
3 sns.countplot(data, x= 'Product_Category', hue= 'Age')
4 plt.show()
```

Gender VS Product

```
1 plt.figure(figsize=(15,5))
2 plt.title('Gender VS Product')
3 sns.countplot(data, x= 'Product_Category', hue= 'Gender')
4 plt.show()
```



Correlation

```
1 #Converting categorical data to numerical datas for better analysis on correlation
2 data_copy= data[['Gender', 'Age', 'Occupation', 'City_Category',
3     'Stay_In_Current_City_Years', 'Marital_Status', 'Product_Category',
4     'Purchase']].copy()
5
6 data_copy['Gender']= data_copy['Gender'].replace(['M', 'F'], [0, 1])
7
8 data_copy['Age']= data_copy['Age'].replace(['0-17', '18-25', '26-35', '36-45', '46-50', '51-55', '55+'], [0, 1, 2, 3, 4, 5, 6])
9
10 data_copy['City_Category']= data_copy['City_Category'].replace(['A', 'C', 'B'], [0, 1, 2])
11
12 data_copy['Stay_In_Current_City_Years']= data_copy['Stay_In_Current_City_Years'].replace(['0', '1', '2', '3', '4+'], [0, 1, 2, 3, 4])
13
14 data_copy['Marital_Status']= data_copy['Marital_Status'].replace(['Unmarried', 'Married'], [0, 1])
```



```
<ipython-input-30-cfb28d409a3b>:6: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a future version. To retain th
data_copy['Gender']= data_copy['Gender'].replace(['M', 'F'], [0, 1])
<ipython-input-30-cfb28d409a3b>:6: FutureWarning: The behavior of Series.replace (and DataFrame.replace) with CategoricalDtype is deprecated. In a futu
data_copy['Gender']= data_copy['Gender'].replace(['M', 'F'], [0, 1])
<ipython-input-30-cfb28d409a3b>:8: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a future version. To retain th
data_copy['Age']= data_copy['Age'].replace(['0-17', '18-25', '26-35', '36-45', '46-50', '51-55', '55+'], [0, 1, 2, 3, 4, 5, 6])
<ipython-input-30-cfb28d409a3b>:8: FutureWarning: The behavior of Series.replace (and DataFrame.replace) with CategoricalDtype is deprecated. In a futu
data_copy['Age']= data_copy['Age'].replace(['0-17', '18-25', '26-35', '36-45', '46-50', '51-55', '55+'], [0, 1, 2, 3, 4, 5, 6])
<ipython-input-30-cfb28d409a3b>:10: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a future version. To retain t
data_copy['City_Category']= data_copy['City_Category'].replace(['A', 'C', 'B'], [0, 1, 2])
<ipython-input-30-cfb28d409a3b>:10: FutureWarning: The behavior of Series.replace (and DataFrame.replace) with CategoricalDtype is deprecated. In a fut
data_copy['City_Category']= data_copy['City_Category'].replace(['A', 'C', 'B'], [0, 1, 2])
<ipython-input-30-cfb28d409a3b>:12: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a future version. To retain t
data_copy['Stay_In_Current_City_Years']= data_copy['Stay_In_Current_City_Years'].replace(['0', '1', '2', '3', '4+'], [0, 1, 2, 3, 4])
<ipython-input-30-cfb28d409a3b>:14: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a future version. To retain t
data_copy['Marital_Status']= data_copy['Marital_Status'].replace(['Unmarried', 'Married'], [0, 1])
<ipython-input-30-cfb28d409a3b>:14: FutureWarning: The behavior of Series.replace (and DataFrame.replace) with CategoricalDtype is deprecated. In a fut
data_copy['Marital_Status']= data_copy['Marital_Status'].replace(['Unmarried', 'Married'], [0, 1])
```

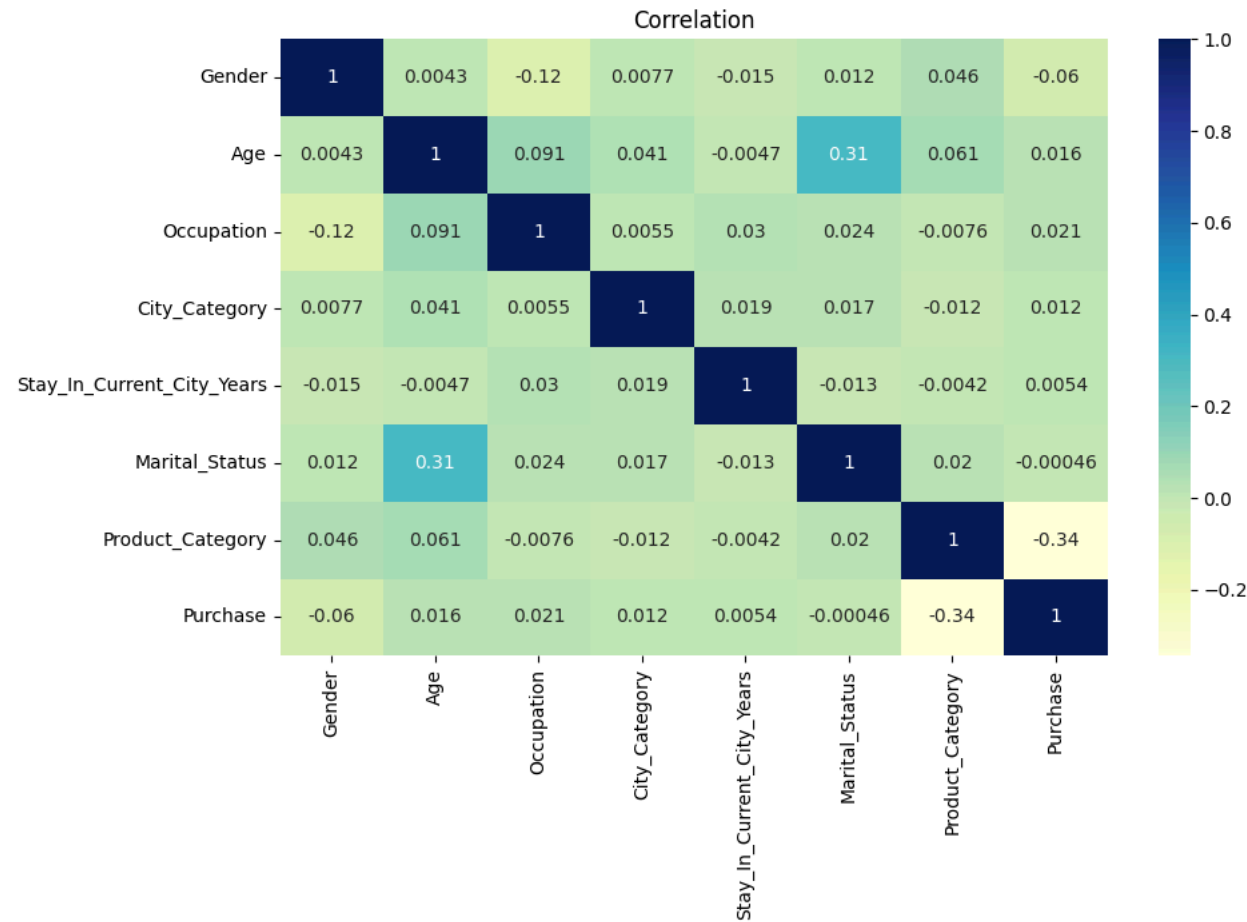
```
1 data_copy.corr()
```



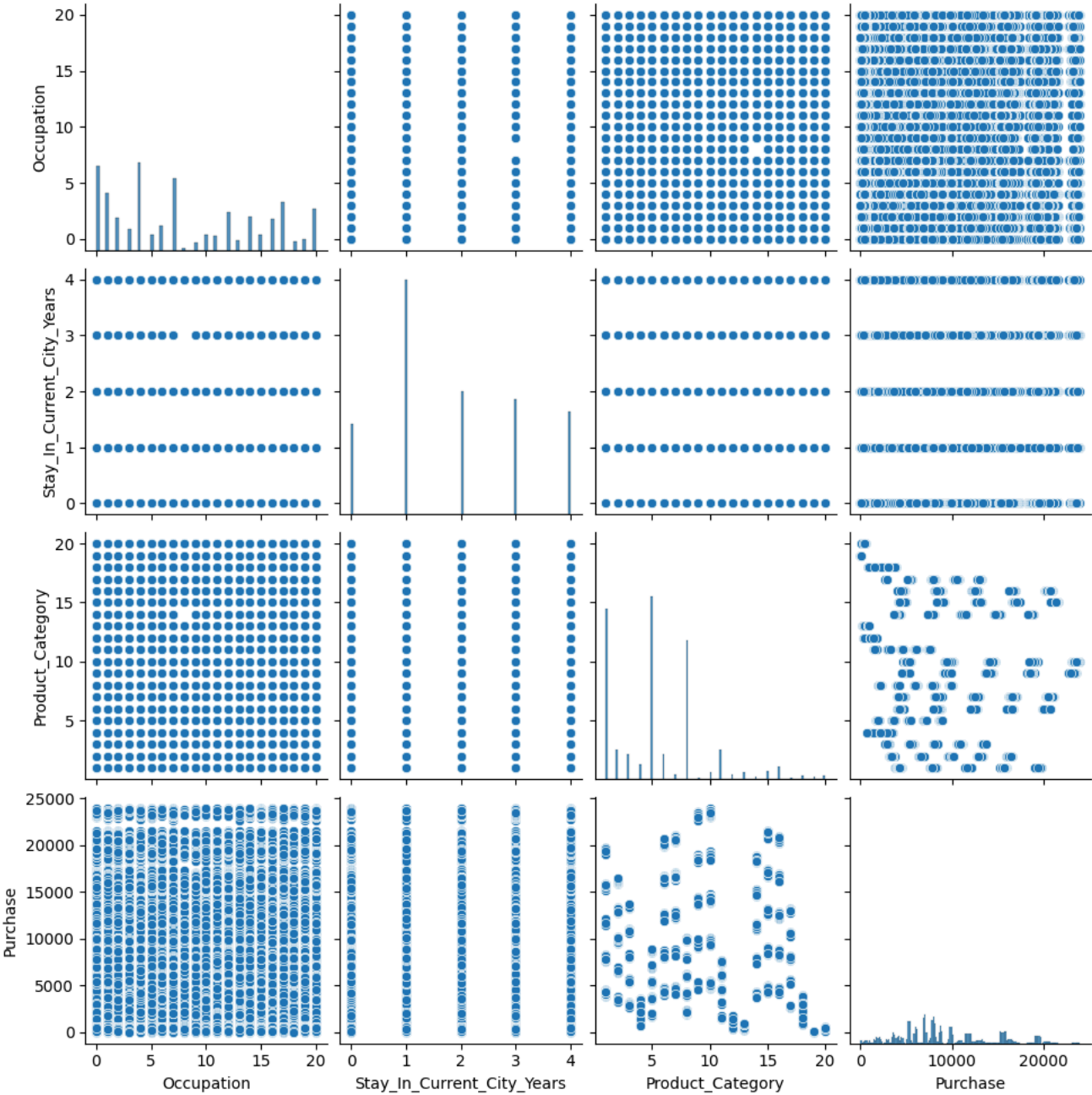
	Gender	Age	Occupation	City_Category	Stay_In_Current_City_Years	Marital_Status	Product_Category	Purchase
Gender	1.000000	0.004262	-0.117291	0.007688	-0.014660	0.011603	0.045594	-0.060346
Age	0.004262	1.000000	0.091463	0.041047	-0.004712	0.311738	0.061197	0.015839
Occupation	-0.117291	0.091463	1.000000	0.005470	0.030005	0.024280	-0.007618	0.020833
City_Category	0.007688	0.041047	0.005470	1.000000	0.019280	0.016846	-0.012204	0.011513
Stay_In_Current_City_Years	-0.014660	-0.004712	0.030005	0.019280	1.000000	-0.012819	-0.004213	0.005422
Marital_Status	0.011603	0.311738	0.024280	0.016846	-0.012819	1.000000	0.019888	-0.000463
Product_Category	0.045594	0.061197	-0.007618	-0.012204	-0.004213	0.019888	1.000000	-0.343703
Purchase	-0.060346	0.015839	0.020833	0.011513	0.005422	-0.000463	-0.343703	1.000000



```
1 plt.figure(figsize=(10,6))
2 sns.heatmap(data_copy.corr(), cmap="YlGnBu", annot=True)
3 plt.title('Correlation')
4 plt.show()
```



```
1 sns.pairplot(data= data_copy)
2 plt.show()
```



3. Missing Value & Outlier Detection

- a. Find the outliers for every continuous variable in the dataset
- Hint: Use boxplots to find the outliers in the given dataset
- b. Remove/clip the data between the 5 percentile and 95 percentile
- Hint: You can use np.clip() for clipping the data

1. Missing Values

```
1 data.isna().any()
```



	0
User_ID	False
Product_ID	False
Gender	False
Age	False
Occupation	False
City_Category	False
Stay_In_Current_City_Years	False
Marital_Status	False
Product_Category	False
Purchase	False

dtype: bool

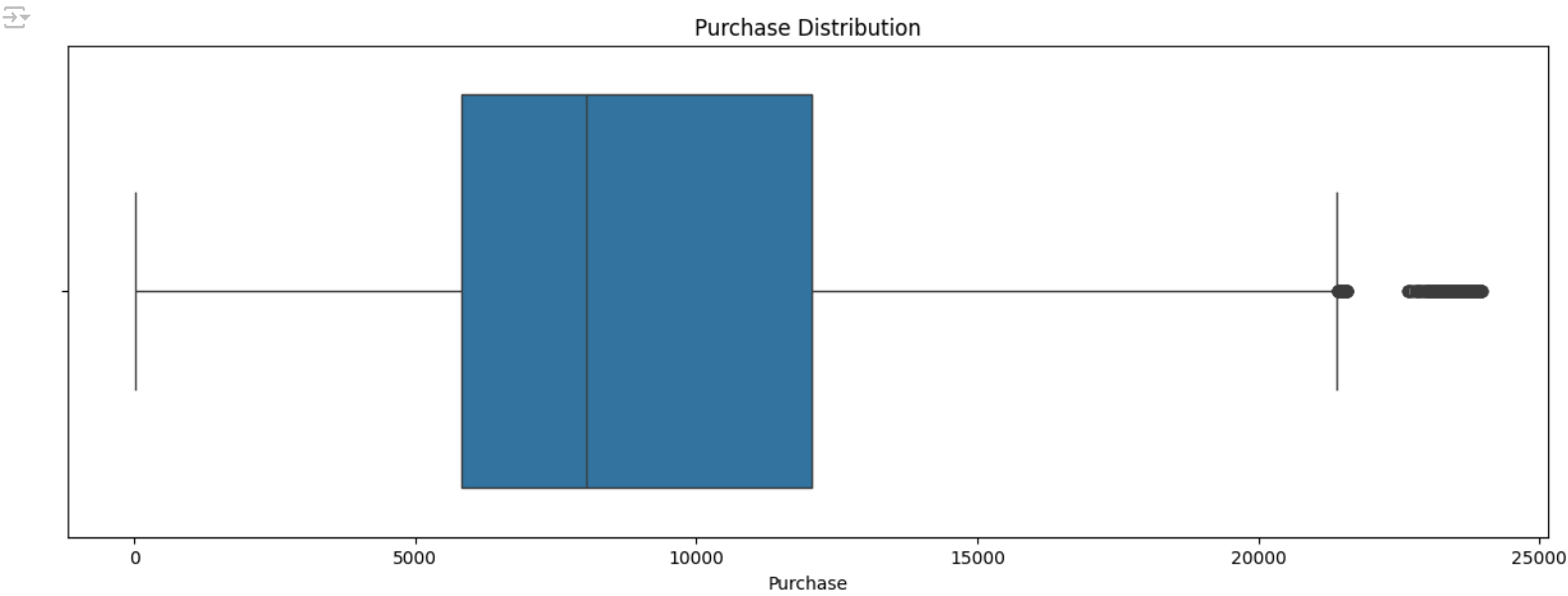
```
1 data.isna().sum().sum()
```

0

>Insight: There are no missing values in dataset.

2. Outlier Detection

```
1 plt.figure(figsize=(15,5))
2 sns.boxplot(data, x= 'Purchase')
3 plt.title('Purchase Distribution')
4 plt.show()
```



```
1 q1=data.Purchase.quantile(0.25)
2 q3=data.Purchase.quantile(0.75)
3 print(q1,q3)
4 IQR=q3-q1
5 outliers = data[((data.Purchase<(q1-1.5*IQR)) | (data.Purchase>(q3+1.5*IQR)))]
6 print("num outliers :",len(outliers))
7 print("percent outliers :",len(outliers)/len(data))
```

```
5823.0 12054.0
num outliers : 2677
percent outliers : 0.004866671029763593
```

Treatement Of Outliers

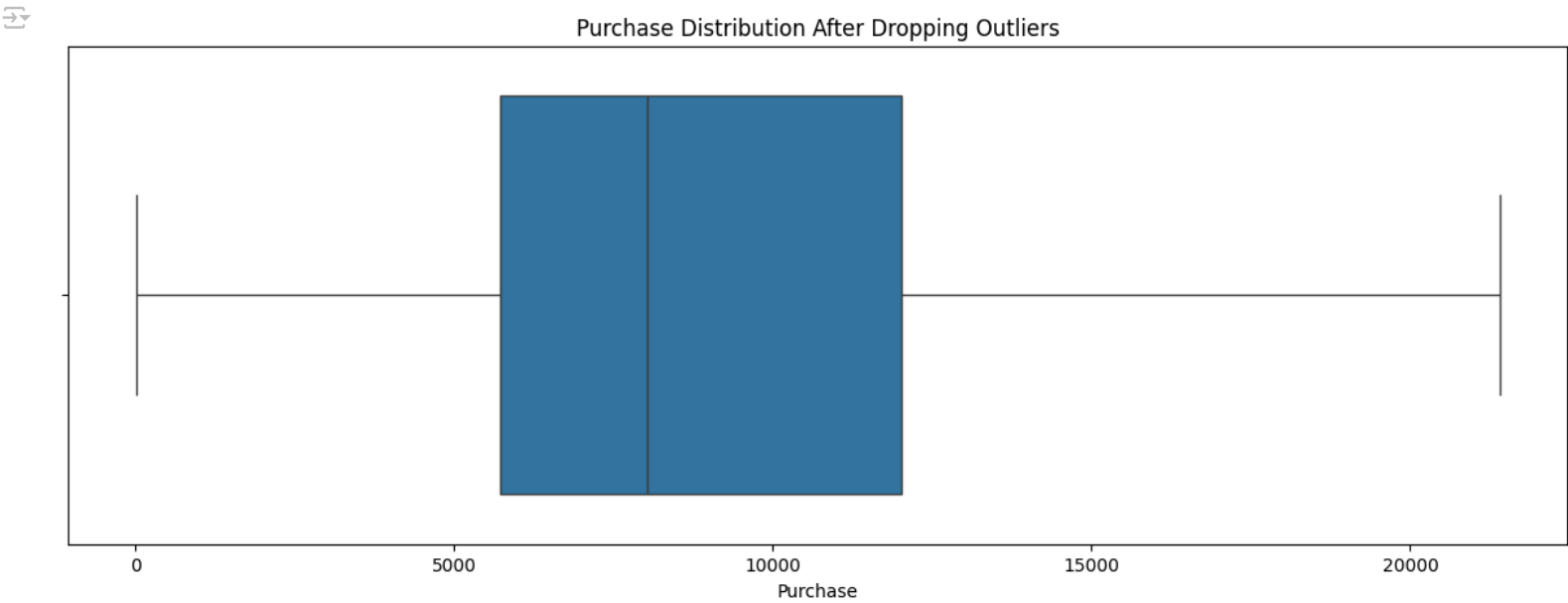
We Can Clip The Data Between 5 Percentile and 95 Percentile Or Drop The Outliers

```
1 #Option 1: Code to clip data
2 # p5= np.percentile(data['Purchase'], 5)
3 # p95= np.percentile(data['Purchase'], 95)
4 # data['Purchase']= np.clip(data['Purchase'], p5, p95)
5
6 #Option 2: Drop the Outliers
7 data=data.drop(data[(data.Purchase > (q3+1.5*IQR)) | (data.Purchase < (q1-1.5*IQR))].index)
```

After Treatement Of Outliers

After Dropping The Outliers

```
1 plt.figure(figsize=(15,5))
2 sns.boxplot(data, x= 'Purchase')
3 plt.title('Purchase Distribution After Dropping Outliers')
4 plt.show()
```



4. How does gender affect the amount spent?

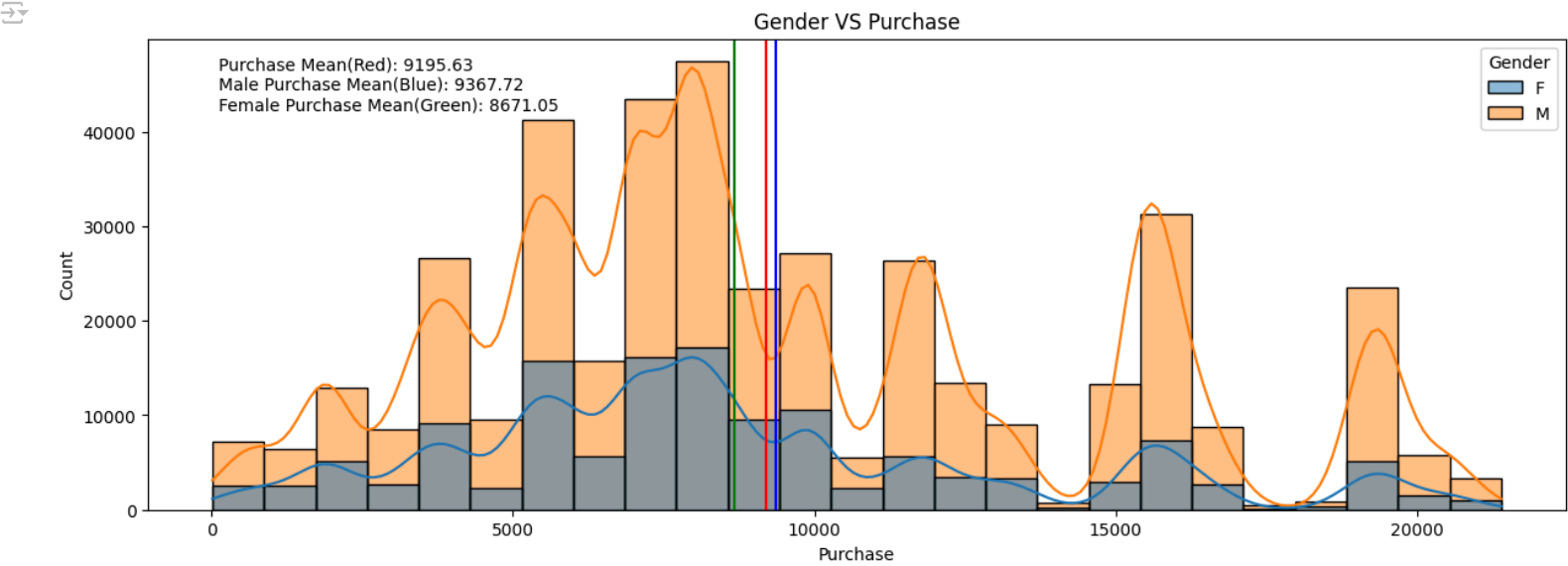
Hint: Use the central limit theorem or bootstrapping to compute the 95% confidence intervals for the average amount spent per gender. First, compute the confidence interval for whatever data is available, and then repeat the same with smaller sample sizes - 300, 3000, and 30000.

```
1 data.groupby('Gender')['Purchase'].describe()
```

```
<ipython-input-40-8f3cb5ae094b>:1: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas:
data.groupby('Gender')['Purchase'].describe()
```

	count	mean	std	min	25%	50%	75%	max
Gender								
F	135220.0	8671.049039	4679.058483	12.0	5429.0	7906.0	11064.0	21398.0
M	412171.0	9367.724355	5009.234088	12.0	5852.0	8089.0	12247.0	21399.0

```
1 plt.figure(figsize=(15,5))
2 sns.histplot(data, bins=25, x= 'Purchase', hue= 'Gender', fill= True, kde= True)
3 plt.title('Gender VS Purchase')
4
5 plt.axvline(x= data['Purchase'].mean(), color='r')
6 mean= data['Purchase'].mean()
7 plt.axvline(x= data[data['Gender']=='M']['Purchase'].mean(), color='b')
8 m_mean= data[data['Gender']=='M']['Purchase'].mean()
9 plt.axvline(x= data[data['Gender']=='F']['Purchase'].mean(), color='g')
10 f_mean= data[data['Gender']=='F']['Purchase'].mean()
11 plt.annotate(f'Purchase Mean(Red): {np.round(mean, 2)} \nMale Purchase Mean(Blue): {np.round(m_mean, 2)} \nFemale Purchase Mean(Green):
12              {np.round(f_mean, 2)}', xy=(0.05, 0.85), xycoords='axes fraction', fontsize=10)
13
14 plt.show()
```



```
1 print('Male Purchase Amount Mean:', data.loc[data['Gender']=='M', 'Purchase'].mean())
2 print('Male Purchase Amount Std Dev:', data.loc[data['Gender']=='M', 'Purchase'].std())
3 print()
4 print('Female Purchase Amount Mean:', data.loc[data['Gender']=='F', 'Purchase'].mean())
5 print('Female Purchase Amount Std Dev:', data.loc[data['Gender']=='F', 'Purchase'].std())
```

```
Male Purchase Amount Mean: 9367.724354697444
Male Purchase Amount Std Dev: 5009.234087946683

Female Purchase Amount Mean: 8671.049038603756
Female Purchase Amount Std Dev: 4679.058483084425
```

95% Confidence Interval For The Average Amount Spent By Male.

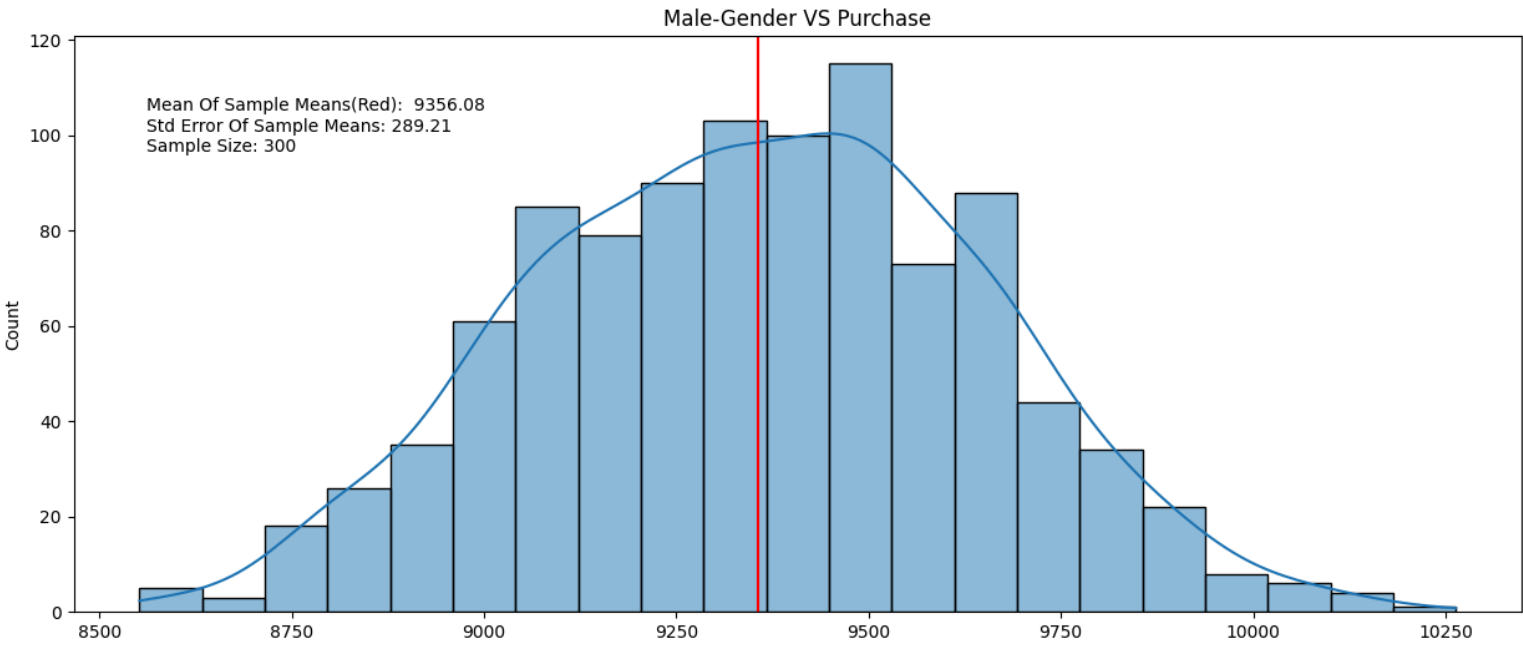
Sample Size: 300

```
1 sample_size= 300
2 Gender= 'M'
3 male_sample_mean_300= [np.mean(data.loc[data['Gender']== Gender, 'Purchase'].sample(sample_size, replace= True)) for i in range(1000)]
4 mean= np.mean(male_sample_mean_300)
5 se= data.loc[data['Gender']== Gender, 'Purchase'].std()/np.sqrt(sample_size)
```

```
1 print('Male Purchase Amount- Mean Of Sample Means:', mean)
2 print('Male Purchase Amount- Std Error Of Sample Means:', se)
```

```
Male Purchase Amount- Mean Of Sample Means: 9356.081343333333
Male Purchase Amount- Std Error Of Sample Means: 289.20826491098666
```

```
1 plt.figure(figsize=(15,6))
2 plt.title('Male-Gender VS Purchase')
3 sns.histplot(male_sample_mean_300, kde= True, fill= True)
4 plt.axvline(x= np.mean(male_sample_mean_300), color='r')
5 plt.annotate(f'Mean Of Sample Means(Red): {np.mean(male_sample_mean_300) : .2f} \nStd Error Of Sample Means:
6              {np.round(se, 2)} \nSample Size: {sample_size}', xy=(0.05, 0.80), xycoords='axes fraction', fontsize=10)
7 plt.show()
```

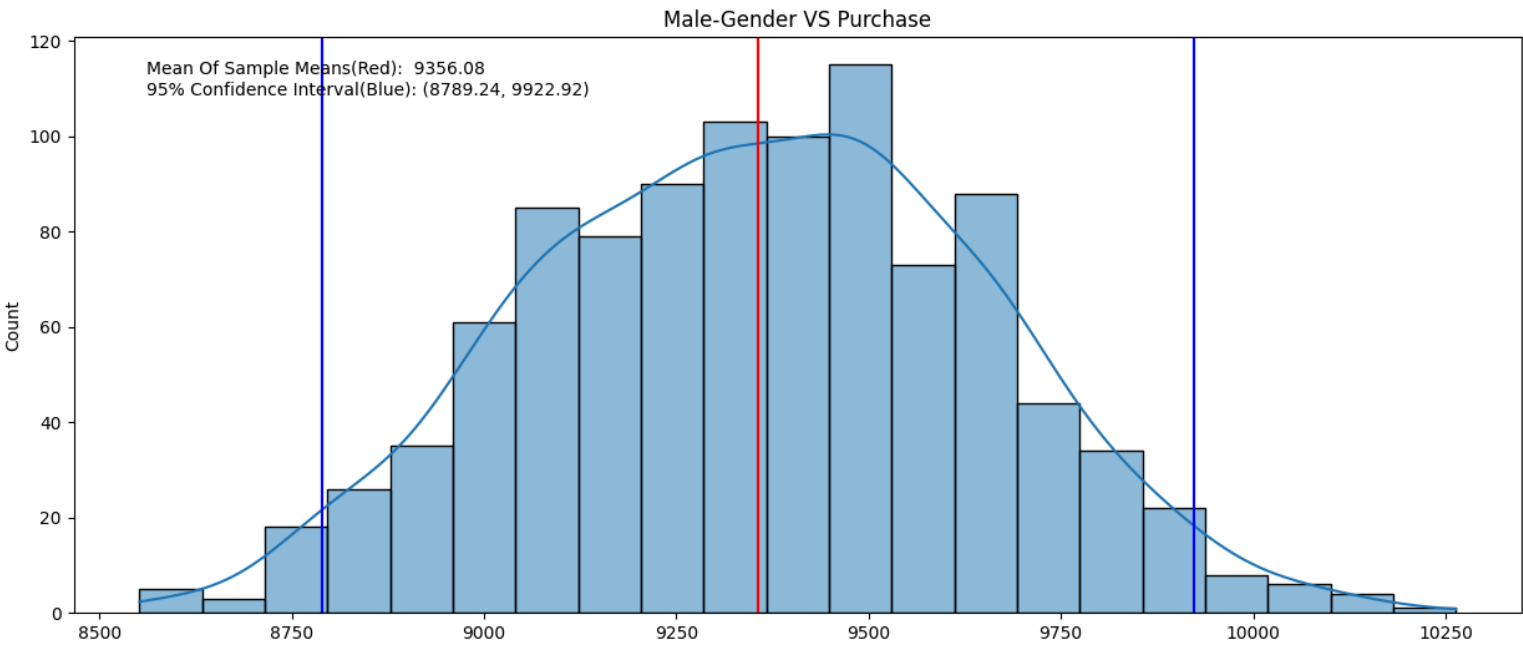


```
1 male_lower_bound= mean + (norm.ppf(0.025) * se)
2 male_upper_bound= mean + (norm.ppf(0.975) * se)
3 print('95% Confidence Interval For The Average Amount Spent By Male For Sample Size 300:', (male_lower_bound, male_upper_bound))
```



95% Confidence Interval For The Average Amount Spent By Male For Sample Size 300: (8789.24356007648, 9922.919126590186)

```
1 plt.figure(figsize=(15,6))
2 plt.title('Male-Gender VS Purchase')
3 sns.histplot(male_sample_mean_300, kde= True, fill= True)
4 plt.axvline(x= np.mean(male_sample_mean_300), color='r')
5 plt.axvline(x= male_lower_bound, color='b')
6 plt.axvline(x= male_upper_bound, color='b')
7 plt.annotate(f'Mean Of Sample Means(Red): {np.mean(male_sample_mean_300) : .2f} \n95% Confidence Interval(Blue):
8             {(np.round(male_lower_bound, 2), np.round(male_upper_bound,2))}', xy=(0.05, 0.90), xycoords='axes fraction', fontsize=10)
9 plt.show()
```



95% Confidence Interval For The Average Amount Spent By Female.

Sample Size: 300

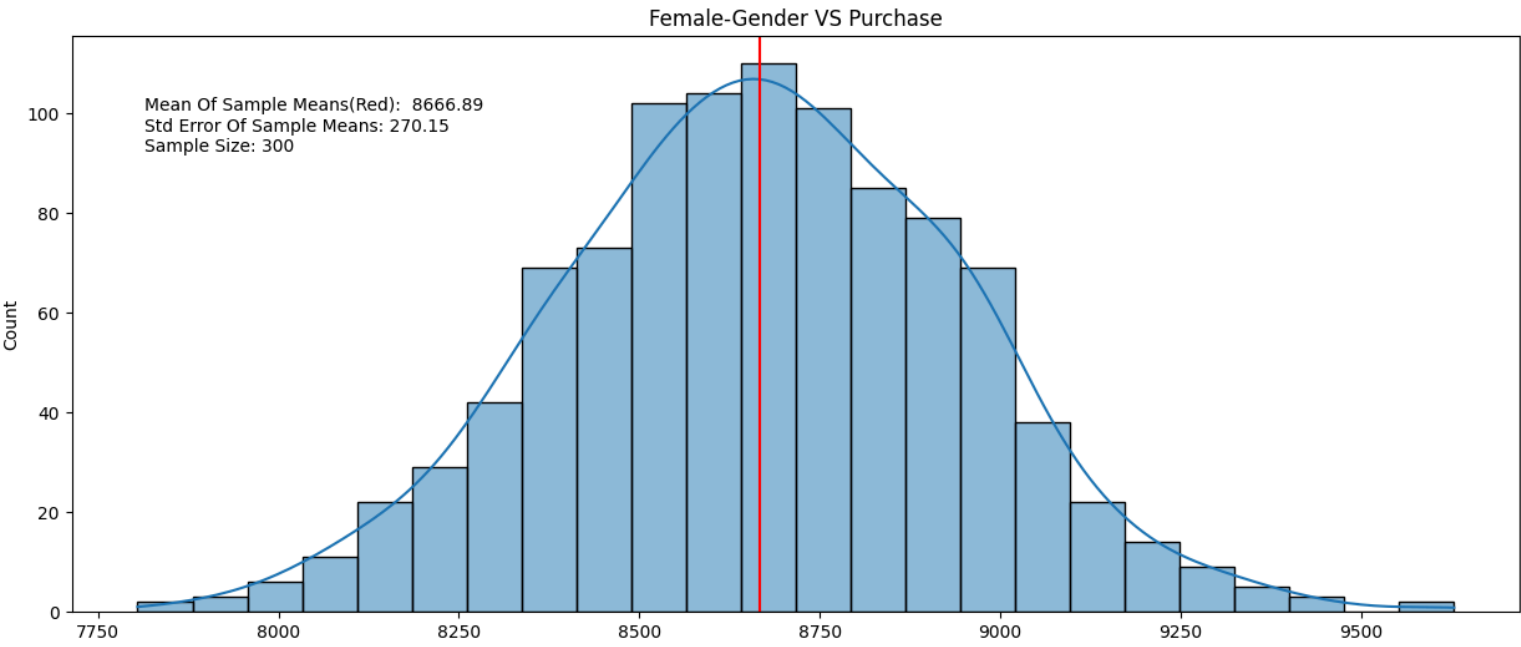
```
1 sample_size= 300
2 Gender= 'F'
3 female_sample_mean_300= [np.mean(data.loc[data['Gender']== Gender, 'Purchase'].sample(sample_size, replace= True)) for i in range(1000)]
4 mean= np.mean(female_sample_mean_300)
5 se= data.loc[data['Gender']== Gender, 'Purchase'].std()/np.sqrt(sample_size)
```

```
1 print('Female Purchase Amount- Mean Of Sample Means:', mean)
2 print('Female Purchase Amount- Std Error Of Sample Means:', se)
```



Female Purchase Amount- Mean Of Sample Means: 8666.891956666666
Female Purchase Amount- Std Error Of Sample Means: 270.14556747627944

```
1 plt.figure(figsize=(15,6))
2 plt.title('Female-Gender VS Purchase')
3 sns.histplot(female_sample_mean_300, kde= True, fill= True)
4 plt.axvline(x= np.mean(female_sample_mean_300), color='r')
5 plt.annotate(f'Mean Of Sample Means(Red): {np.mean(female_sample_mean_300) : .2f} \nStd Error Of Sample Means:
6             {np.round(se, 2)} \nSample Size: {sample_size}', xy=(0.05, 0.80), xycoords='axes fraction', fontsize=10)
7 plt.show()
```

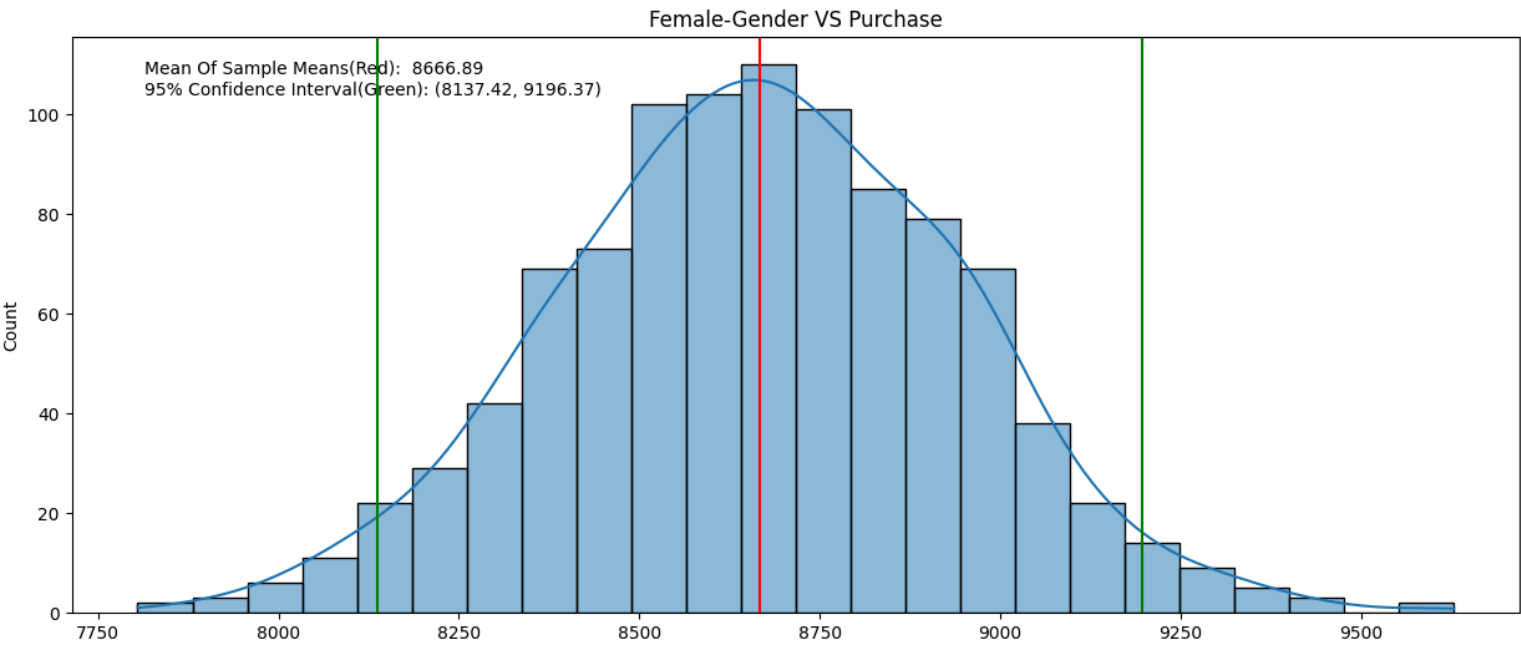


```
1 female_lower_bound= mean + (norm.ppf(0.025) * se)
2 female_upper_bound= mean + (norm.ppf(0.975) * se)
3 print('95% Confidence Interval For The Average Amount Spent By Female For Sample Size 300:', (female_lower_bound, female_upper_bound))
```



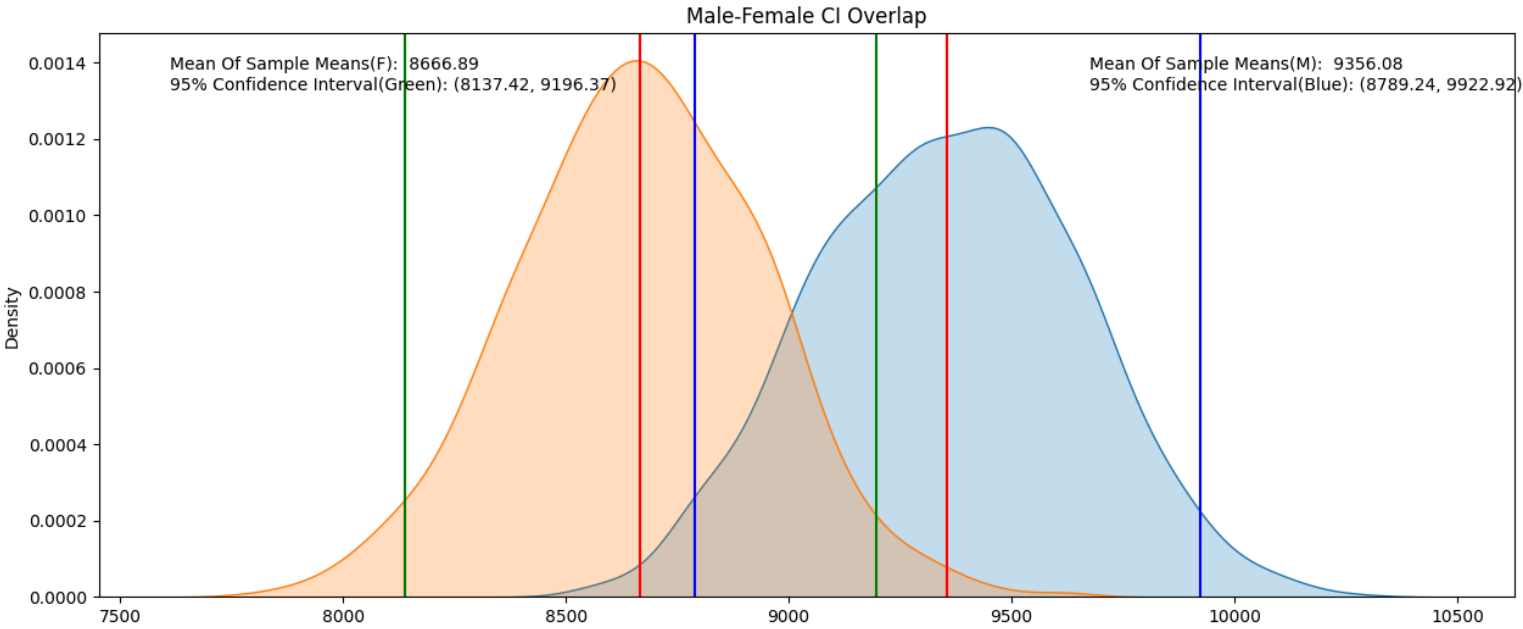
95% Confidence Interval For The Average Amount Spent By Female For Sample Size 300: (8137.416373830023, 9196.36753950331)

```
1 plt.figure(figsize=(15,6))
2 plt.title('Female-Gender VS Purchase')
3 sns.histplot(female_sample_mean_300, kde= True, fill= True)
4 plt.axvline(x= np.mean(female_sample_mean_300), color='r')
5 plt.axvline(x= female_lower_bound, color='g')
6 plt.axvline(x= female_upper_bound, color='g')
7 plt.annotate(f'Mean Of Sample Means(Red): {np.mean(female_sample_mean_300) : .2f} \n95% Confidence Interval(Green):
8             {(np.round(female_lower_bound, 2), np.round(female_upper_bound,2))}', xy=(0.05, 0.90), xycoords='axes fraction', fontsize=10)
9 plt.show()
```



Overlapping ?

```
1 plt.figure(figsize=(15,6))
2 plt.title('Male-Female CI Overlap')
3 sns.kdeplot(male_sample_mean_300, fill= True)
4 sns.kdeplot(female_sample_mean_300, fill= True)
5 plt.axvline(x= np.mean(male_sample_mean_300), color='r')
6 plt.axvline(x= male_lower_bound, color='b')
7 plt.axvline(x= male_upper_bound, color='b')
8 plt.axvline(x= np.mean(female_sample_mean_300), color='r')
9 plt.axvline(x= female_lower_bound, color='g')
10 plt.axvline(x= female_upper_bound, color='g')
11 plt.annotate(f'Mean Of Sample Means(M): {np.mean(male_sample_mean_300) : .2f} \n95% Confidence Interval(Blue):
12             {(np.round(male_lower_bound, 2), np.round(male_upper_bound,2))}', xy=(0.70, 0.90), xycoords='axes fraction', fontsize=10)
13 plt.annotate(f'Mean Of Sample Means(F): {np.mean(female_sample_mean_300) : .2f} \n95% Confidence Interval(Green):
14             {(np.round(female_lower_bound, 2), np.round(female_upper_bound,2))}', xy=(0.05, 0.90), xycoords='axes fraction', fontsize=10)
15
16 plt.show()
```



95% Confidence Interval For The Average Amount Spent By Male.

Sample Size: 3000

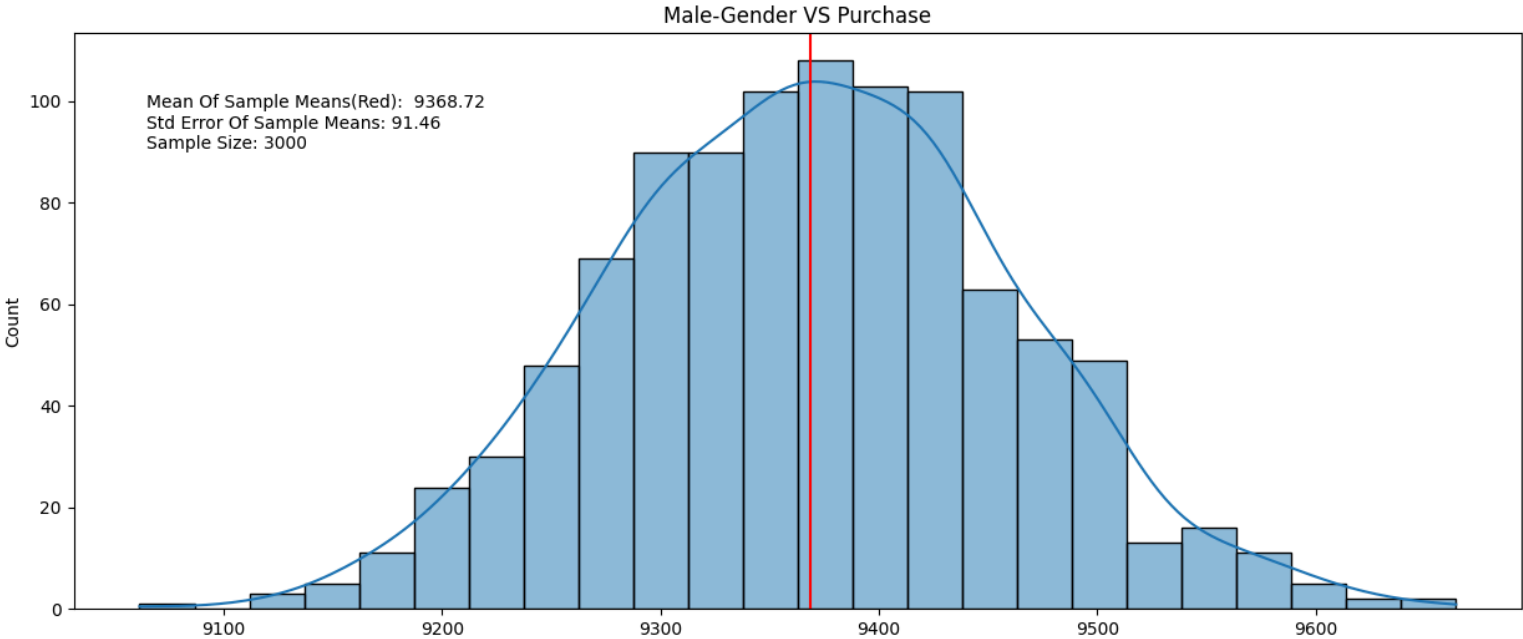
```
1 sample_size= 3000
2 Gender= 'M'
3 male_sample_mean_3000= [np.mean(data.loc[data['Gender']== Gender, 'Purchase'].sample(sample_size, replace= True)) for i in range(1000)]
4 mean= np.mean(male_sample_mean_3000)
5 se= data.loc[data['Gender']== Gender, 'Purchase'].std()/np.sqrt(sample_size)
```

```
1 print('Male Purchase Amount- Mean Of Sample Means:', mean)
2 print('Male Purchase Amount- Std Error Of Sample Means:', se)
```



Male Purchase Amount- Mean Of Sample Means: 9368.720682
Male Purchase Amount- Std Error Of Sample Means: 91.45568352640717

```
1 plt.figure(figsize=(15,6))
2 plt.title('Male-Gender VS Purchase')
3 sns.histplot(male_sample_mean_3000, kde= True, fill= True)
4 plt.axvline(x= np.mean(male_sample_mean_3000), color='r')
5 plt.annotate(f'Mean Of Sample Means(Red): {np.mean(male_sample_mean_3000) : .2f} \nStd Error Of Sample Means:
6             {np.round(se, 2)} \nSample Size: {sample_size}', xy=(0.05, 0.80), xycoords='axes fraction', fontsize=10)
7 plt.show()
```

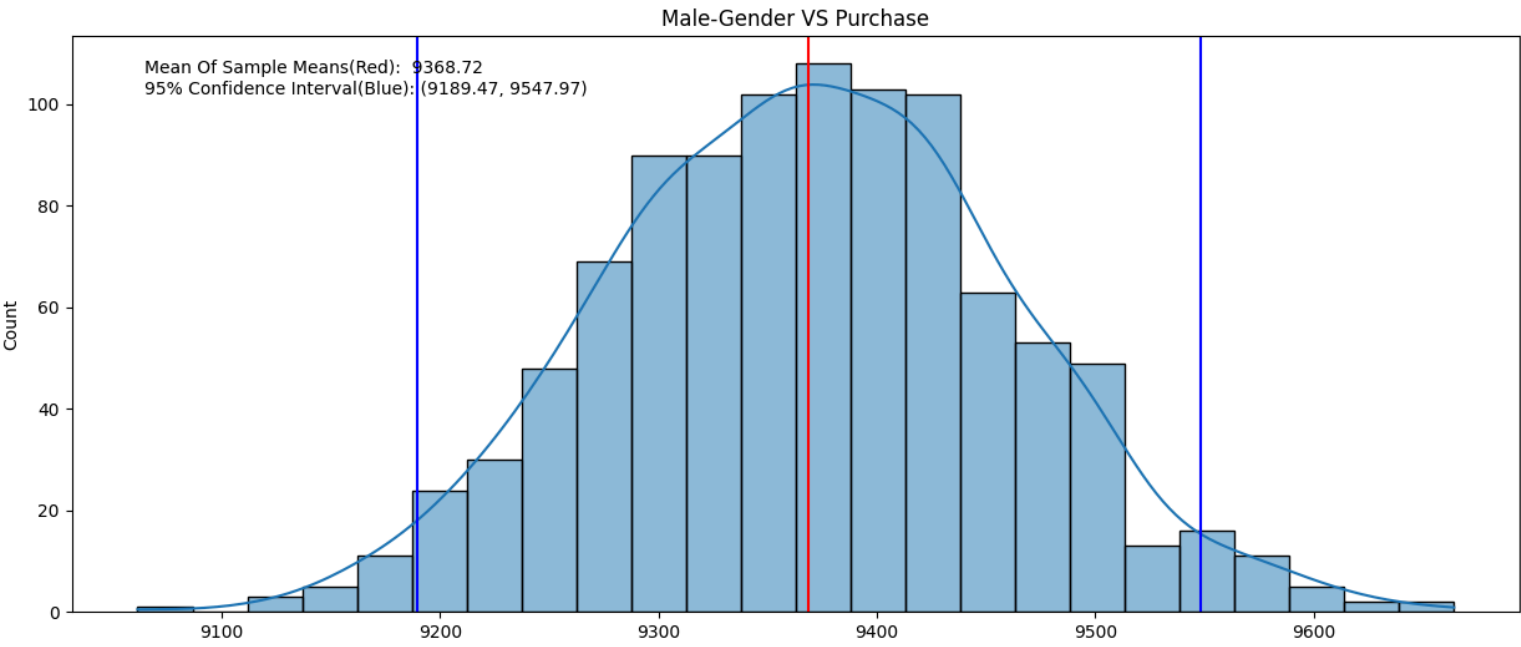


```
1 male_lower_bound= mean + (norm.ppf(0.025) * se)
2 male_upper_bound= mean + (norm.ppf(0.975) * se)
3 print('95% Confidence Interval For The Average Amount Spent By Male For Sample Size 3000:', (male_lower_bound, male_upper_bound))
```



95% Confidence Interval For The Average Amount Spent By Male For Sample Size 3000: (9189.470836106748, 9547.97052789325)

```
1 plt.figure(figsize=(15,6))
2 plt.title('Male-Gender VS Purchase')
3 sns.histplot(male_sample_mean_3000, kde= True, fill= True)
4 plt.axvline(x= np.mean(male_sample_mean_3000), color='r')
5 plt.axvline(x= male_lower_bound, color='b')
6 plt.axvline(x= male_upper_bound, color='b')
7 plt.annotate(f'Mean Of Sample Means(Red): {np.mean(male_sample_mean_3000) : .2f} \n95% Confidence Interval(Blue):
8             {(np.round(male_lower_bound, 2), np.round(male_upper_bound,2))}', xy=(0.05, 0.90), xycoords='axes fraction', fontsize=10)
9 plt.show()
```

95% Confidence Interval For The Average Amount Spent By Female.

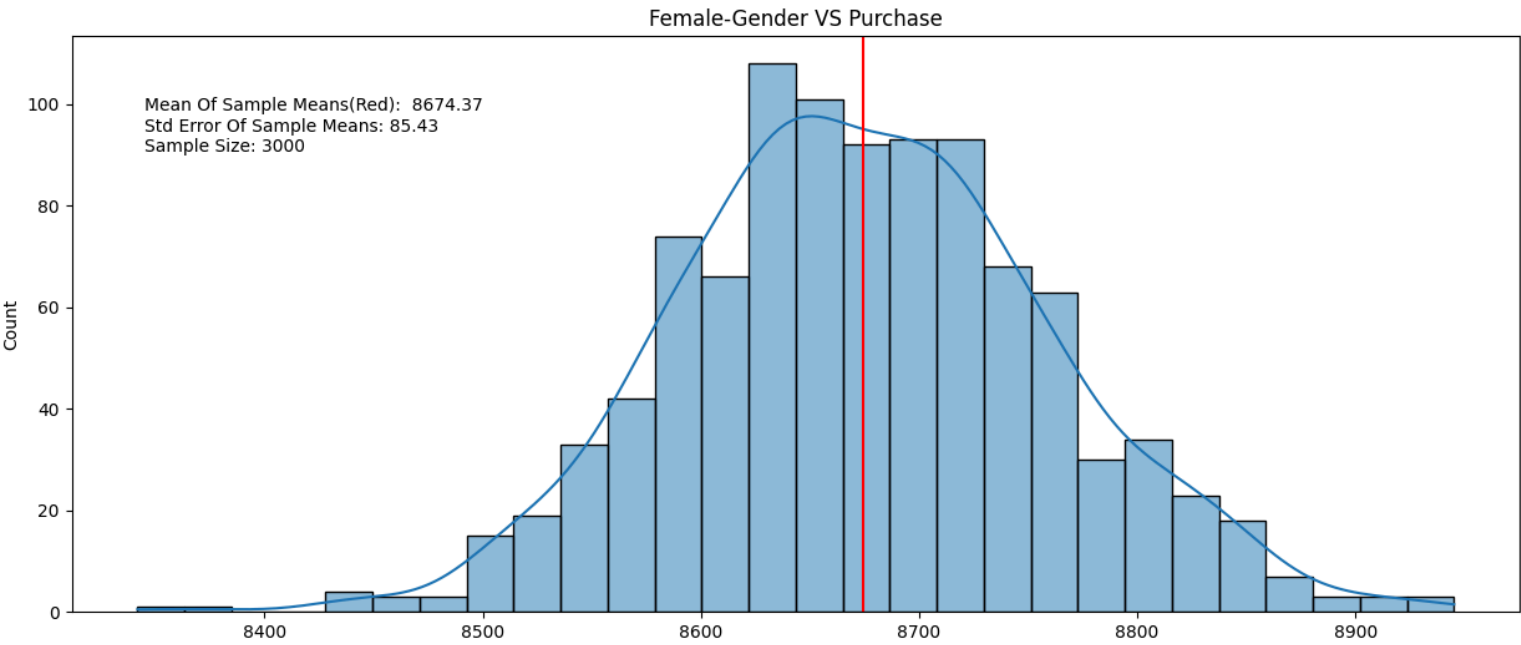
Sample Size: 3000

```
1 sample_size= 3000
2 Gender= 'F'
3 female_sample_mean_3000= [np.mean(data.loc[data['Gender']== Gender, 'Purchase'].sample(sample_size, replace= True)) for i in range(1000)]
4 mean= np.mean(female_sample_mean_3000)
5 se= data.loc[data['Gender']== Gender, 'Purchase'].std()/np.sqrt(sample_size)
```

```
1 print('Female Purchase Amount- Mean Of Sample Means:', mean)
2 print('Female Purchase Amount- Std Error Of Sample Means:', se)
```

```
Female Purchase Amount- Mean Of Sample Means: 8674.368553333336
Female Purchase Amount- Std Error Of Sample Means: 85.4275293023748
```

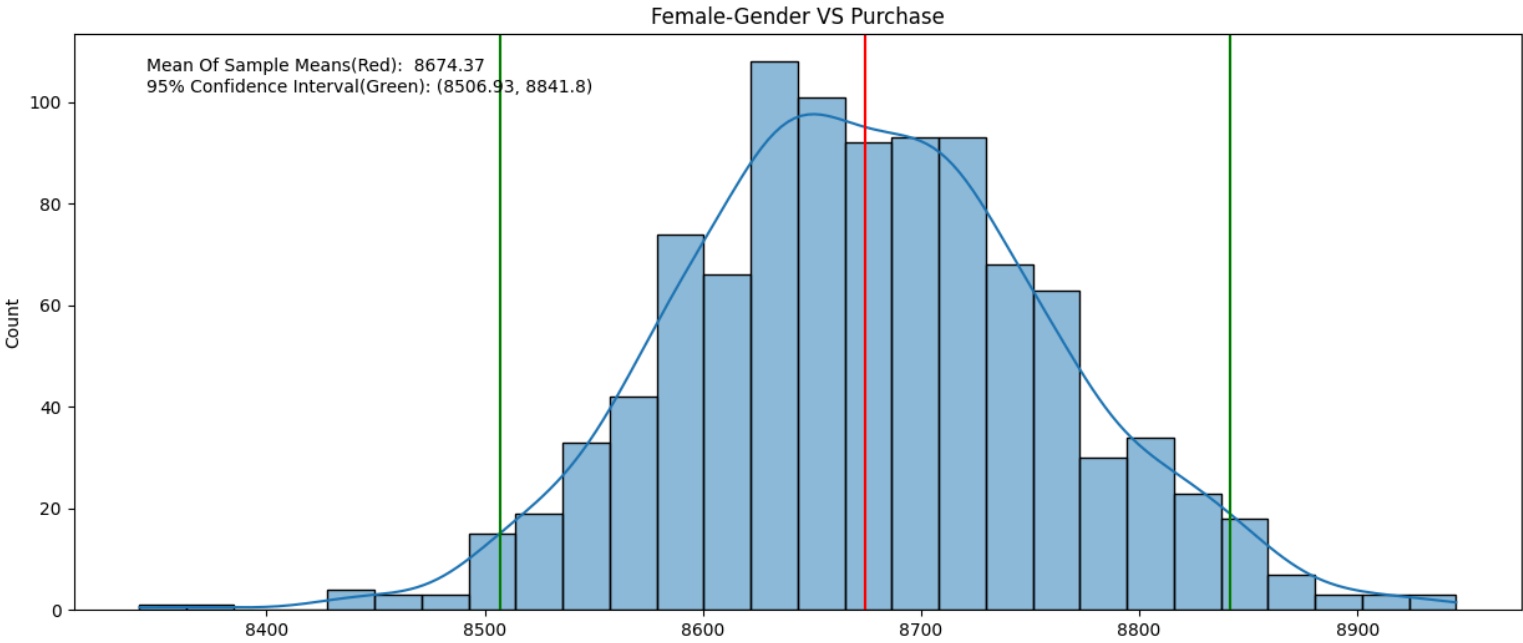
```
1 plt.figure(figsize=(15,6))
2 plt.title('Female-Gender VS Purchase')
3 sns.histplot(female_sample_mean_3000, kde= True, fill= True)
4 plt.axvline(x= np.mean(female_sample_mean_3000), color='r')
5 plt.annotate(f'Mean Of Sample Means(Red): {np.mean(female_sample_mean_3000) : .2f} \nStd Error Of Sample Means:
6             {np.round(se, 2)} \nSample Size: {sample_size}', xy=(0.05, 0.80), xycoords='axes fraction', fontsize=10)
7 plt.show()
```



```
1 female_lower_bound= mean + (norm.ppf(0.025) * se)
2 female_upper_bound= mean + (norm.ppf(0.975) * se)
3 print('95% Confidence Interval For The Average Amount Spent By Female For Sample Size 3000:', (female_lower_bound, female_upper_bound))
```

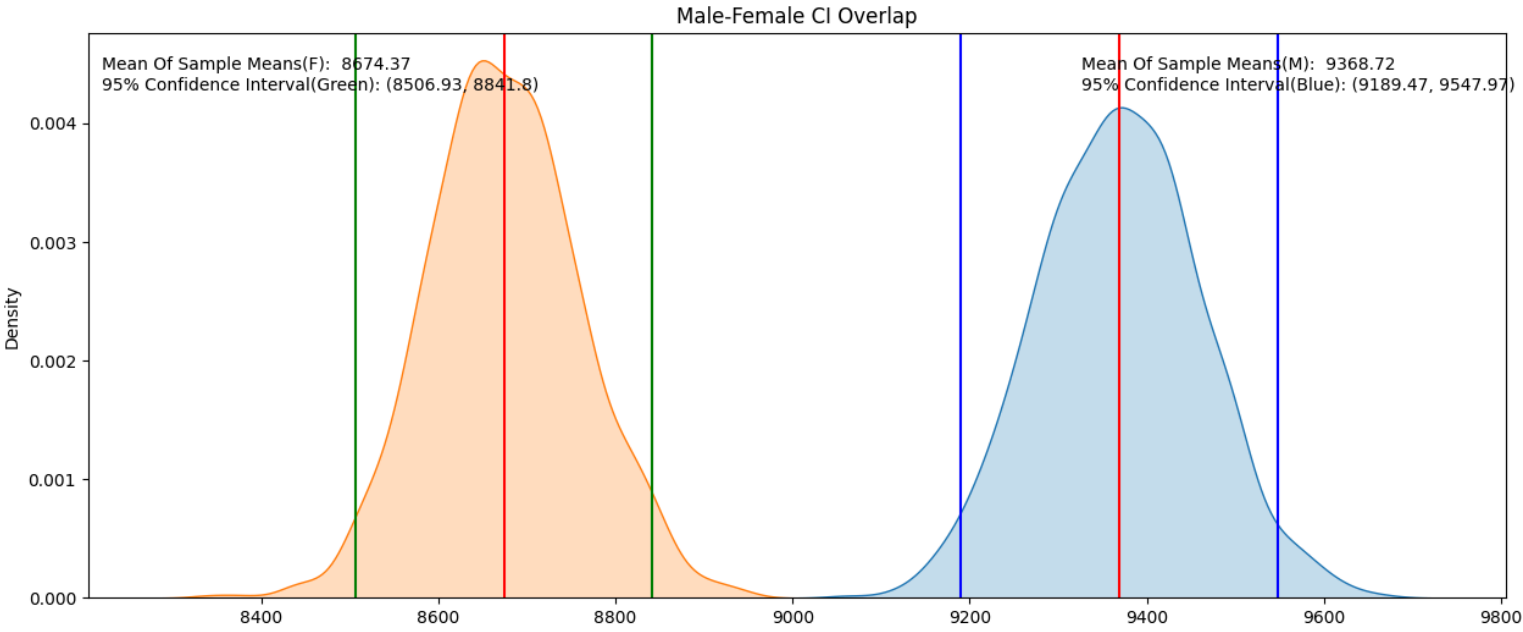
```
95% Confidence Interval For The Average Amount Spent By Female For Sample Size 3000: (8506.93367261244, 8841.80343405423)
```

```
1 plt.figure(figsize=(15,6))
2 plt.title('Female-Gender VS Purchase')
3 sns.histplot(female_sample_mean_3000, kde= True, fill= True)
4 plt.axvline(x= np.mean(female_sample_mean_3000), color='r')
5 plt.axvline(x= female_lower_bound, color='g')
6 plt.axvline(x= female_upper_bound, color='g')
7 plt.annotate(f'Mean Of Sample Means(Red): {np.mean(female_sample_mean_3000) : .2f} \n95% Confidence Interval(Green):
8             {(np.round(female_lower_bound, 2), np.round(female_upper_bound,2))}', xy=(0.05, 0.90), xycoords='axes fraction', fontsize=10)
9 plt.show()
```



Overlapping ?

```
1 plt.figure(figsize=(15,6))
2 plt.title('Male-Female CI Overlap')
3 sns.kdeplot(male_sample_mean_3000, fill= True)
4 sns.kdeplot(female_sample_mean_3000, fill= True)
5 plt.axvline(x= np.mean(male_sample_mean_3000), color='r')
6 plt.axvline(x= male_lower_bound, color='b')
7 plt.axvline(x= male_upper_bound, color='b')
8 plt.axvline(x= np.mean(female_sample_mean_3000), color='r')
9 plt.axvline(x= female_lower_bound, color='g')
10 plt.axvline(x= female_upper_bound, color='g')
11 plt.annotate(f'Mean Of Sample Means(M): {np.mean(male_sample_mean_3000) : .2f} \n95% Confidence Interval(Blue):
12             {(np.round(male_lower_bound, 2), np.round(male_upper_bound,2))}', xy=(0.70, 0.90), xycoords='axes fraction', fontsize=10)
13 plt.annotate(f'Mean Of Sample Means(F): {np.mean(female_sample_mean_3000) : .2f} \n95% Confidence Interval(Green):
14             {(np.round(female_lower_bound, 2), np.round(female_upper_bound,2))}', xy=(0.01, 0.90), xycoords='axes fraction', fontsize=10)
15
16 plt.show()
```



95% Confidence Interval For The Average Amount Spent By Male.

Sample Size: 30000

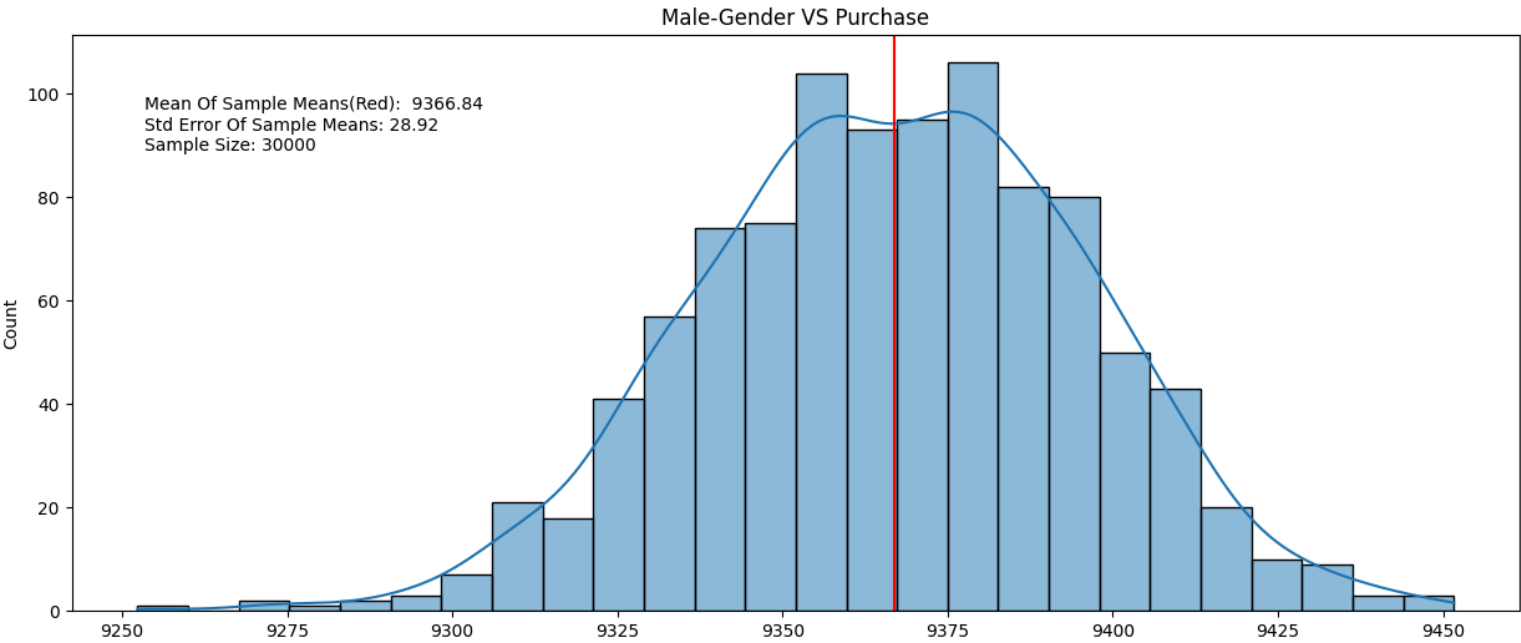
```
1 sample_size= 30000
2 Gender= 'M'
3 male_sample_mean_30000= [np.mean(data.loc[data['Gender']== Gender, 'Purchase'].sample(sample_size, replace= True)) for i in range(1000)]
4 mean= np.mean(male_sample_mean_30000)
5 se= data.loc[data['Gender']== Gender, 'Purchase'].std()/np.sqrt(sample_size)

1 print('Male Purchase Amount- Mean Of Sample Means:', mean)
2 print('Male Purchase Amount- Std Error Of Sample Means:', se)

Male Purchase Amount- Mean Of Sample Means: 9366.8404163
Male Purchase Amount- Std Error Of Sample Means: 28.92082649109867

1 plt.figure(figsize=(15,6))
2 plt.title('Male-Gender VS Purchase')
3 sns.histplot(male_sample_mean_30000, kde= True, fill= True)
4 plt.axvline(x= np.mean(male_sample_mean_30000), color='r')
5 plt.annotate(f'Mean Of Sample Means(Red): {np.mean(male_sample_mean_30000) : .2f} \nStd Error Of Sample Means:
6             {np.round(se, 2)} \nSample Size: {sample_size}', xy=(0.05, 0.80), xycoords='axes fraction', fontsize=10)
```

```
7 plt.show()
```

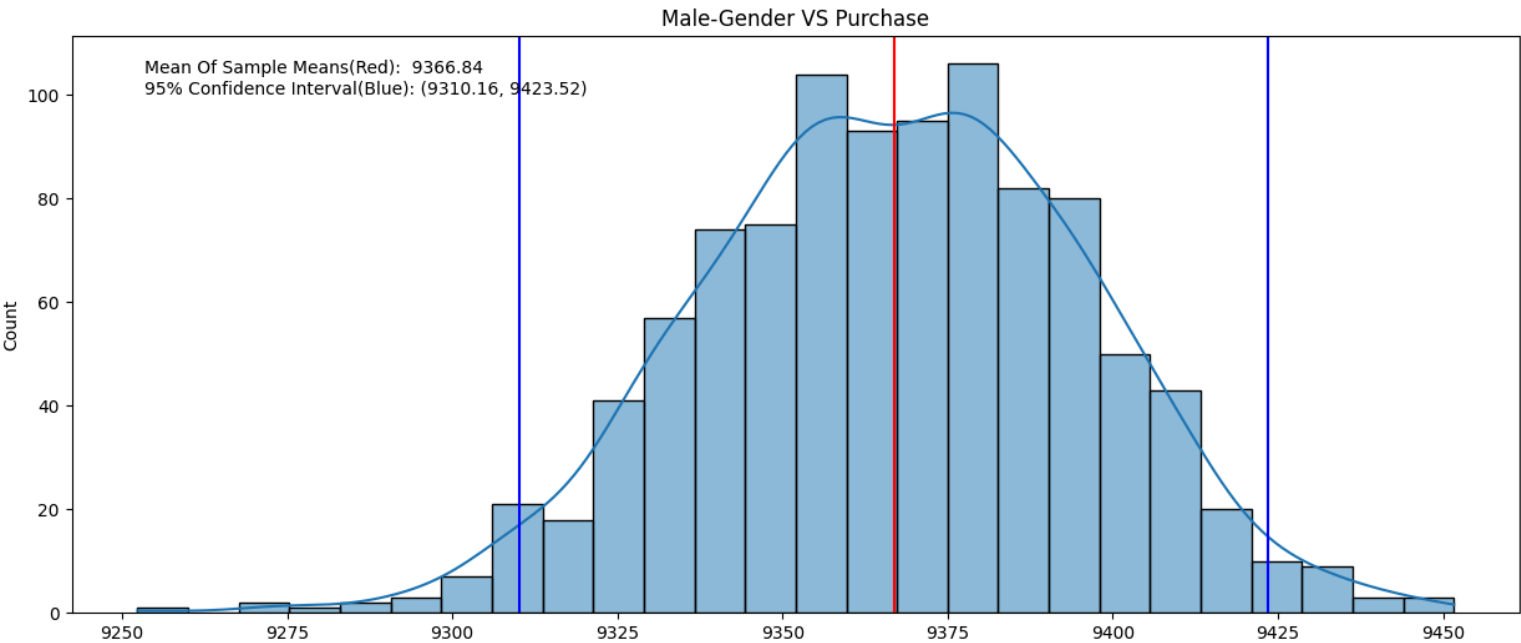


```
1 male_lower_bound= mean + (norm.ppf(0.025) * se)
2 male_upper_bound= mean + (norm.ppf(0.975) * se)
3 print('95% Confidence Interval For The Average Amount Spent By Male For Sample Size 30000:', (male_lower_bound, male_upper_bound))
```



95% Confidence Interval For The Average Amount Spent By Male For Sample Size 30000: (9310.156637974314, 9423.524194625685)

```
1 plt.figure(figsize=(15,6))
2 plt.title('Male-Gender VS Purchase')
3 sns.histplot(male_sample_mean_30000, kde= True, fill= True)
4 plt.axvline(x= np.mean(male_sample_mean_30000), color='r')
5 plt.axvline(x= male_lower_bound, color='b')
6 plt.axvline(x= male_upper_bound, color='b')
7 plt.annotate(f'Mean Of Sample Means(Red): {np.mean(male_sample_mean_30000) : .2f} \n95% Confidence Interval(Blue):
8             {(np.round(male_lower_bound, 2), np.round(male_upper_bound,2))}', xy=(0.05, 0.90), xycoords='axes fraction', fontsize=10)
9 plt.show()
```



95% Confidence Interval For The Average Amount Spent By Female.

Sample Size: 30000

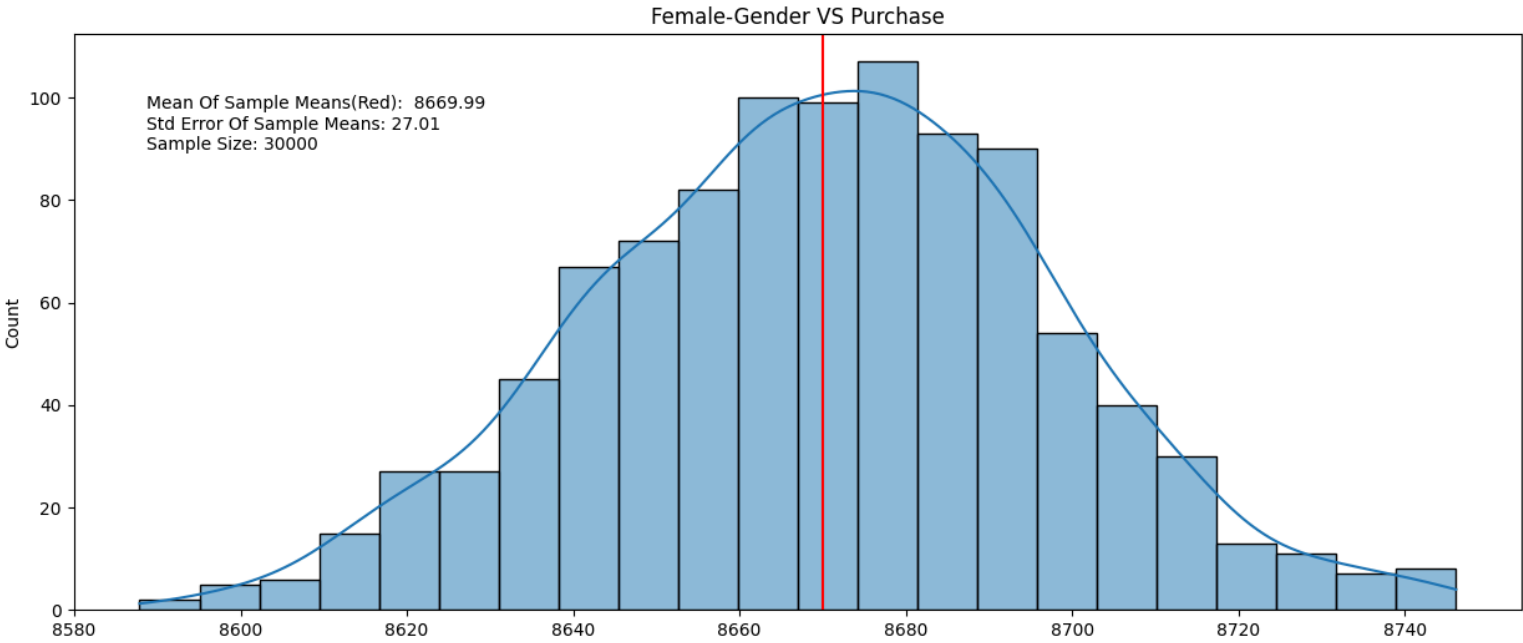
```
1 sample_size= 30000
2 Gender= 'F'
3 female_sample_mean_30000= [np.mean(data.loc[data['Gender']== Gender, 'Purchase'].sample(sample_size, replace= True)) for i in range(1000)]
4 mean= np.mean(female_sample_mean_30000)
5 se= data.loc[data['Gender']== Gender, 'Purchase'].std()/np.sqrt(sample_size)
```

```
1 print('Female Purchase Amount- Mean Of Sample Means:', mean)
2 print('Female Purchase Amount- Std Error Of Sample Means:', se)
```



Female Purchase Amount- Mean Of Sample Means: 8669.9850057
Female Purchase Amount- Std Error Of Sample Means: 27.014556747627946

```
1 plt.figure(figsize=(15,6))
2 plt.title('Female-Gender VS Purchase')
3 sns.histplot(female_sample_mean_30000, kde= True, fill= True)
4 plt.axvline(x= np.mean(female_sample_mean_30000), color='r')
5 plt.annotate(f'Mean Of Sample Means(Red): {np.mean(female_sample_mean_30000) : .2f} \nStd Error Of Sample Means:
6             {np.round(se, 2)} \nSample Size: {sample_size}', xy=(0.05, 0.80), xycoords='axes fraction', fontsize=10)
7 plt.show()
```

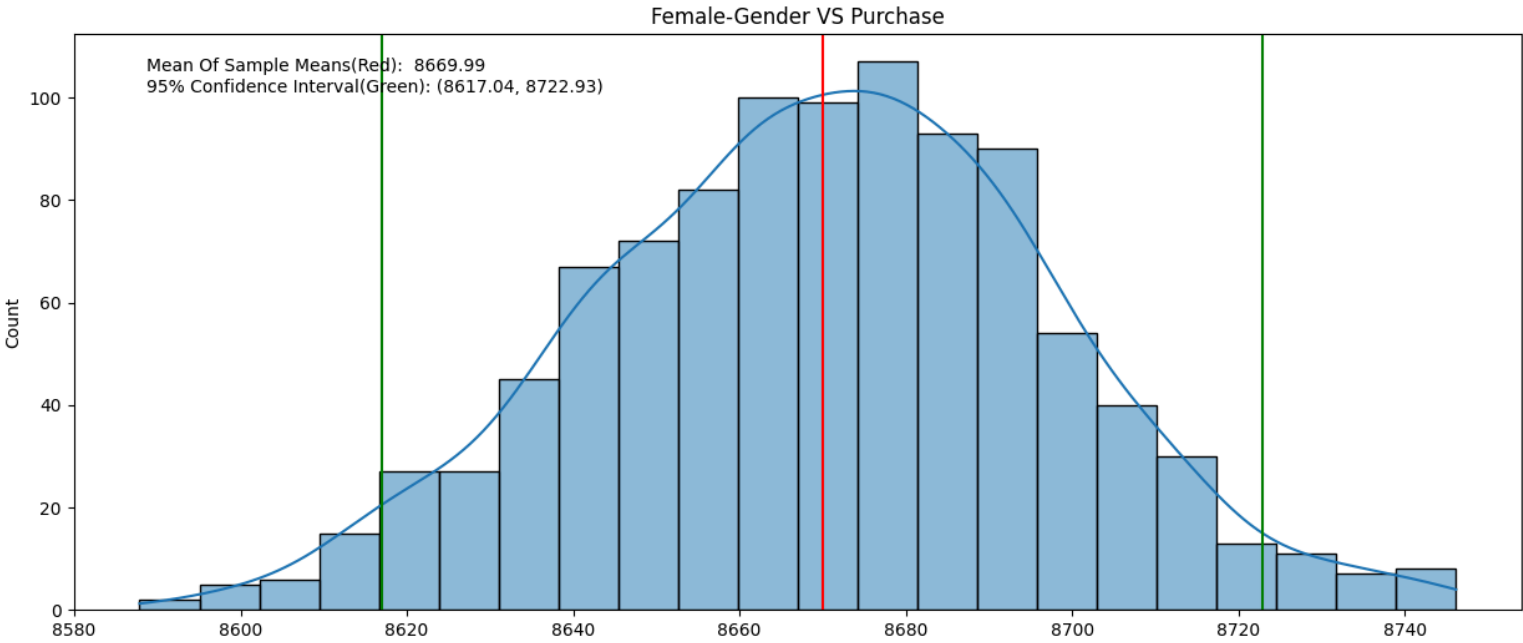


```
1 female_lower_bound= mean + (norm.ppf(0.025) * se)
2 female_upper_bound= mean + (norm.ppf(0.975) * se)
3 print('95% Confidence Interval For The Average Amount Spent By Female For Sample Size 30000:', (female_lower_bound, female_upper_bound))
```



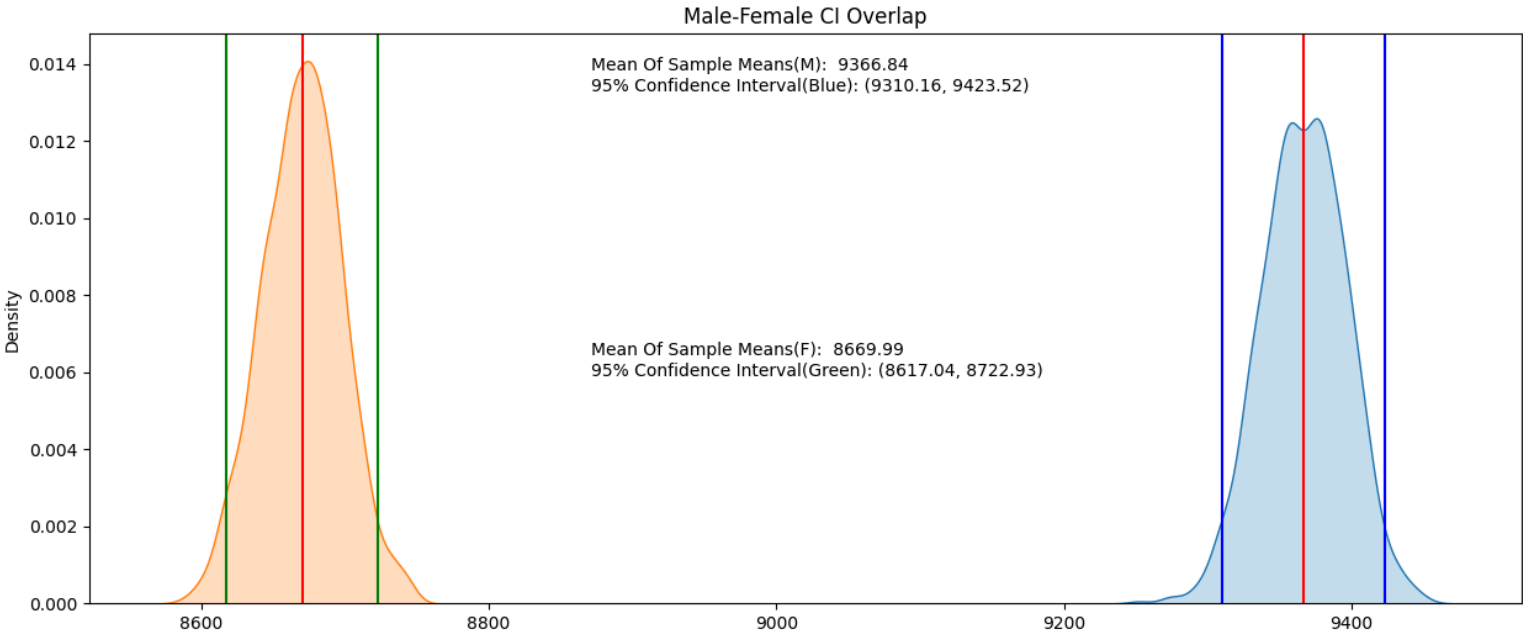
95% Confidence Interval For The Average Amount Spent By Female For Sample Size 30000: (8617.037447416336, 8722.932563983664)

```
1 plt.figure(figsize=(15,6))
2 plt.title('Female-Gender VS Purchase')
3 sns.histplot(female_sample_mean_30000, kde= True, fill= True)
4 plt.axvline(x= np.mean(female_sample_mean_30000), color='r')
5 plt.axvline(x= female_lower_bound, color='g')
6 plt.axvline(x= female_upper_bound, color='g')
7 plt.annotate(f'Mean Of Sample Means(Red): {np.mean(female_sample_mean_30000) : .2f} \n95% Confidence Interval(Green):
8             {(np.round(female_lower_bound, 2), np.round(female_upper_bound,2))}', xy=(0.05, 0.90), xycoords='axes fraction', fontsize=10)
9 plt.show()
```



Overlapping ?

```
1 plt.figure(figsize=(15,6))
2 plt.title('Male-Female CI Overlap')
3 sns.kdeplot(male_sample_mean_30000, fill= True)
4 sns.kdeplot(female_sample_mean_30000, fill= True)
5 plt.axvline(x= np.mean(male_sample_mean_30000), color='r')
6 plt.axvline(x= male_lower_bound, color='b')
7 plt.axvline(x= male_upper_bound, color='b')
8 plt.axvline(x= np.mean(female_sample_mean_30000), color='r')
9 plt.axvline(x= female_lower_bound, color='g')
10 plt.axvline(x= female_upper_bound, color='g')
11 plt.annotate(f'Mean Of Sample Means(M): {np.mean(male_sample_mean_30000) : .2f} \n95% Confidence Interval(Blue):
12             {(np.round(male_lower_bound, 2), np.round(male_upper_bound,2))}', xy=(0.35, 0.90), xycoords='axes fraction', fontsize=10)
13 plt.annotate(f'Mean Of Sample Means(F): {np.mean(female_sample_mean_30000) : .2f} \n95% Confidence Interval(Green):
14             {(np.round(female_lower_bound, 2), np.round(female_upper_bound,2))}', xy=(0.35, 0.40), xycoords='axes fraction', fontsize=10)
15
16 plt.show()
```



Conclusion: Males consistently outspend females on Black Friday, with higher average spending confirmed as sample size increases. Increasing the sample size reduces the standard error and narrows the confidence intervals, providing more precise estimates.

5. How does Marital_Status affect the amount spent?

Hint: Use the central limit theorem and bootstrapping to compute the 95% confidence intervals for the average amount spent per Marital_Status. First, compute the confidence interval for whatever data is available, and then repeat the same with smaller sample sizes - 300, 3000, and 30000.

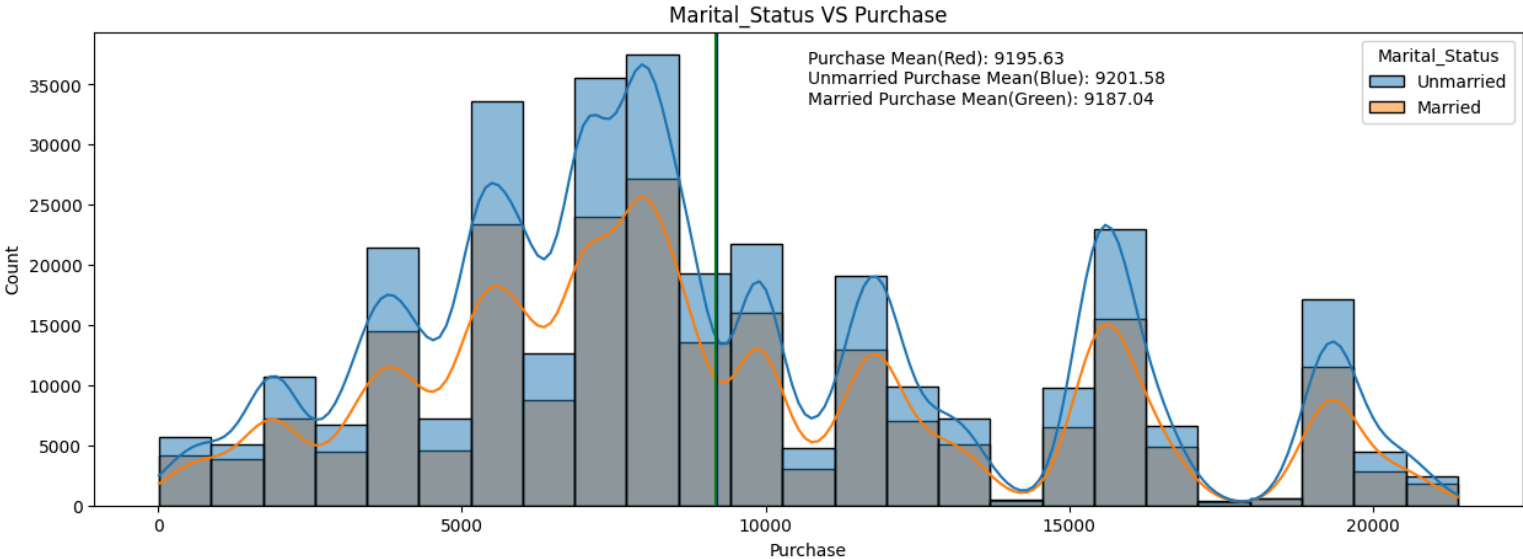
```
1 data.groupby('Marital_Status')['Purchase'].describe()
```



```
<ipython-input-76-7dd739a0ebc2>:1: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas:
data.groupby('Marital_Status')['Purchase'].describe()
```

	count	mean	std	min	25%	50%	75%	max
Marital_Status								
Unmarried	323242.0	9201.581849	4948.327397	12.0	5480.0	8035.0	12028.0	21399.0
Married	224149.0	9187.040076	4925.205232	12.0	5833.0	8042.0	12006.0	21398.0

```
1 plt.figure(figsize=(15,5))
2 sns.histplot(data, bins=25, x= 'Purchase', hue= 'Marital_Status', fill= True, kde= True)
3 plt.title('Marital_Status VS Purchase')
4 plt.axvline(x= data['Purchase'].mean(), color='r')
5 mean= data['Purchase'].mean()
6 plt.axvline(x= data[data['Marital_Status']=='Unmarried']['Purchase'].mean(), color='b')
7 um_mean= data[data['Marital_Status']=='Unmarried']['Purchase'].mean()
8 plt.axvline(x= data[data['Marital_Status']=='Married']['Purchase'].mean(), color='g')
9 mm_mean= data[data['Marital_Status']=='Married']['Purchase'].mean()
10 plt.annotate(f'Purchase Mean(Red): {np.round(mean, 2)} \nUnmarried Purchase Mean(Blue): {np.round(um_mean, 2)} \nMarried Purchase Mean(Green):
11 {np.round(mm_mean, 2)}', xy=(0.5, 0.85), xycoords='axes fraction', fontsize=10)
12 plt.show()
```



```
1 print('Unmarried Purchase Amount Mean:', data.loc[data['Marital_Status']== 'Unmarried', 'Purchase'].mean())
2 print('Unmarried Purchase Amount Std Dev:', data.loc[data['Marital_Status']== 'Unmarried', 'Purchase'].std())
3 print()
4 print('Married Purchase Amount Mean:', data.loc[data['Marital_Status']== 'Married', 'Purchase'].mean())
5 print('Married Purchase Amount Std Dev:', data.loc[data['Marital_Status']== 'Married', 'Purchase'].std())
```

Unmarried Purchase Amount Mean: 9201.581848893398
Unmarried Purchase Amount Std Dev: 4948.327397459098

Married Purchase Amount Mean: 9187.040076020861
Married Purchase Amount Std Dev: 4925.205231851271

95% Confidence Interval For The Average Amount Spent By Unmarried.

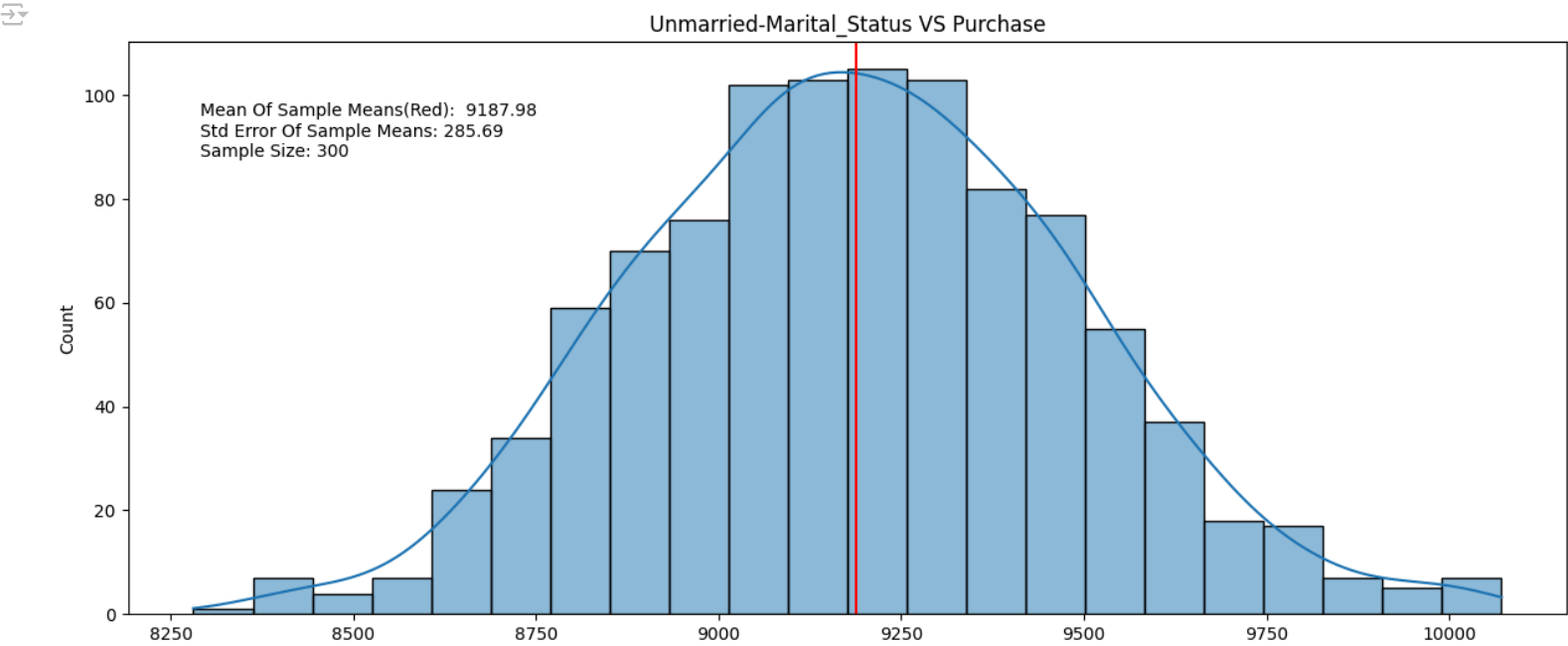
Sample Size: 300

```
1 sample_size= 300
2 Marital_Status= 'Unmarried'
3 Unmarried_sample_mean_300= [np.mean(data.loc[data['Marital_Status']== Marital_Status, 'Purchase'].sample(sample_size, replace= True)) for i in range(1000)]
4 mean= np.mean(Unmarried_sample_mean_300)
5 se= data.loc[data['Marital_Status']== Marital_Status, 'Purchase'].std()/np.sqrt(sample_size)
```

```
1 print('Unmarried Purchase Amount- Mean Of Sample Means:', mean)
2 print('Unmarried Purchase Amount- Std Error Of Sample Means:', se)
```

Unmarried Purchase Amount- Mean Of Sample Means: 9187.97647
Unmarried Purchase Amount- Std Error Of Sample Means: 285.69181549614103

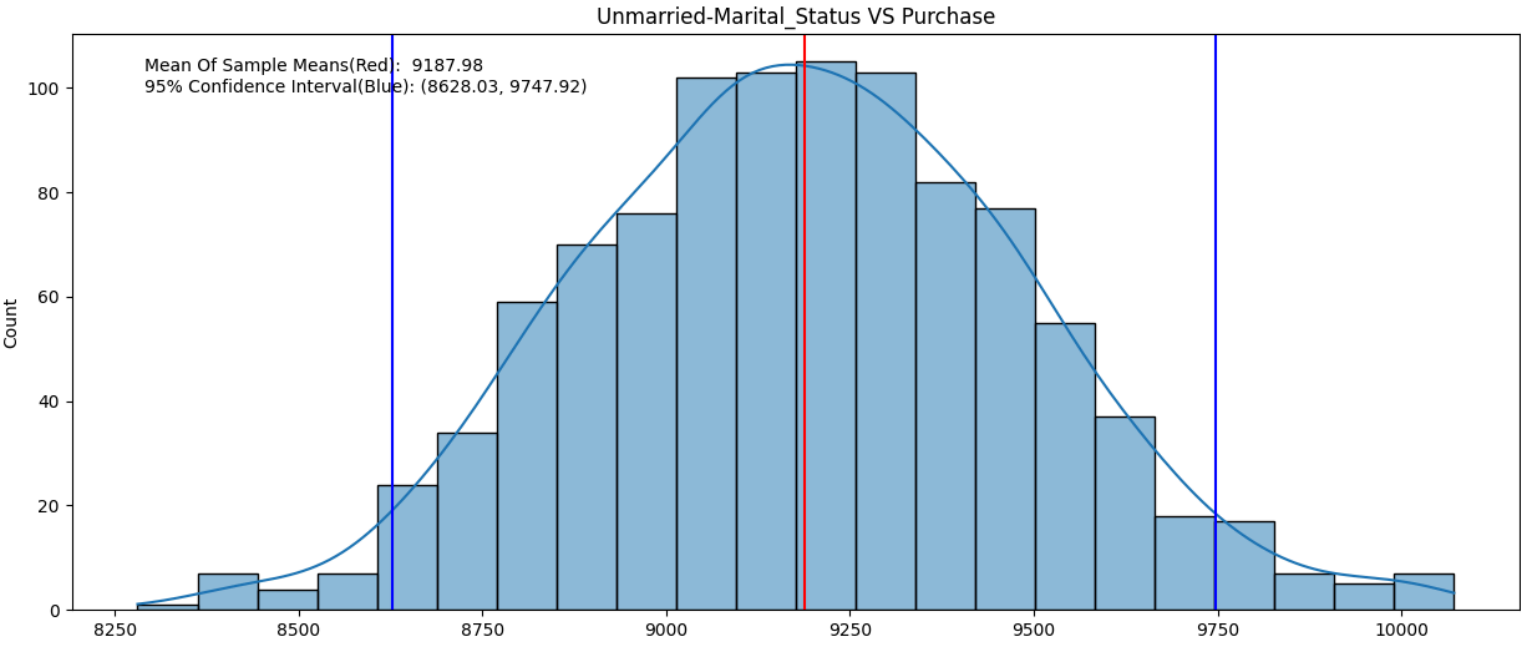
```
1 plt.figure(figsize=(15,6))
2 plt.title('Unmarried-Marital_Status VS Purchase')
3 sns.histplot(Unmarried_sample_mean_300, kde= True, fill= True)
4 plt.axvline(x= np.mean(Unmarried_sample_mean_300), color='r')
5 plt.annotate(f'Mean Of Sample Means(Red): {np.mean(Unmarried_sample_mean_300) : .2f} \nStd Error Of Sample Means:
6             {np.round(se, 2)} \nSample Size: {sample_size}', xy=(0.05, 0.80), xycoords='axes fraction', fontsize=10)
7 plt.show()
```



```
1 Unmarried_lower_bound= mean + (norm.ppf(0.025) * se)
2 Unmarried_upper_bound= mean + (norm.ppf(0.975) * se)
3 print('95% Confidence Interval For The Average Amount Spent By Unmarried For Sample Size 300:', (Unmarried_lower_bound, Unmarried_upper_bound))
```

95% Confidence Interval For The Average Amount Spent By Unmarried For Sample Size 300: (8628.0308009497, 9747.922139050299)

```
1 plt.figure(figsize=(15,6))
2 plt.title('Unmarried-Marital_Status VS Purchase')
3 sns.histplot(Unmarried_sample_mean_300, kde= True, fill= True)
4 plt.axvline(x= np.mean(Unmarried_sample_mean_300), color='r')
5 plt.axvline(x= Unmarried_lower_bound, color='b')
6 plt.axvline(x= Unmarried_upper_bound, color='b')
7 plt.annotate(f'Mean Of Sample Means(Red): {np.mean(Unmarried_sample_mean_300) : .2f} \n95% Confidence Interval(Blue):
8             {(np.round(Unmarried_lower_bound, 2), np.round(Unmarried_upper_bound,2))}', xy=(0.05, 0.90), xycoords='axes fraction', fontsize=10)
9 plt.show()
```



95% Confidence Interval For The Average Amount Spent By Married.

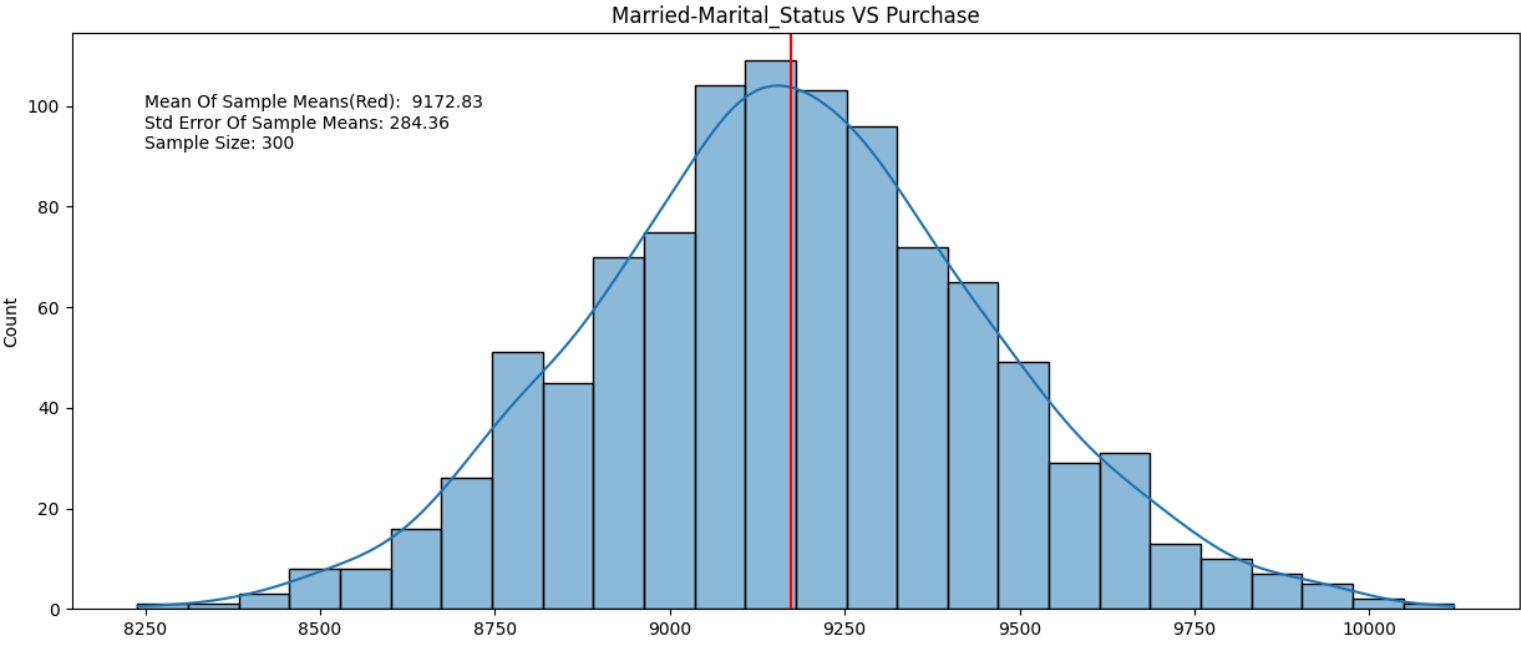
Sample Size: 300

```
1 sample_size= 300
2 Marital_Status= 'Married'
3 Married_sample_mean_300= [np.mean(data.loc[data['Marital_Status']== Marital_Status, 'Purchase'].sample(sample_size, replace= True)) for i in range(1000)]
4 mean= np.mean(Married_sample_mean_300)
5 se= data.loc[data['Marital_Status']== Marital_Status, 'Purchase'].std()/np.sqrt(sample_size)
```

```
1 print('Married Purchase Amount- Mean Of Sample Means:', mean)
2 print('Married Purchase Amount- Std Error Of Sample Means:', se)
```

```
Married Purchase Amount- Mean Of Sample Means: 9172.833166666667
Married Purchase Amount- Std Error Of Sample Means: 284.3568566423484
```

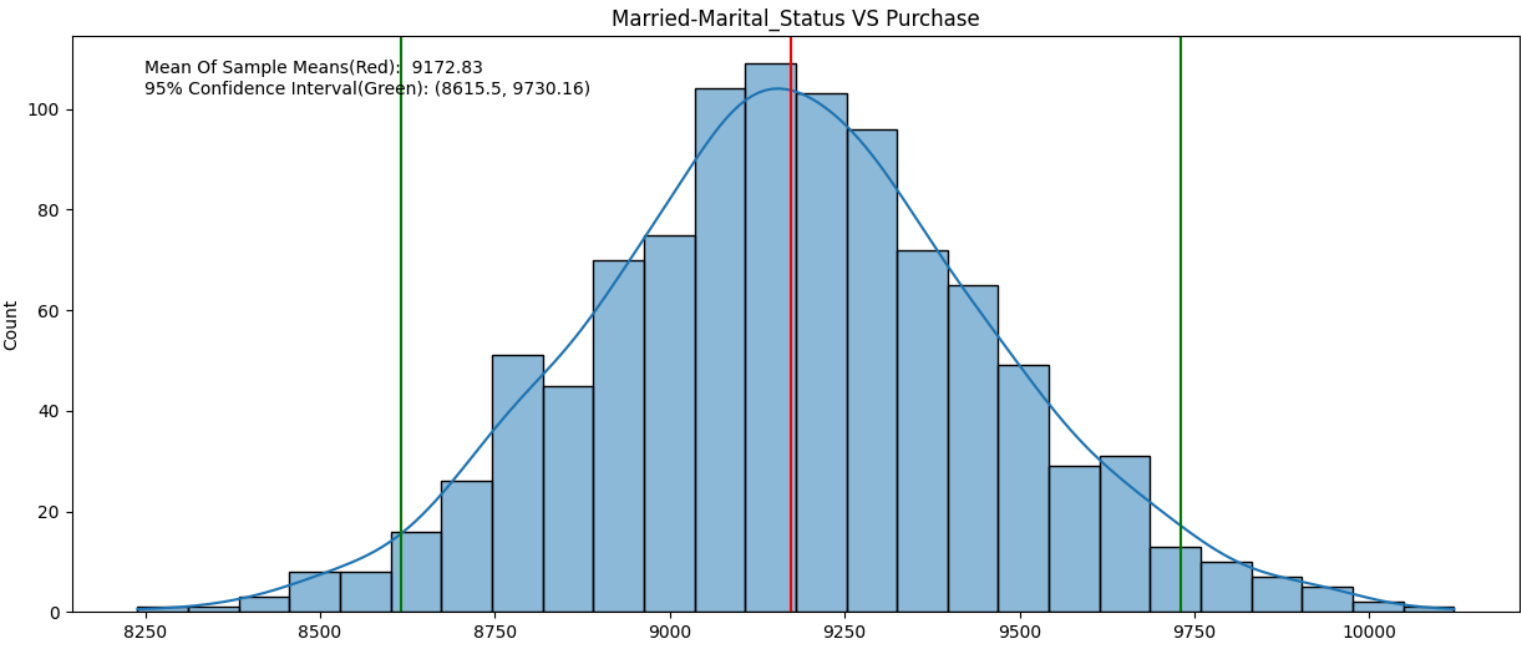
```
1 plt.figure(figsize=(15,6))
2 plt.title('Married-Marital_Status VS Purchase')
3 sns.histplot(Married_sample_mean_300, kde= True, fill= True)
4 plt.axvline(x= np.mean(Married_sample_mean_300), color='r')
5 plt.annotate(f'Mean Of Sample Means(Red): {np.mean(Married_sample_mean_300) : .2f} \nStd Error Of Sample Means:
6             {np.round(se, 2)} \nSample Size: {sample_size}', xy=(0.05, 0.80), xycoords='axes fraction', fontsize=10)
7 plt.show()
```



```
1 Married_lower_bound= mean + (norm.ppf(0.025) * se)
2 Married_upper_bound= mean + (norm.ppf(0.975) * se)
3 print('95% Confidence Interval For The Average Amount Spent By Married For Sample Size 300:', (Married_lower_bound, Married_upper_bound))
```

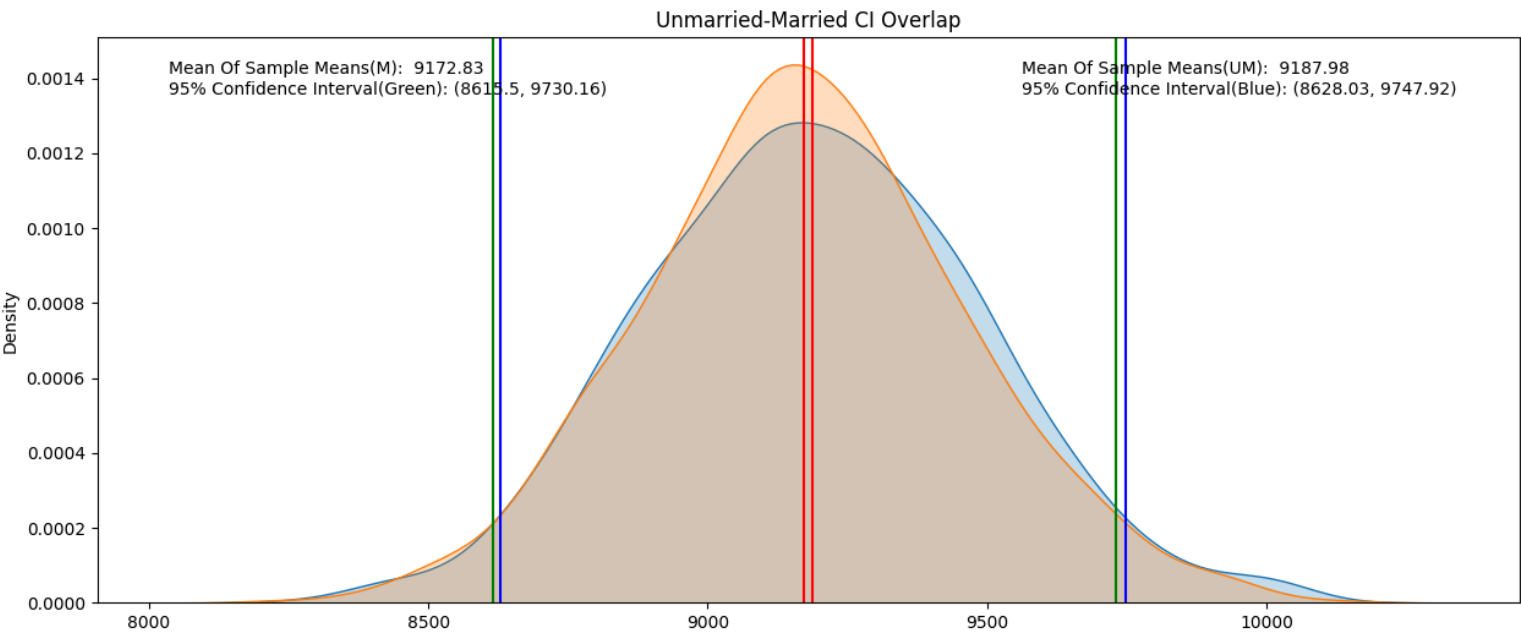
```
95% Confidence Interval For The Average Amount Spent By Married For Sample Size 300: (8615.503968890645, 9730.162364442689)
```

```
1 plt.figure(figsize=(15,6))
2 plt.title('Married-Marital_Status VS Purchase')
3 sns.histplot(Married_sample_mean_300, kde= True, fill= True)
4 plt.axvline(x= np.mean(Married_sample_mean_300), color='r')
5 plt.axvline(x= Married_lower_bound, color='g')
6 plt.axvline(x= Married_upper_bound, color='g')
7 plt.annotate(f'Mean Of Sample Means(Red): {np.mean(Married_sample_mean_300) : .2f} \n95% Confidence Interval(Green):
8             {(np.round(Married_lower_bound, 2), np.round(Married_upper_bound,2))}', xy=(0.05, 0.90), xycoords='axes fraction', fontsize=10)
9 plt.show()
```



Overlapping ?

```
1 plt.figure(figsize=(15,6))
2 plt.title('Unmarried-Married CI Overlap')
3 sns.kdeplot(Unmarried_sample_mean_300, fill= True)
4 sns.kdeplot(Married_sample_mean_300, fill= True)
5 plt.axvline(x= np.mean(Unmarried_sample_mean_300), color='r')
6 plt.axvline(x= Unmarried_lower_bound, color='b')
7 plt.axvline(x= Unmarried_upper_bound, color='b')
8 plt.axvline(x= np.mean(Married_sample_mean_300), color='r')
9 plt.axvline(x= Married_lower_bound, color='g')
10 plt.axvline(x= Married_upper_bound, color='g')
11 plt.annotate(f'Mean Of Sample Means(UM): {np.mean(Unmarried_sample_mean_300) : .2f} \n95% Confidence Interval(Blue):
12 {(np.round(Unmarried_lower_bound, 2), np.round(Unmarried_upper_bound,2))}', xy=(0.65, 0.90), xycoords='axes fraction', fontsize=10)
13 plt.annotate(f'Mean Of Sample Means(M): {np.mean(Married_sample_mean_300) : .2f} \n95% Confidence Interval(Green):
14 {(np.round(Married_lower_bound, 2), np.round(Married_upper_bound,2))}', xy=(0.05, 0.90), xycoords='axes fraction', fontsize=10)
15
16 plt.show()
```



95% Confidence Interval For The Average Amount Spent By Unmarried.

Sample Size: 3000

```
1 sample_size= 3000
2 Marital_Status= 'Unmarried'
3 Unmarried_sample_mean_3000= [np.mean(data.loc[data['Marital_Status']== Marital_Status, 'Purchase'].sample(sample_size, replace= True)) for i in range(1000)]
4 mean= np.mean(Unmarried_sample_mean_3000)
5 se= data.loc[data['Marital_Status']== Marital_Status, 'Purchase'].std()/np.sqrt(sample_size)

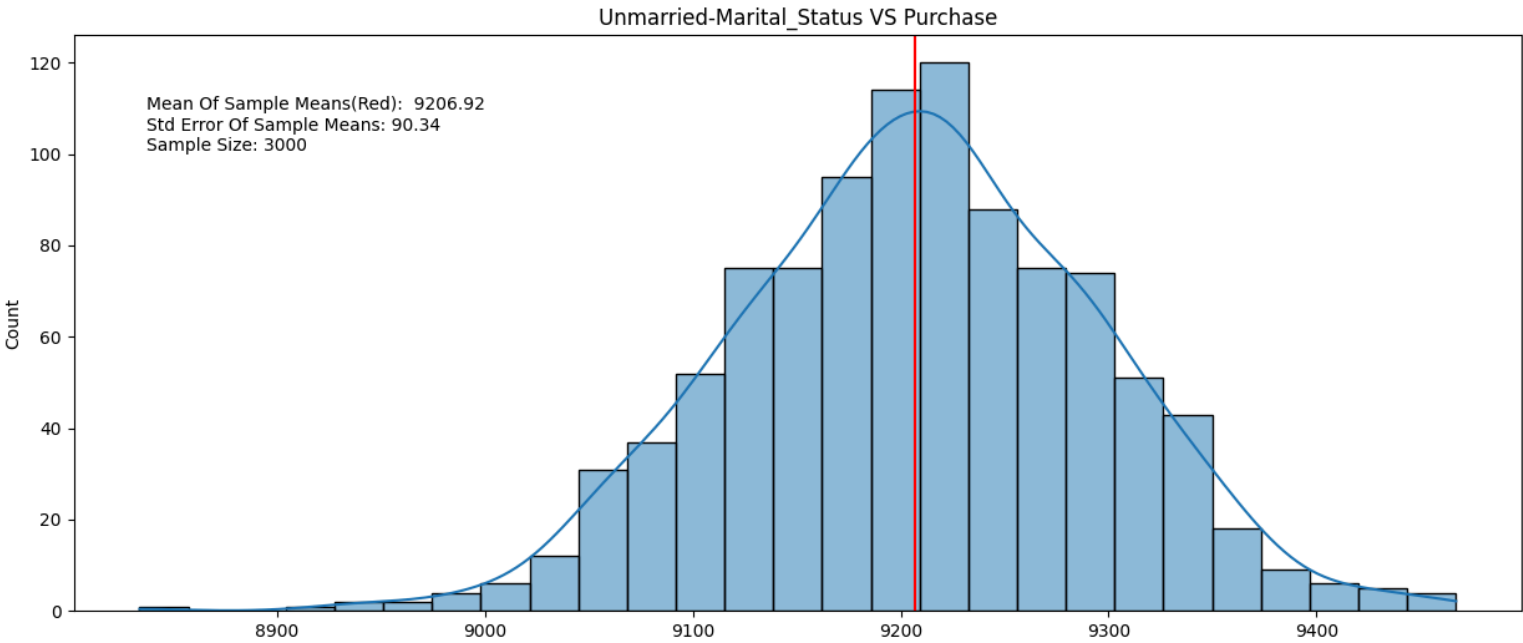
1 print('Unmarried Purchase Amount- Mean Of Sample Means:', mean)
2 print('Unmarried Purchase Amount- Std Error Of Sample Means:', se)

Unmarried Purchase Amount- Mean Of Sample Means: 9206.923052666665
Unmarried Purchase Amount- Std Error Of Sample Means: 90.34368458363933
```

```
1 plt.figure(figsize=(15,6))
2 plt.title('Unmarried-Marital_Status VS Purchase')
3 sns.histplot(Unmarried_sample_mean_3000, kde= True, fill= True)
4 plt.axvline(x= np.mean(Unmarried_sample_mean_3000), color='r')
5 plt.annotate(f'Mean Of Sample Means(Red): {np.mean(Unmarried_sample_mean_3000) : .2f} \nStd Error Of Sample Means:
```



```
6         {np.round(se, 2)} \nSample Size: {sample_size}', xy=(0.05, 0.80), xycoords='axes fraction', fontsize=10)
```

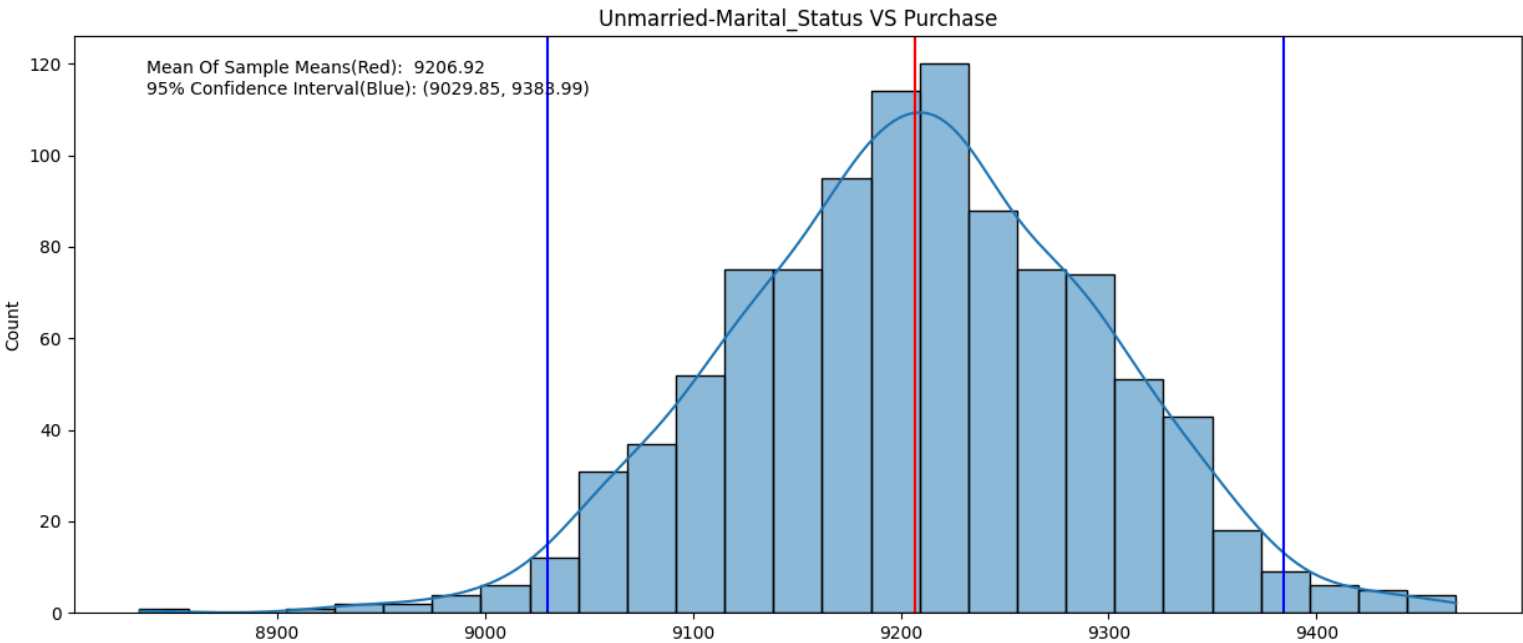


```
1 Unmarried_lower_bound= mean + (norm.ppf(0.025) * se)
2 Unmarried_upper_bound= mean + (norm.ppf(0.975) * se)
3 print('95% Confidence Interval For The Average Amount Spent By Unmarried For Sample Size 30000:', (Unmarried_lower_bound, Unmarried_upper_bound))
```



95% Confidence Interval For The Average Amount Spent By Unmarried For Sample Size 30000: (9029.852684652085, 9383.993420681245)

```
1 plt.figure(figsize=(15,6))
2 plt.title('Unmarried-Marital_Status VS Purchase')
3 sns.histplot(Unmarried_sample_mean_3000, kde= True, fill= True)
4 plt.axvline(x= np.mean(Unmarried_sample_mean_3000), color='r')
5 plt.axvline(x= Unmarried_lower_bound, color='b')
6 plt.axvline(x= Unmarried_upper_bound, color='b')
7 plt.annotate(f'Mean Of Sample Means(Red): {np.mean(Unmarried_sample_mean_3000) : .2f} \n95% Confidence Interval(Blue):
8             {(np.round(Unmarried_lower_bound, 2), np.round(Unmarried_upper_bound,2))}', xy=(0.05, 0.90), xycoords='axes fraction', fontsize=10)
9 plt.show()
```



95% Confidence Interval For The Average Amount Spent By Married.

Sample Size: 3000

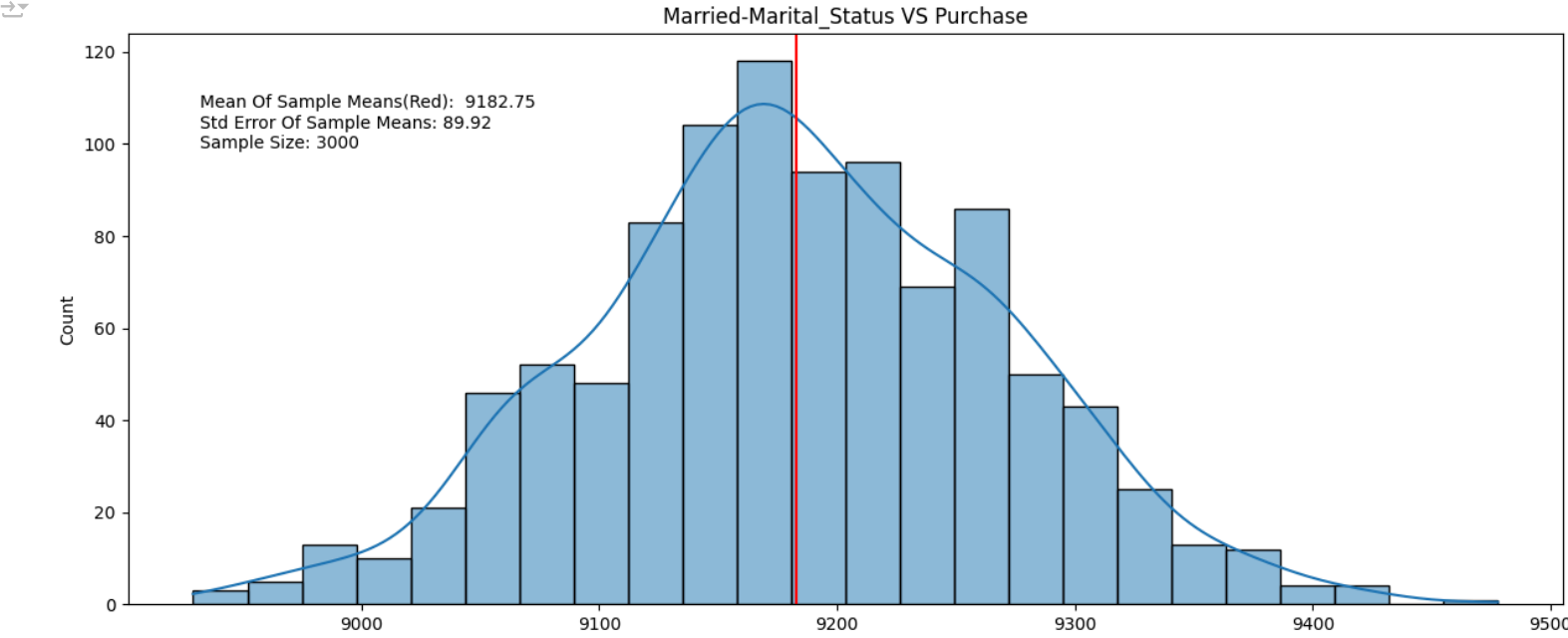
```
1 sample_size= 3000
2 Marital_Status= 'Married'
3 Married_sample_mean_3000= [np.mean(data.loc[data['Marital_Status']== Marital_Status, 'Purchase'].sample(sample_size, replace= True)) for i in range(1000)]
4 mean= np.mean(Married_sample_mean_3000)
5 se= data.loc[data['Marital_Status']== Marital_Status, 'Purchase'].std()/np.sqrt(sample_size)
```

```
1 print('Married Purchase Amount- Mean Of Sample Means:', mean)
2 print('Married Purchase Amount- Std Error Of Sample Means:', se)
```



Married Purchase Amount- Mean Of Sample Means: 9182.752427000001
Married Purchase Amount- Std Error Of Sample Means: 89.92153352758008

```
1 plt.figure(figsize=(15,6))
2 plt.title('Married-Marital_Status VS Purchase')
3 sns.histplot(Married_sample_mean_3000, kde= True, fill= True)
4 plt.axvline(x= np.mean(Married_sample_mean_3000), color='r')
5 plt.annotate(f'Mean Of Sample Means(Red): {np.mean(Married_sample_mean_3000) : .2f} \nStd Error Of Sample Means:
6             {np.round(se, 2)} \nSample Size: {sample_size}', xy=(0.05, 0.80), xycoords='axes fraction', fontsize=10)
7 plt.show()
```

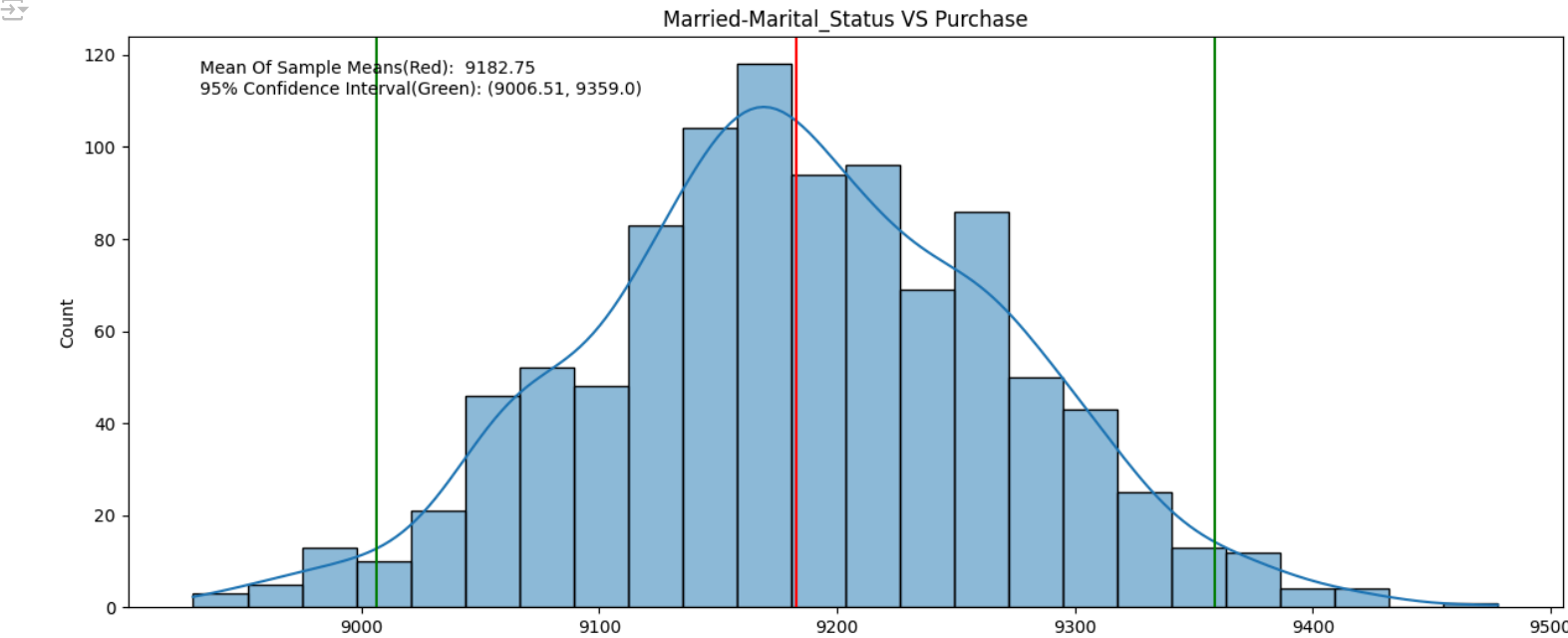


```
1 Married_lower_bound= mean + (norm.ppf(0.025) * se)
2 Married_upper_bound= mean + (norm.ppf(0.975) * se)
3 print('95% Confidence Interval For The Average Amount Spent By Married For Sample Size 3000:', (Married_lower_bound, Married_upper_bound))
```



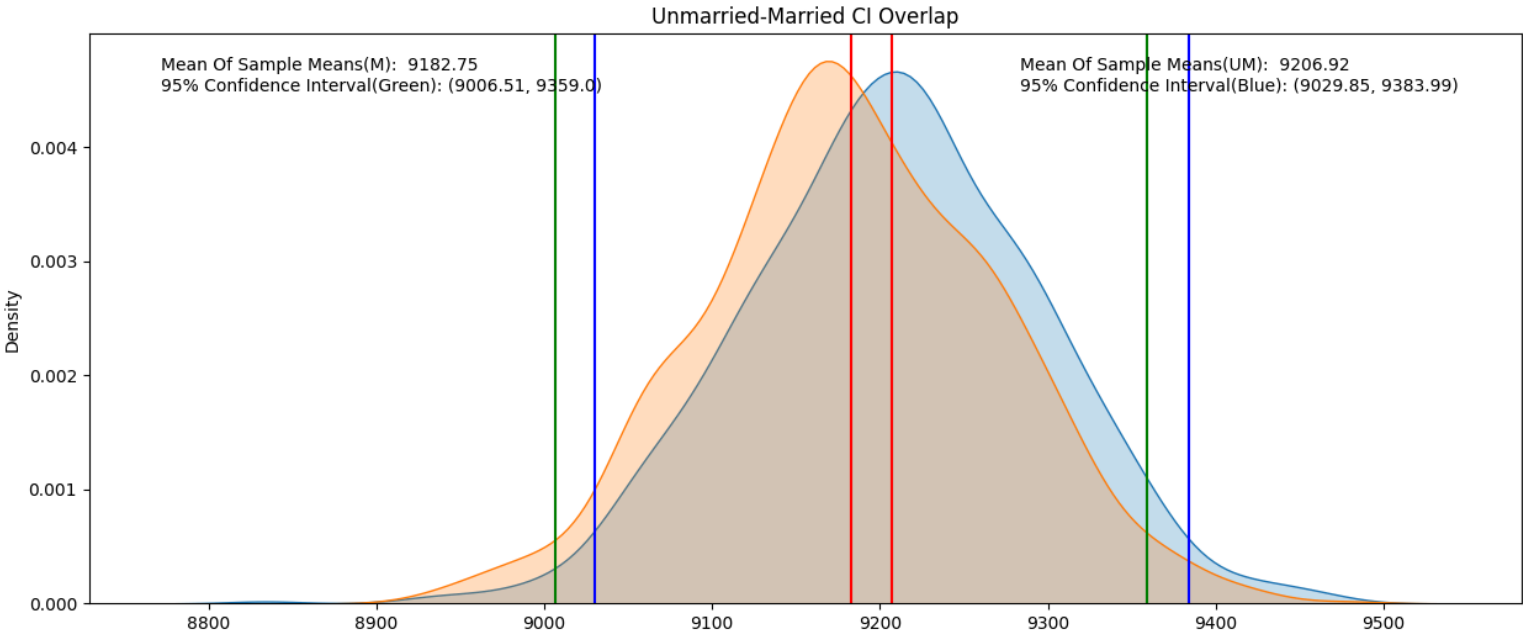
```
95% Confidence Interval For The Average Amount Spent By Married For Sample Size 3000: (9006.509459851333, 9358.99539414867)
```

```
1 plt.figure(figsize=(15,6))
2 plt.title('Married-Marital_Status VS Purchase')
3 sns.histplot(Married_sample_mean_3000, kde= True, fill= True)
4 plt.axvline(x= np.mean(Married_sample_mean_3000), color='r')
5 plt.axvline(x= Married_lower_bound, color='g')
6 plt.axvline(x= Married_upper_bound, color='g')
7 plt.annotate(f'Mean Of Sample Means(Red): {np.mean(Married_sample_mean_3000) : .2f} \n95% Confidence Interval(Green):
8             {(np.round(Married_lower_bound, 2), np.round(Married_upper_bound,2))}', xy=(0.05, 0.90), xycoords='axes fraction', fontsize=10)
9 plt.show()
```



Overlapping ?

```
1 plt.figure(figsize=(15,6))
2 plt.title('Unmarried-Married CI Overlap')
3 sns.kdeplot(Unmarried_sample_mean_3000, fill= True)
4 sns.kdeplot(Married_sample_mean_3000, fill= True)
5 plt.axvline(x= np.mean(Unmarried_sample_mean_3000), color='r')
6 plt.axvline(x= Unmarried_lower_bound, color='b')
7 plt.axvline(x= Unmarried_upper_bound, color='b')
8 plt.axvline(x= np.mean(Married_sample_mean_3000), color='r')
9 plt.axvline(x= Married_lower_bound, color='g')
10 plt.axvline(x= Married_upper_bound, color='g')
11 plt.annotate(f'Mean Of Sample Means(UM): {np.mean(Unmarried_sample_mean_3000) : .2f} \n95% Confidence Interval(Blue):
12             {(np.round(Unmarried_lower_bound, 2), np.round(Unmarried_upper_bound,2))}', xy=(0.65, 0.90), xycoords='axes fraction', fontsize=10)
13 plt.annotate(f'Mean Of Sample Means(M): {np.mean(Married_sample_mean_3000) : .2f} \n95% Confidence Interval(Green):
14             {(np.round(Married_lower_bound, 2), np.round(Married_upper_bound,2))}', xy=(0.05, 0.90), xycoords='axes fraction', fontsize=10)
15
16 plt.show()
```



95% Confidence Interval For The Average Amount Spent By Unmarried.

Sample Size: 30000

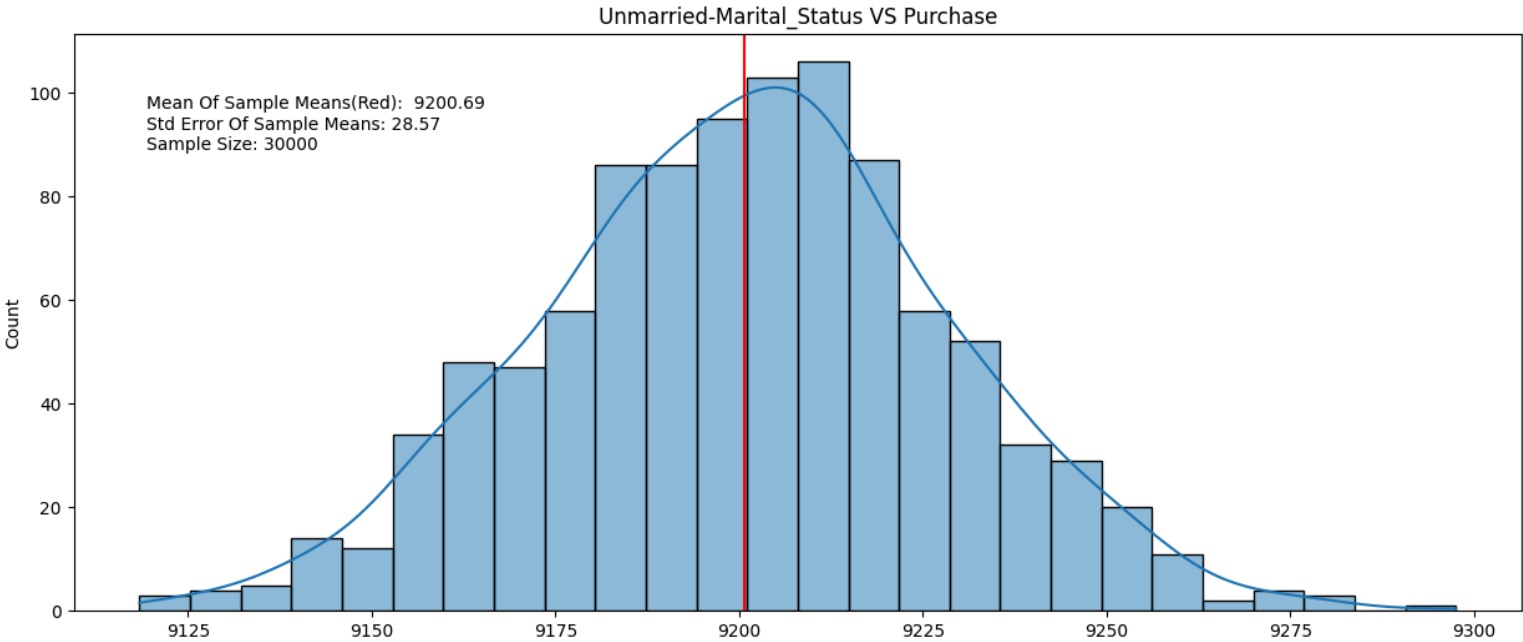
```
1 sample_size= 30000
2 Marital_Status= 'Unmarried'
3 Unmarried_sample_mean_30000= [np.mean(data.loc[data['Marital_Status']== Marital_Status, 'Purchase'].sample(sample_size, replace= True)) for i in
4                               range(1000)]
5 mean= np.mean(Unmarried_sample_mean_30000)
6 se= data.loc[data['Marital_Status']== Marital_Status, 'Purchase'].std()/np.sqrt(sample_size)

1 print('Unmarried Purchase Amount- Mean Of Sample Means:', mean)
2 print('Unmarried Purchase Amount- Std Error Of Sample Means:', se)
```



Unmarried Purchase Amount- Mean Of Sample Means: 9200.686519699999
Unmarried Purchase Amount- Std Error Of Sample Means: 28.569181549614107

```
1 plt.figure(figsize=(15,6))
2 plt.title('Unmarried-Marital_Status VS Purchase')
3 sns.histplot(Unmarried_sample_mean_30000, kde= True, fill= True)
4 plt.axvline(x= np.mean(Unmarried_sample_mean_30000), color='r')
5 plt.annotate(f'Mean Of Sample Means(Red): {np.mean(Unmarried_sample_mean_30000) : .2f} \nStd Error Of Sample Means:
6             {np.round(se, 2)} \nSample Size: {sample_size}', xy=(0.05, 0.80), xycoords='axes fraction', fontsize=10)
7 plt.show()
```

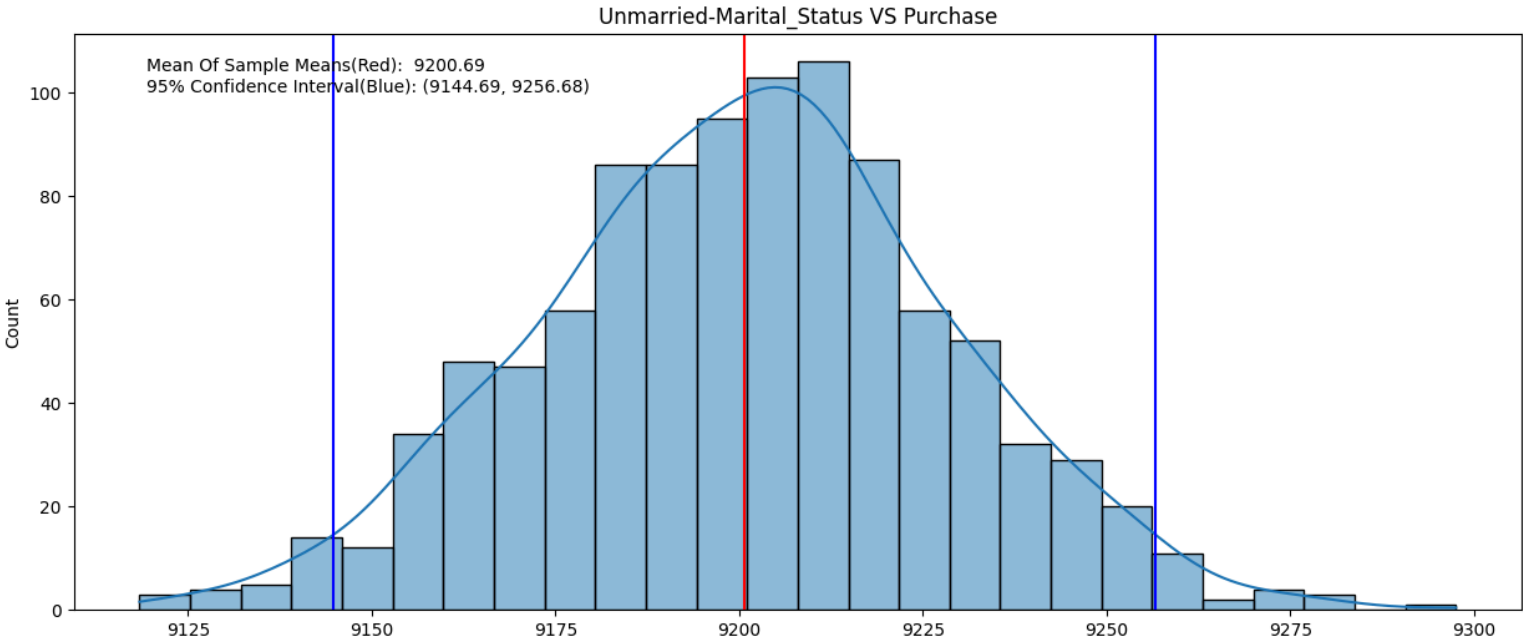


```
1 Unmarried_lower_bound= mean + (norm.ppf(0.025) * se)
2 Unmarried_upper_bound= mean + (norm.ppf(0.975) * se)
3 print('95% Confidence Interval For The Average Amount Spent By Unmarried For Sample Size 30000:', (Unmarried_lower_bound, Unmarried_upper_bound))
```



95% Confidence Interval For The Average Amount Spent By Unmarried For Sample Size 30000: (9144.691952794969, 9256.68108660503)

```
1 plt.figure(figsize=(15,6))
2 plt.title('Unmarried-Marital_Status VS Purchase')
3 sns.histplot(Unmarried_sample_mean_30000, kde= True, fill= True)
4 plt.axvline(x= np.mean(Unmarried_sample_mean_30000), color='r')
5 plt.axvline(x= Unmarried_lower_bound, color='b')
6 plt.axvline(x= Unmarried_upper_bound, color='b')
7 plt.annotate(f'Mean Of Sample Means(Red): {np.mean(Unmarried_sample_mean_30000) : .2f} \n95% Confidence Interval(Blue):
8             {(np.round(Unmarried_lower_bound, 2), np.round(Unmarried_upper_bound,2))}', xy=(0.05, 0.90), xycoords='axes fraction', fontsize=10)
9 plt.show()
```



95% Confidence Interval For The Average Amount Spent By Married.

Sample Size: 30000

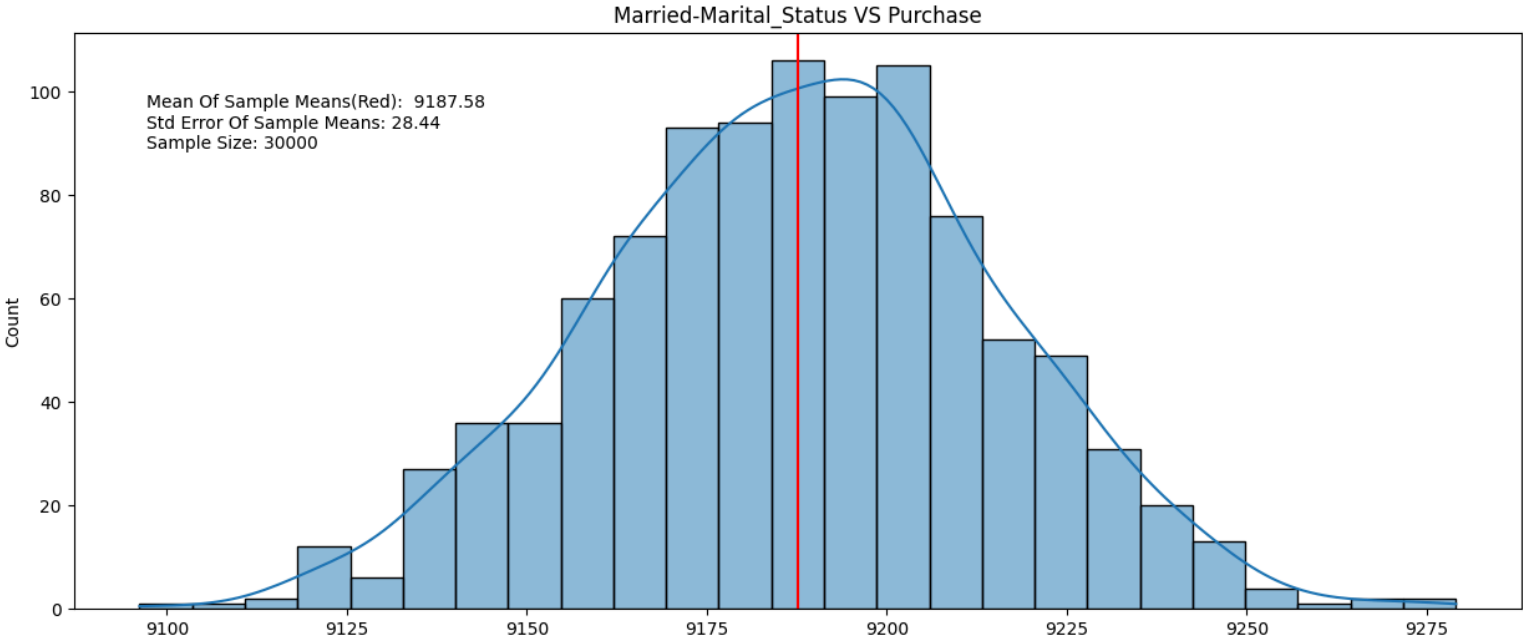
```
1 sample_size= 30000
2 Marital_Status= 'Married'
3 Married_sample_mean_30000= [np.mean(data.loc[data['Marital_Status']== Marital_Status, 'Purchase'].sample(sample_size, replace= True)) for i in range(10000)]
4 mean= np.mean(Married_sample_mean_30000)
5 se= data.loc[data['Marital_Status']== Marital_Status, 'Purchase'].std()/np.sqrt(sample_size)
```

```
1 print('Married Purchase Amount- Mean Of Sample Means:', mean)
2 print('Married Purchase Amount- Std Error Of Sample Means:', se)
```



Married Purchase Amount- Mean Of Sample Means: 9187.5834651
Married Purchase Amount- Std Error Of Sample Means: 28.435685664234843

```
1 plt.figure(figsize=(15,6))
2 plt.title('Married-Marital_Status VS Purchase')
3 sns.histplot(Married_sample_mean_30000, kde= True, fill= True)
4 plt.axvline(x= np.mean(Married_sample_mean_30000), color='r')
5 plt.annotate(f'Mean Of Sample Means(Red): {np.mean(Married_sample_mean_30000) : .2f} \nStd Error Of Sample Means: {np.round(se, 2)} \nSample Size: {sample_size}', xy=(0.05, 0.80), xycoords='axes fraction', fontsize=10)
6
7 plt.show()
```

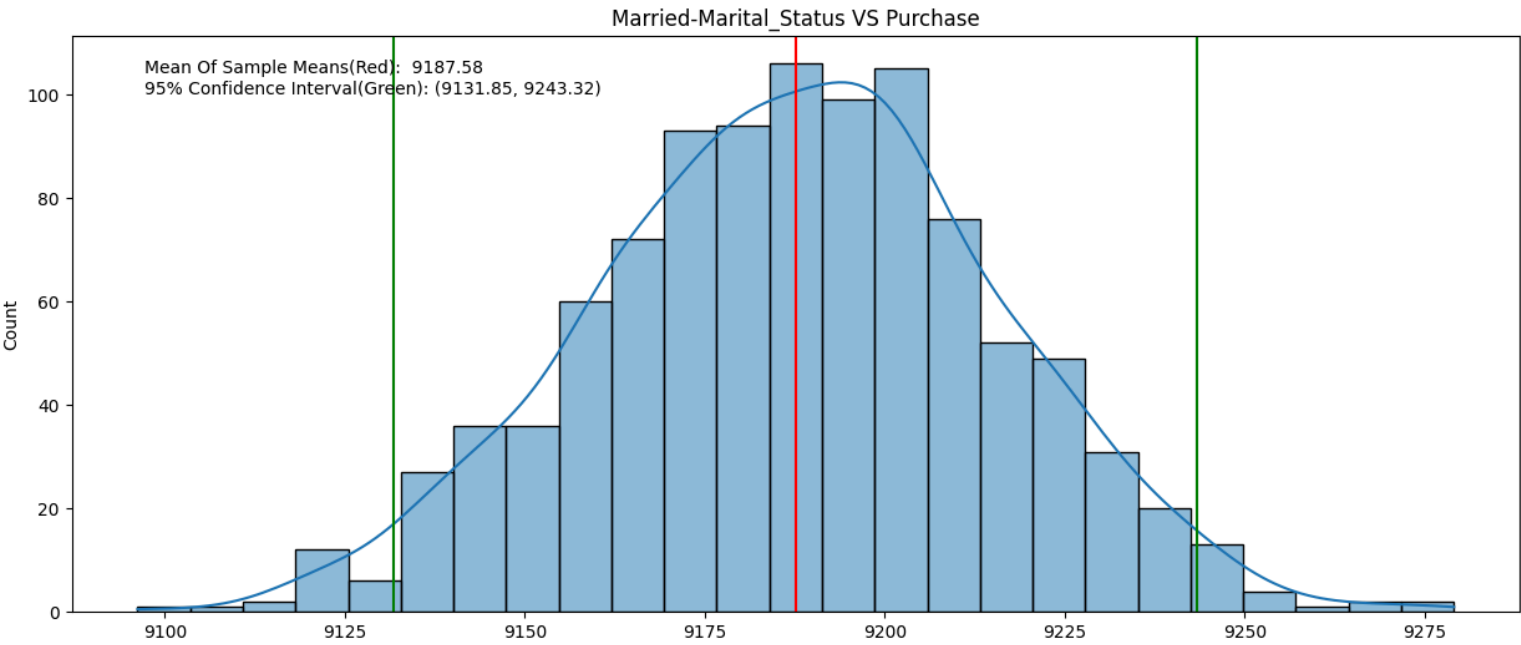


```
1 Married_lower_bound= mean + (norm.ppf(0.025) * se)
2 Married_upper_bound= mean + (norm.ppf(0.975) * se)
3 print('95% Confidence Interval For The Average Amount Spent By Married For Sample Size 30000:', (Married_lower_bound, Married_upper_bound))
```



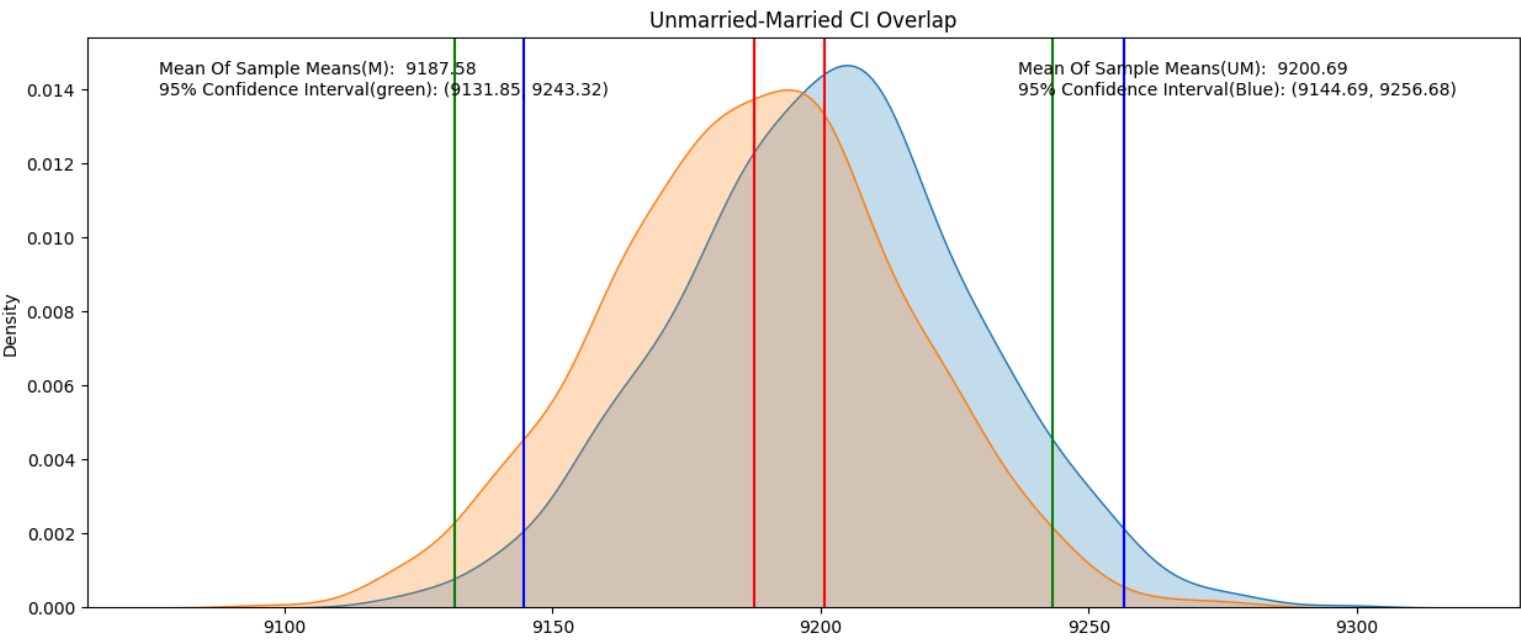
95% Confidence Interval For The Average Amount Spent By Married For Sample Size 30000: (9131.850545322399, 9243.316384877602)

```
1 plt.figure(figsize=(15,6))
2 plt.title('Married-Marital_Status VS Purchase')
3 sns.histplot(Married_sample_mean_30000, kde= True, fill= True)
4 plt.axvline(x= np.mean(Married_sample_mean_30000), color='r')
5 plt.axvline(x= Married_lower_bound, color='g')
6 plt.axvline(x= Married_upper_bound, color='g')
7 plt.annotate(f'Mean Of Sample Means(Red): {np.mean(Married_sample_mean_30000) : .2f} \n95% Confidence Interval(Green): {(np.round(Married_lower_bound, 2), np.round(Married_upper_bound,2))}', xy=(0.05, 0.90), xycoords='axes fraction', fontsize=10)
8
9 plt.show()
```



Overlapping ?

```
1 plt.figure(figsize=(15,6))
2 plt.title('Unmarried-Married CI Overlap')
3 sns.kdeplot(Unmarried_sample_mean_30000, fill= True)
4 sns.kdeplot(Married_sample_mean_30000, fill= True)
5 plt.axvline(x= np.mean(Unmarried_sample_mean_30000), color='r')
6 plt.axvline(x= Unmarried_lower_bound, color='b')
7 plt.axvline(x= Unmarried_upper_bound, color='b')
8 plt.axvline(x= np.mean(Married_sample_mean_30000), color='r')
9 plt.axvline(x= Married_lower_bound, color='g')
10 plt.axvline(x= Married_upper_bound, color='g')
11 plt.annotate(f'Mean Of Sample Means(UM): {np.mean(Unmarried_sample_mean_30000)} : .2f} \n95% Confidence Interval(Blue):
12 {(np.round(Unmarried_lower_bound, 2), np.round(Unmarried_upper_bound,2))}', xy=(0.65, 0.90), xycoords='axes fraction', fontsize=10)
13 plt.annotate(f'Mean Of Sample Means(M): {np.mean(Married_sample_mean_30000)} : .2f} \n95% Confidence Interval(green):
14 {(np.round(Married_lower_bound, 2), np.round(Married_upper_bound,2))}', xy=(0.05, 0.90), xycoords='axes fraction', fontsize=10)
15
16 plt.show()
```



Conclusion: Unmarried and married individuals spend nearly the same amount, with negligible differences in their average spending patterns. Confidence intervals consistently overlap, even as sample size increases, confirming no significant distinction between the groups.

6. How does Age affect the amount spent?

Hint: Use the central limit theorem or bootstrapping to compute the 95% confidence intervals for the average amount spent per Age Cat. First, compute the confidence interval for whatever data is available, and then repeat the same with smaller sample sizes - 300, 3000, and 30000.

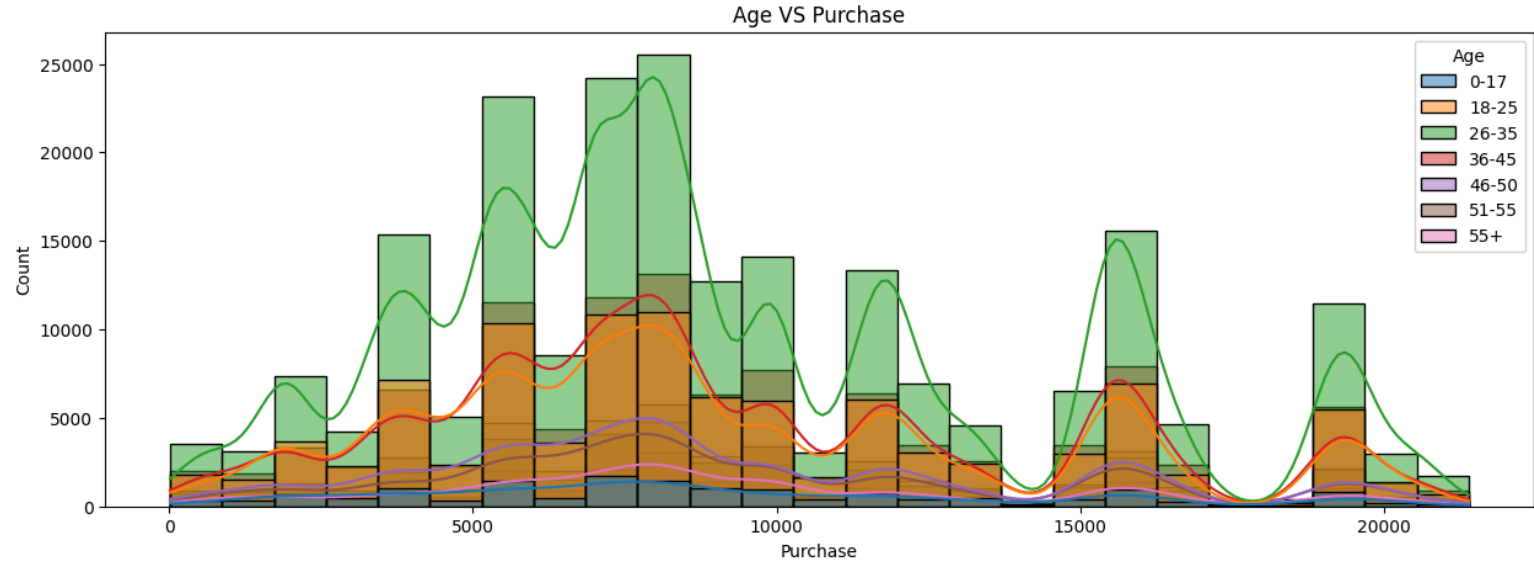
```
1 data.groupby('Age')['Purchase'].describe()
```

```
<ipython-input-112-864c5c92db2c>:1: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas
data.groupby('Age')['Purchase'].describe()
```

	count	mean	std	min	25%	50%	75%	max
Age								
0-17	15032.0	8867.447046	5030.052846	12.0	5324.0	7974.5	11833.25	21342.0
18-25	99334.0	9124.031731	4978.831062	12.0	5412.0	8020.0	12004.00	21398.0
26-35	218661.0	9193.469924	4937.410901	12.0	5471.0	8021.0	12018.00	21398.0
36-45	109409.0	9254.202214	4927.744433	12.0	5866.0	8051.0	12065.00	21399.0
46-50	45442.0	9128.985080	4867.413951	12.0	5879.0	8025.0	11958.00	21391.0
51-55	38191.0	9423.121704	4953.644650	12.0	6007.0	8118.0	12123.00	21388.0
55+	21322.0	9216.650220	4861.626596	12.0	6007.0	8092.5	11837.75	21345.0

```
1 plt.figure(figsize=(15,5))
2 sns.histplot(data, bins=25, x= 'Purchase', hue= 'Age', fill= True, kde= True)
3 plt.title('Age VS Purchase')
```

```
Text(0.5, 1.0, 'Age VS Purchase')
```



95% Confidence Interval For The Average Amount Spent By Age Category.

Sample Size: 300

```
1 sample_size= 300
2 age_cat= ['0-17', '18-25', '26-35', '36-45', '46-50', '51-55', '55+']

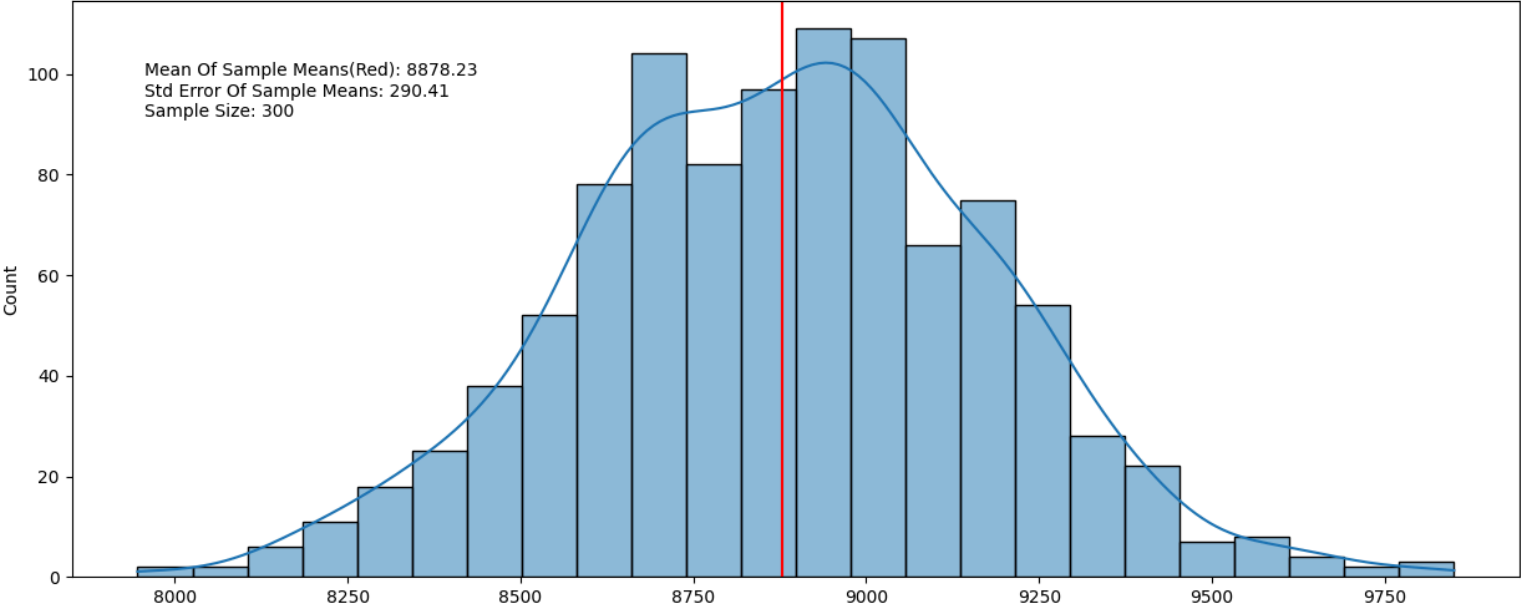
1 age_sample_means={}
2 age_means={}
3 age_se={}
4 for i in age_cat:
5     age_sample_means[f'{i}_sample_mean_{sample_size}'] = [np.mean(data.loc[data['Age']== i, 'Purchase'].sample(sample_size, replace= True))
6                                                             for _ in range(1000)]
7     age_means[f'{i}_mean_{sample_size}'] = np.mean(age_sample_means[f'{i}_sample_mean_{sample_size}'])
8     age_se[f'{i}_se_{sample_size}']= data.loc[data['Age']== i, 'Purchase'].std()/np.sqrt(sample_size)
9     print(f'{i} Purchase Amount- Mean Of Sample Means:', age_means[f'{i}_mean_{sample_size}'])
10    print(f'{i} Purchase Amount- Std Error Of Sample Means:', age_se[f'{i}_se_{sample_size}'])
11    print('-'*100)
```

```
0-17 Purchase Amount- Mean Of Sample Means: 8878.225433333333
0-17 Purchase Amount- Std Error Of Sample Means: 290.41023646300073
-----
18-25 Purchase Amount- Mean Of Sample Means: 9122.511129999999
18-25 Purchase Amount- Std Error Of Sample Means: 287.4529453833825
-----
26-35 Purchase Amount- Mean Of Sample Means: 9195.46614
26-35 Purchase Amount- Std Error Of Sample Means: 285.06155129664086
-----
36-45 Purchase Amount- Mean Of Sample Means: 9258.750153333332
36-45 Purchase Amount- Std Error Of Sample Means: 284.5034575042815
-----
46-50 Purchase Amount- Mean Of Sample Means: 9139.064263333332
46-50 Purchase Amount- Std Error Of Sample Means: 281.0202754959616
-----
51-55 Purchase Amount- Mean Of Sample Means: 9427.776003333334
51-55 Purchase Amount- Std Error Of Sample Means: 285.9988072070772
-----
55+ Purchase Amount- Mean Of Sample Means: 9209.10426
55+ Purchase Amount- Std Error Of Sample Means: 280.6861423932631
-----
```

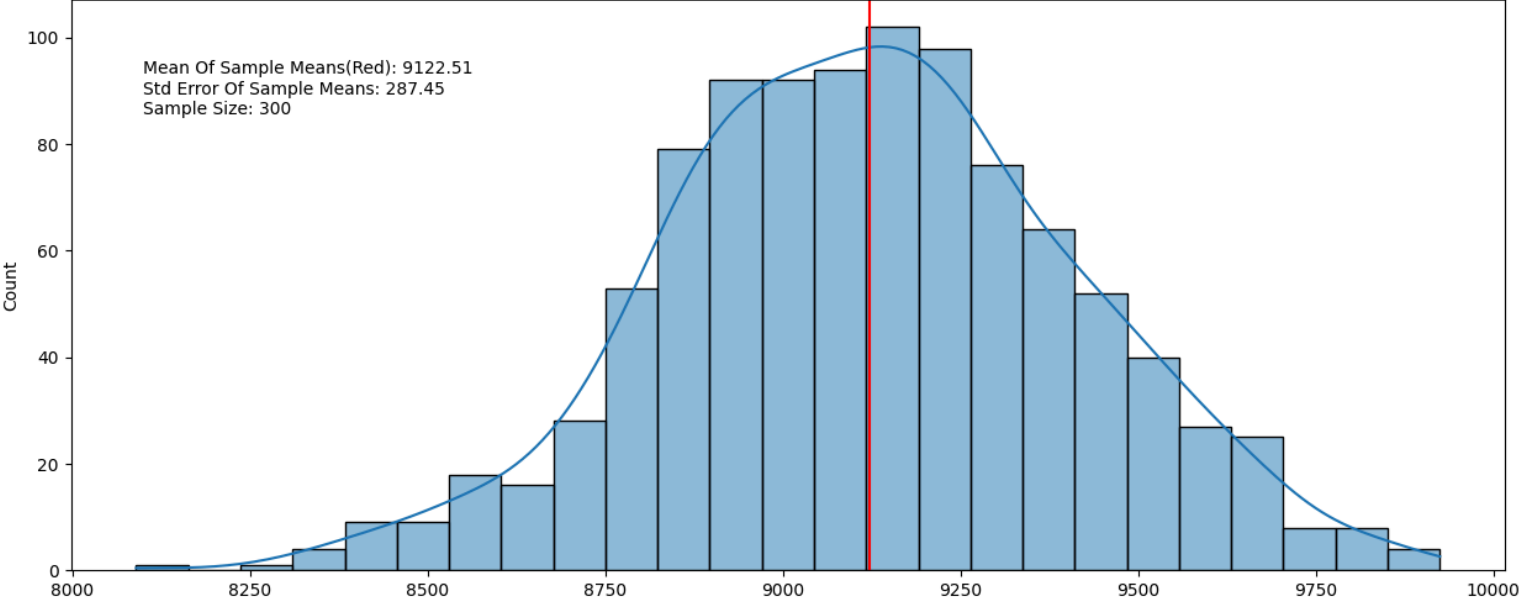
```
1 for i in age_cat:
2     plt.figure(figsize=(15,6))
3     plt.title(f'{i}-Age VS Purchase')
4     sns.histplot(age_sample_means[f'{i}_sample_mean_{sample_size}'], kde= True, fill= True)
5     plt.axvline(x= age_means[f'{i}_mean_{sample_size}'], color='r')
6     temp_mean= age_means[f'{i}_mean_{sample_size}']
7     temp_se= age_se[f'{i}_se_{sample_size}']
8     plt.annotate(f'Mean Of Sample Means(Red): {np.round(temp_mean, 2)} \nStd Error Of Sample Means: {np.round(temp_se, 2)}
9                    \nSample Size: {sample_size}', xy=(0.05, 0.80), xycoords='axes fraction', fontsize=10)
10    plt.show()
```



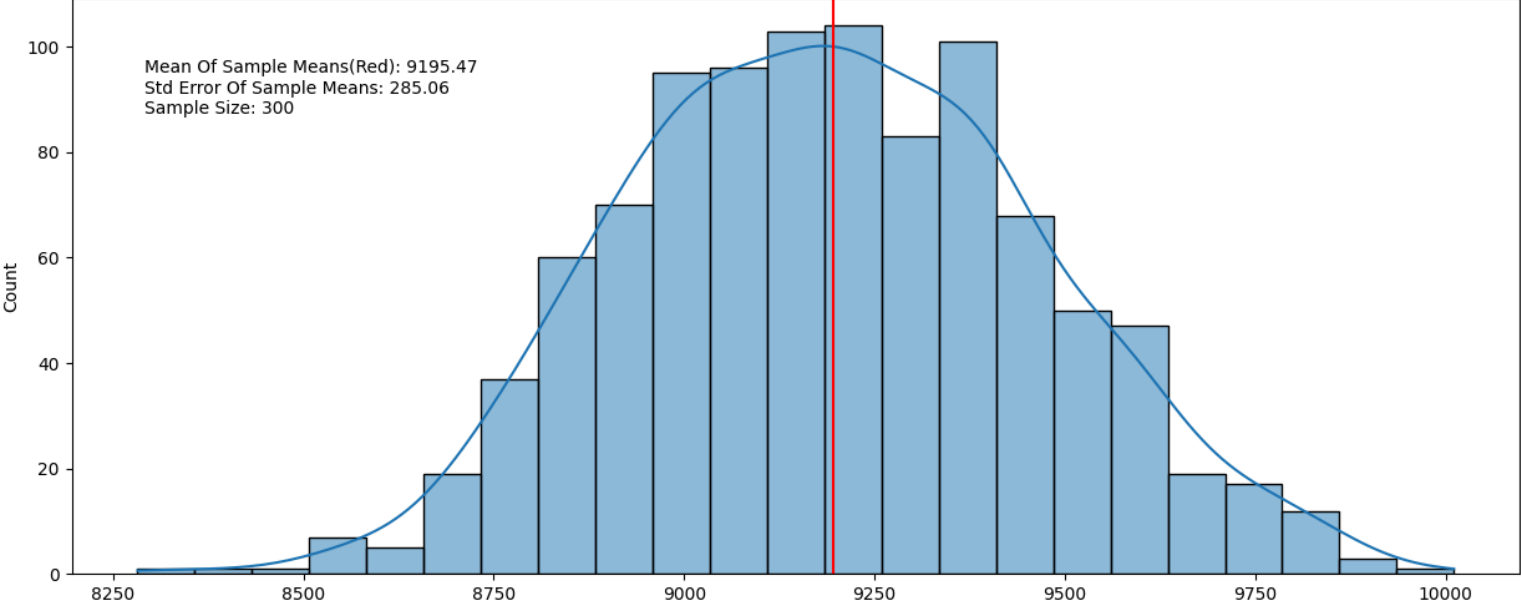
0-17-Age VS Purchase



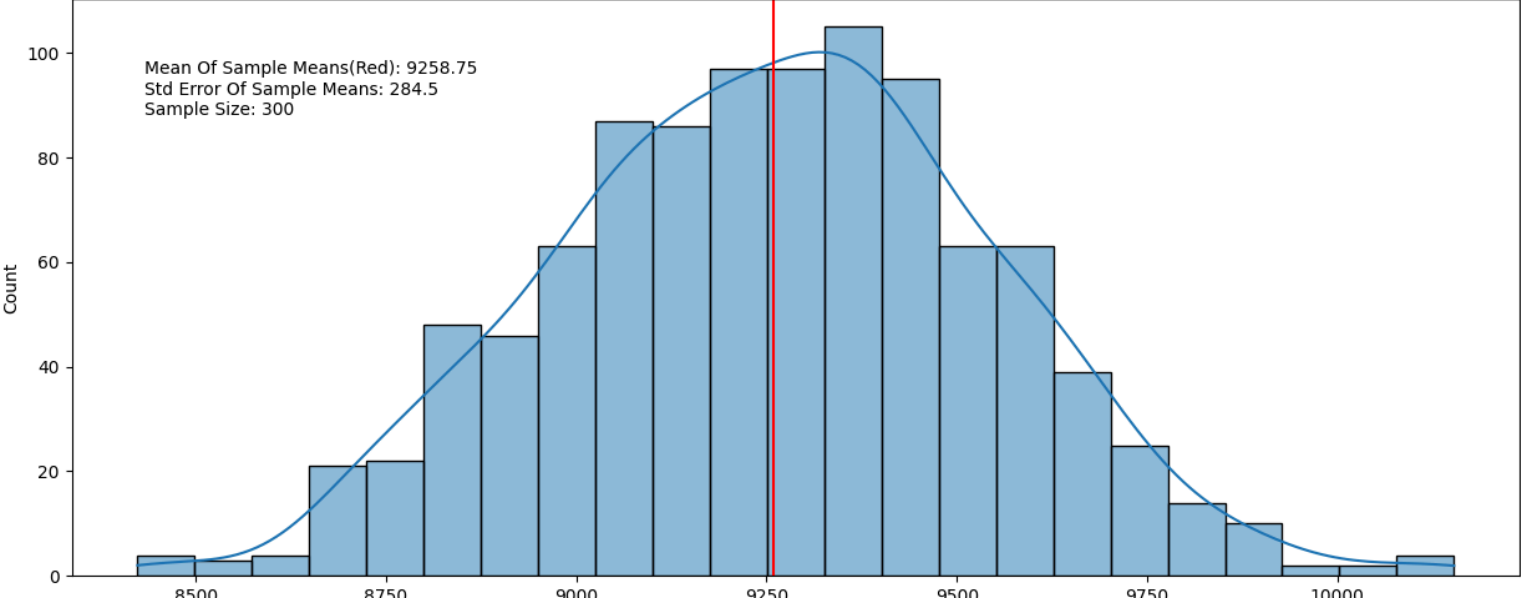
18-25-Age VS Purchase

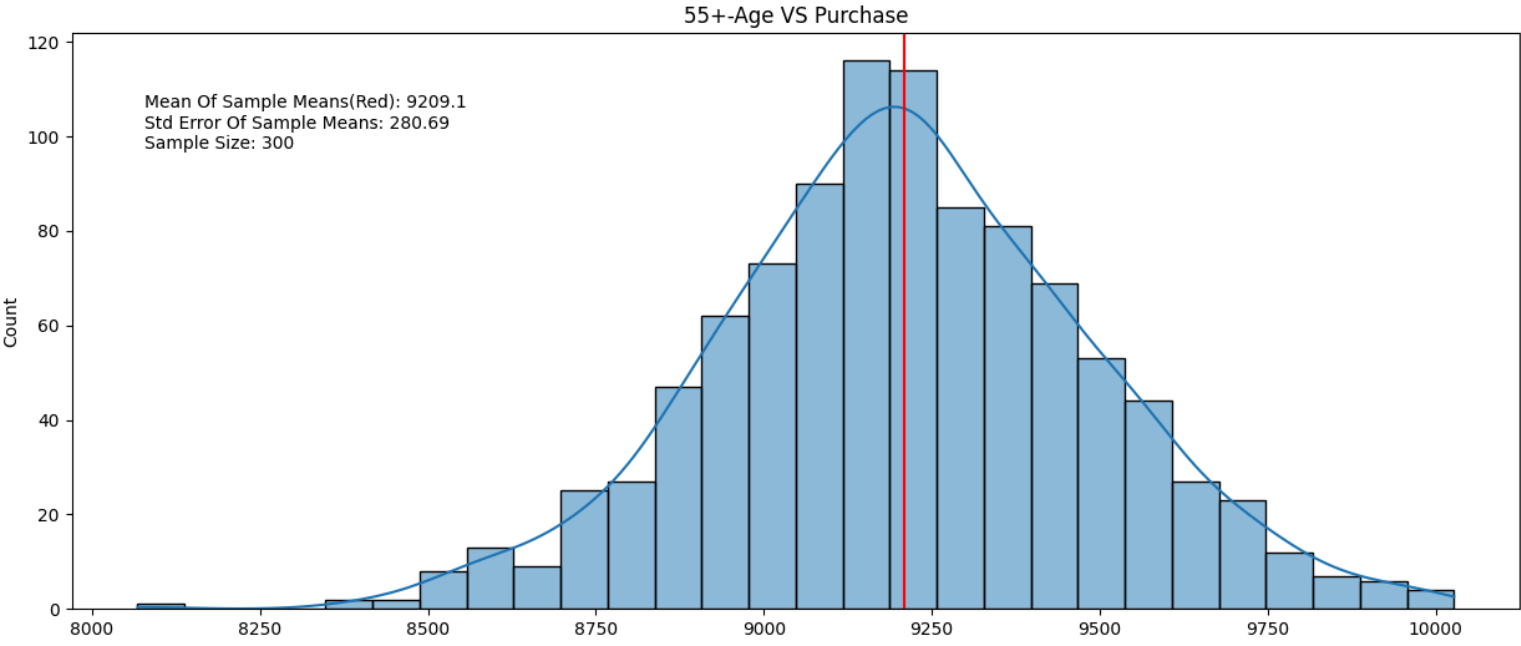
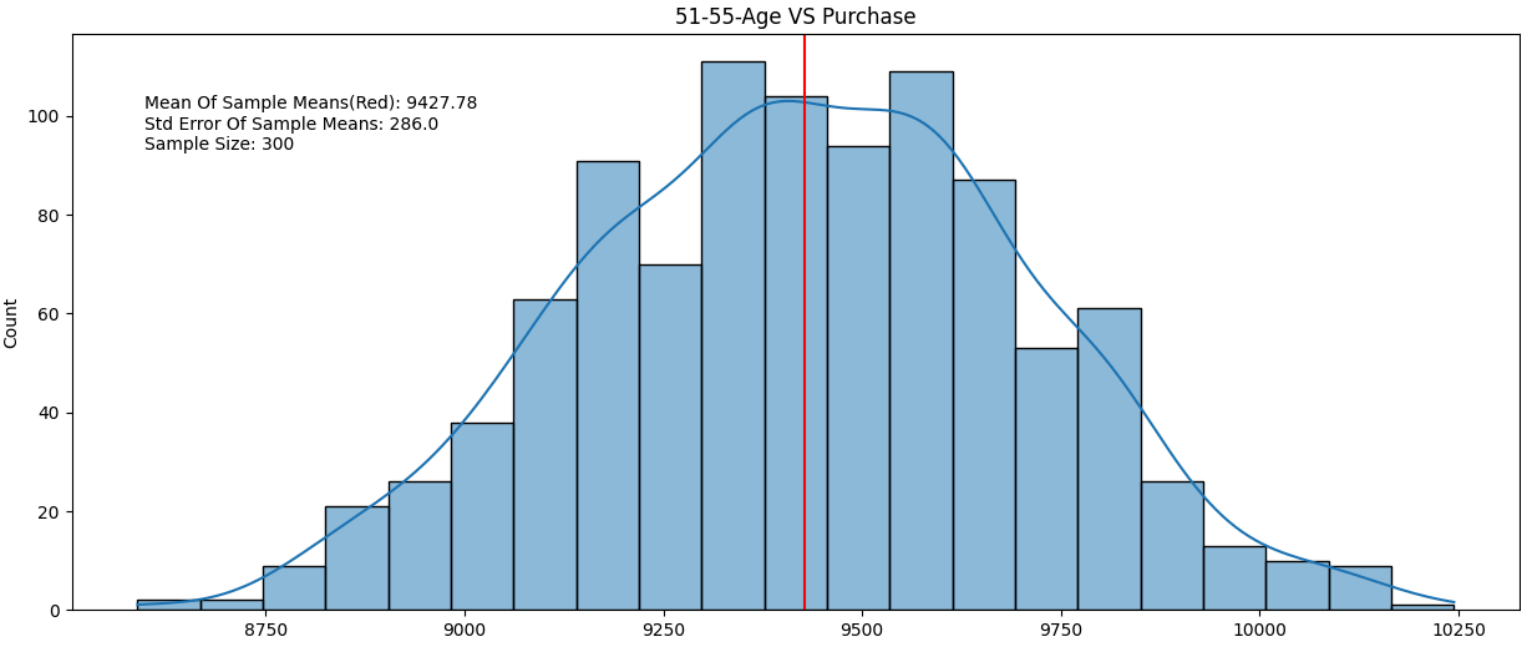
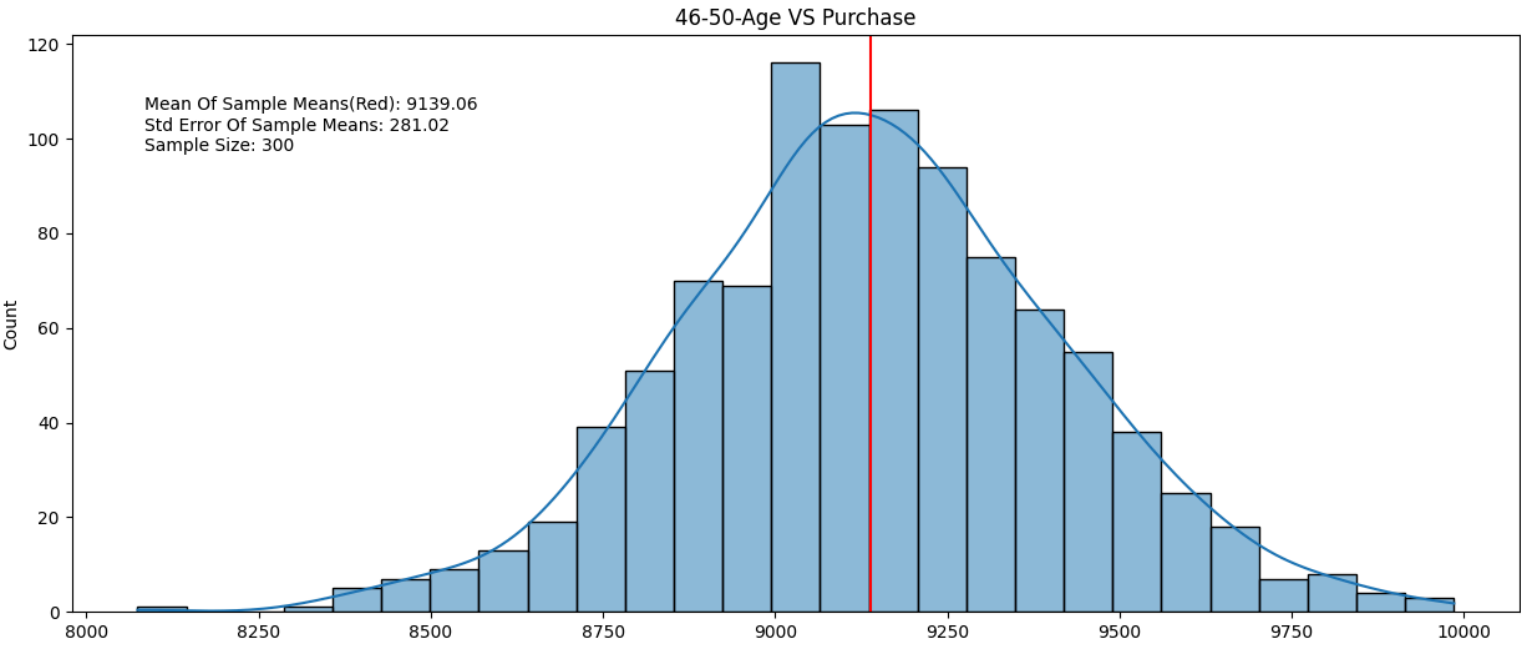


26-35-Age VS Purchase



36-45-Age VS Purchase






```
1 age_ci={}
2 for i in age_cat:
3     age_ci[f'{i}_lower_bound_{sample_size}'] = age_means[f'{i}_mean_{sample_size}'] + (norm.ppf(0.025) * age_se[f'{i}_se_{sample_size}'])
4     age_ci[f'{i}_upper_bound_{sample_size}'] = age_means[f'{i}_mean_{sample_size}'] + (norm.ppf(0.975) * age_se[f'{i}_se_{sample_size}'])
5     print(f'95% Confidence Interval For The Average Amount Spent By {i} For Sample Size {sample_size}:', (age_ci[f'{i}_lower_bound_{sample_size}'],
6                                                                                                     age_ci[f'{i}_upper_bound_{sample_size}']))
7     print('-'*100)
```

➡ 95% Confidence Interval For The Average Amount Spent By 0-17 For Sample Size 300: (8309.031829124091, 9447.419037542575)

95% Confidence Interval For The Average Amount Spent By 18-25 For Sample Size 300: (8559.11370979861, 9685.908550201388)

95% Confidence Interval For The Average Amount Spent By 26-35 For Sample Size 300: (8636.755766081467, 9754.176513918534)

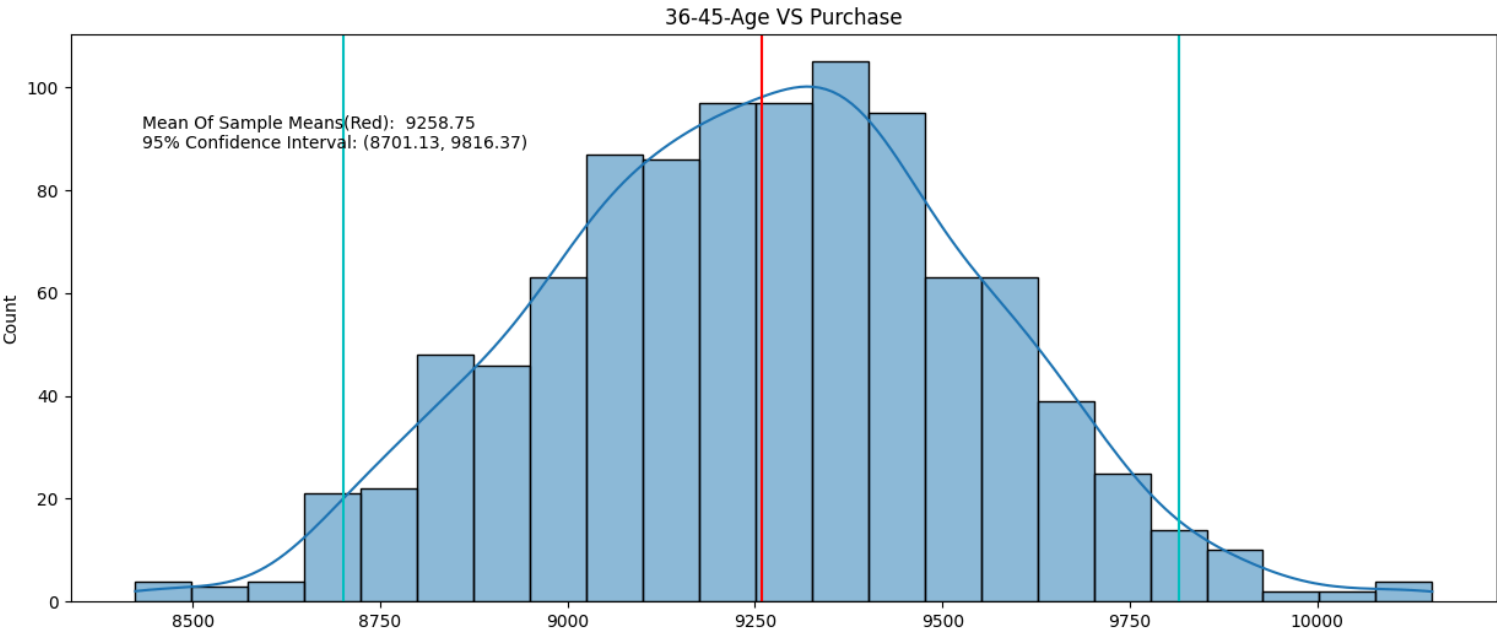
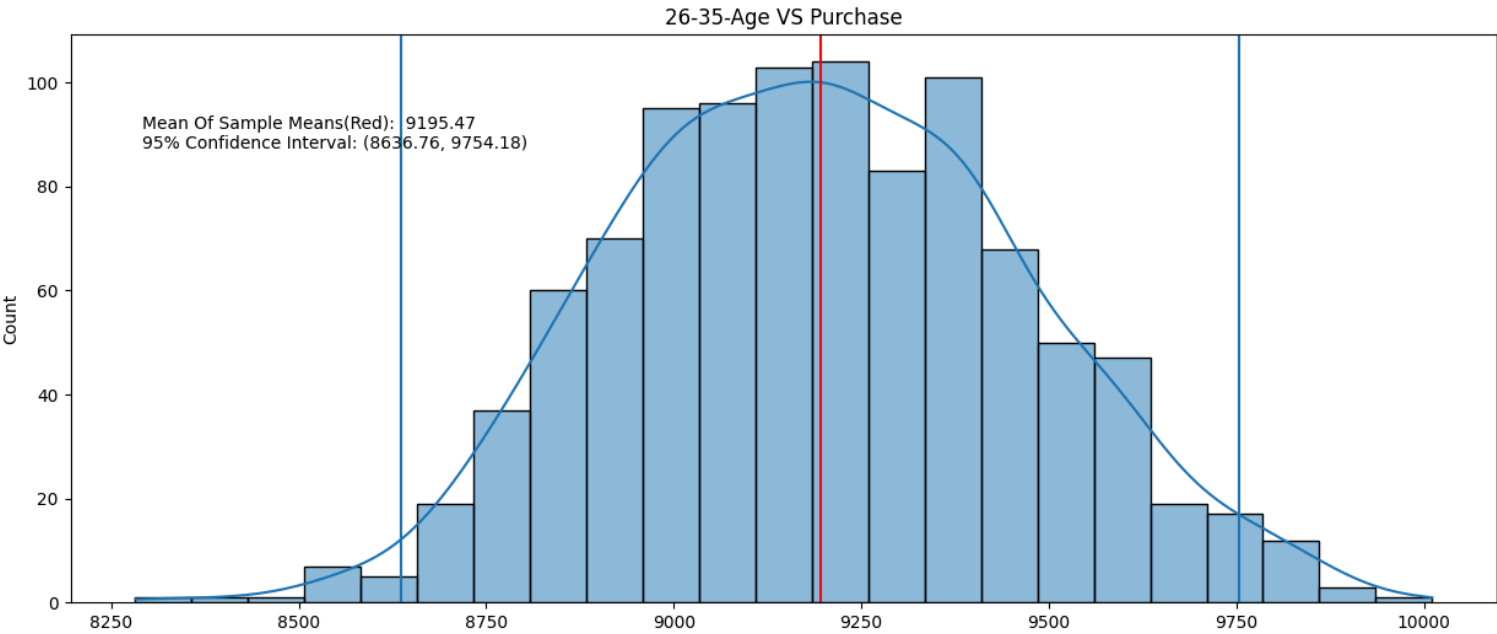
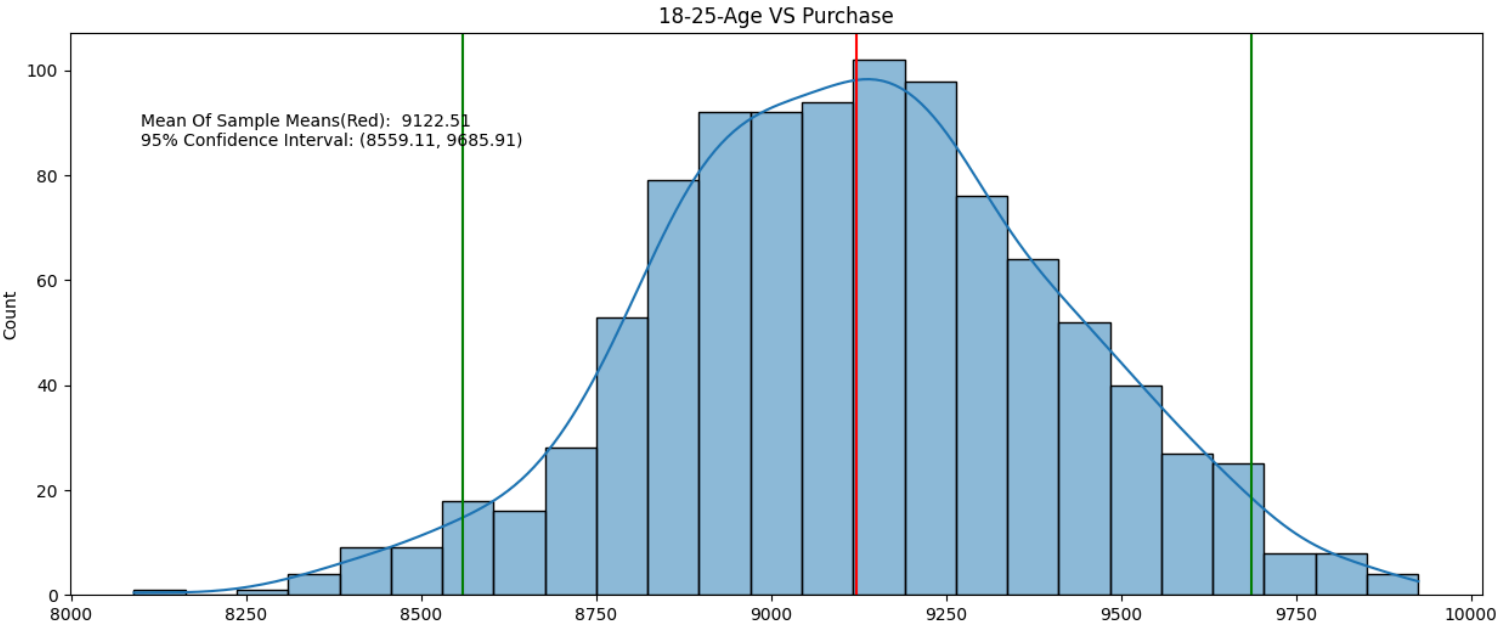
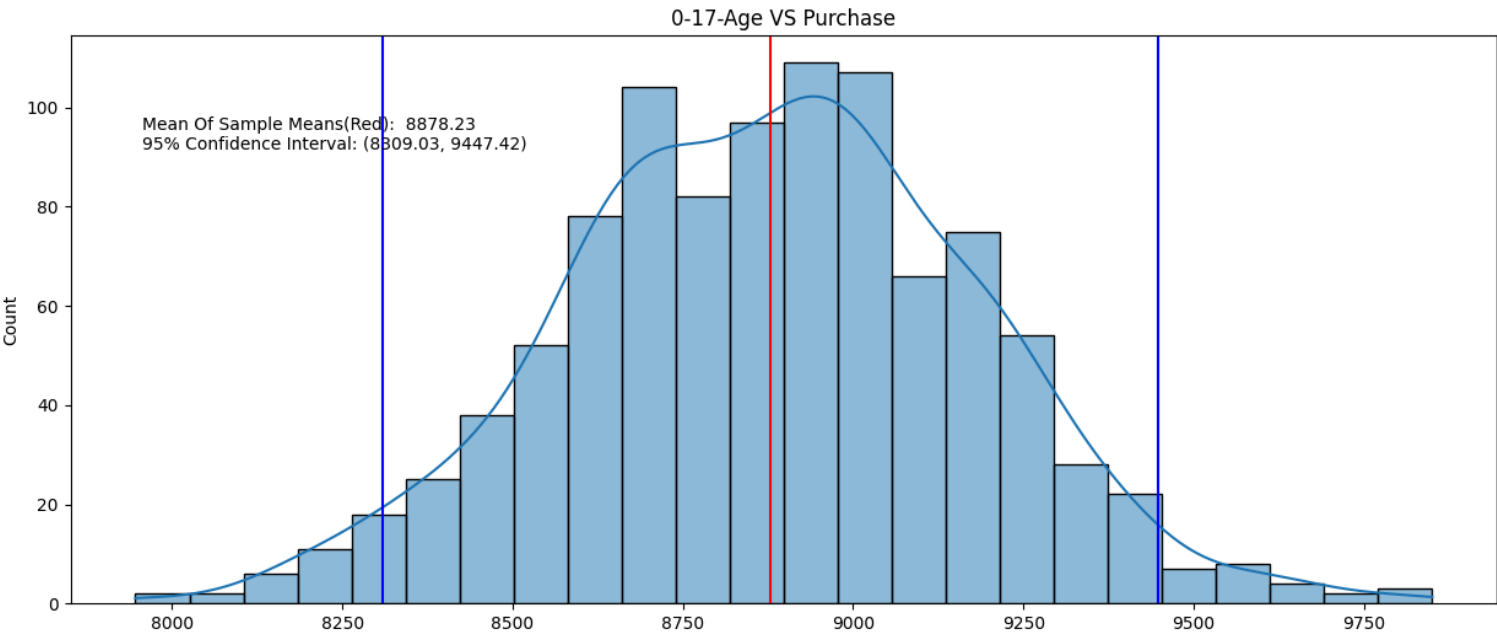
95% Confidence Interval For The Average Amount Spent By 36-45 For Sample Size 300: (8701.133623147818, 9816.366683518845)

95% Confidence Interval For The Average Amount Spent By 46-50 For Sample Size 300: (8588.274644435724, 9689.85388223094)

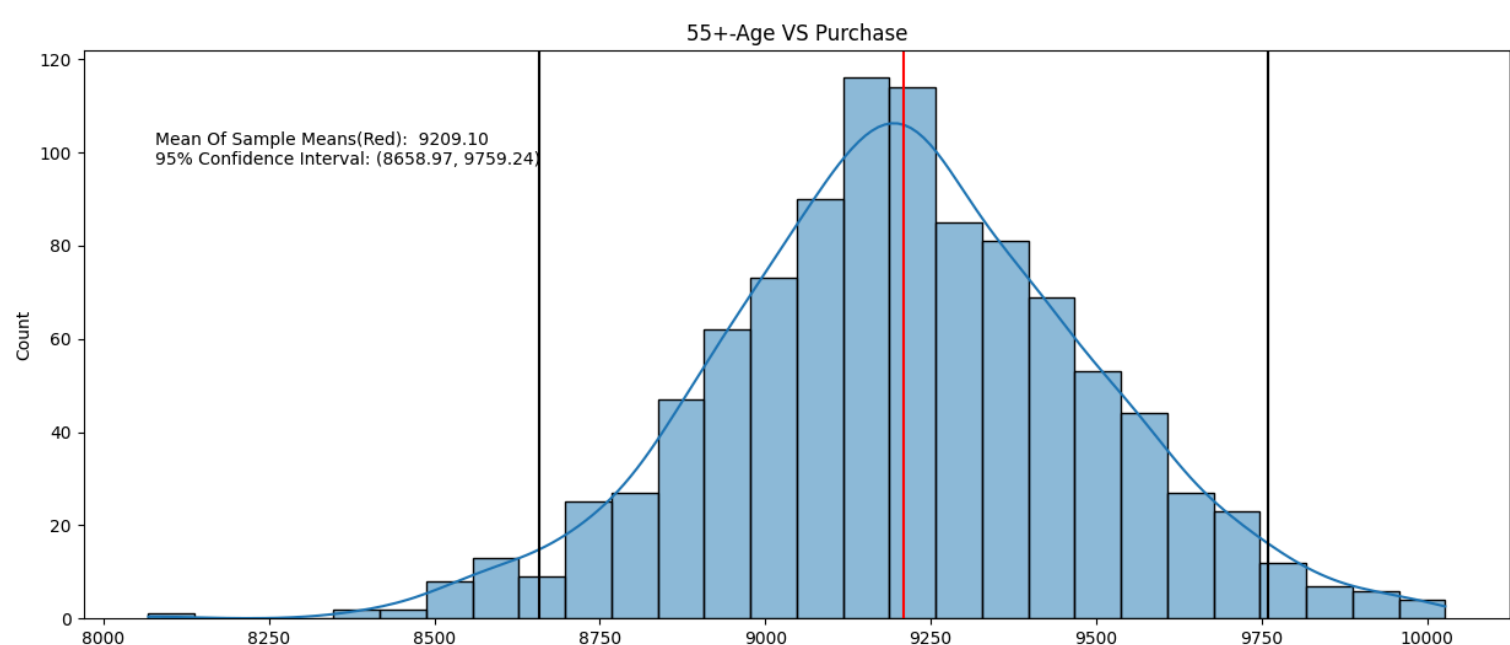
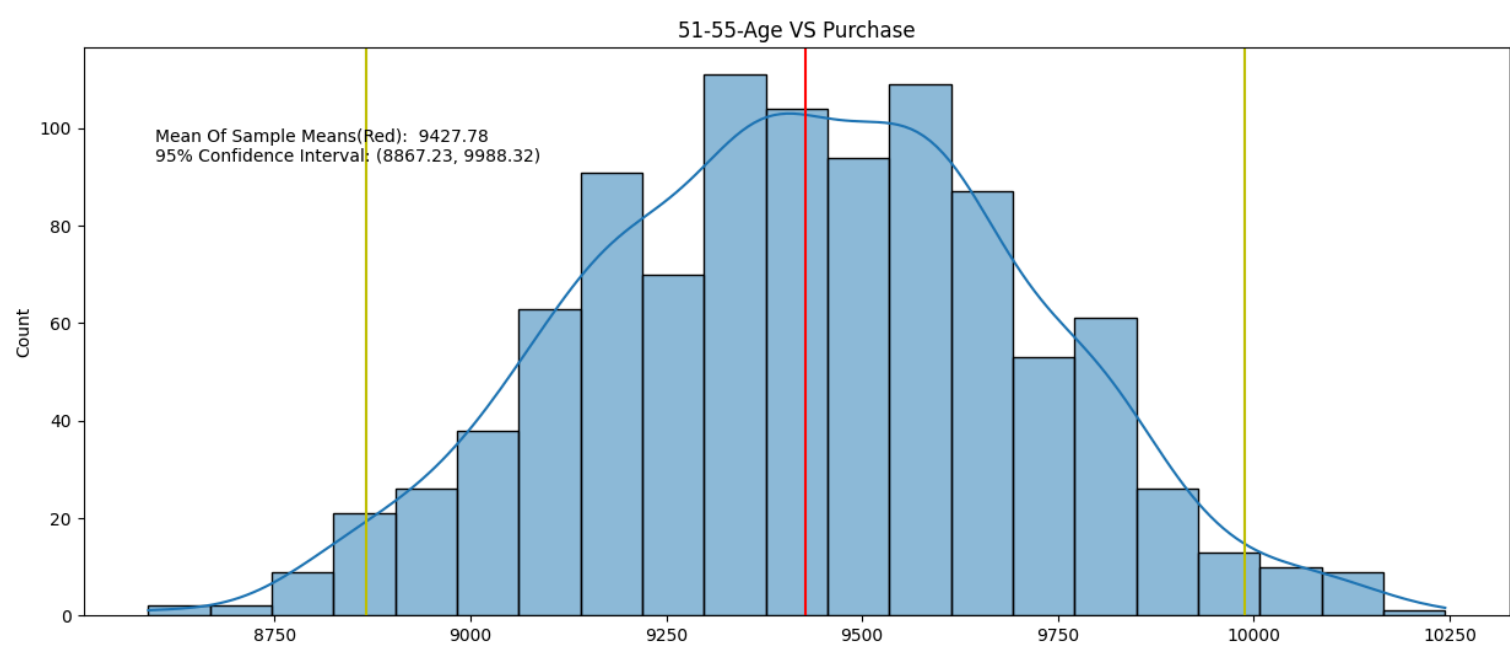
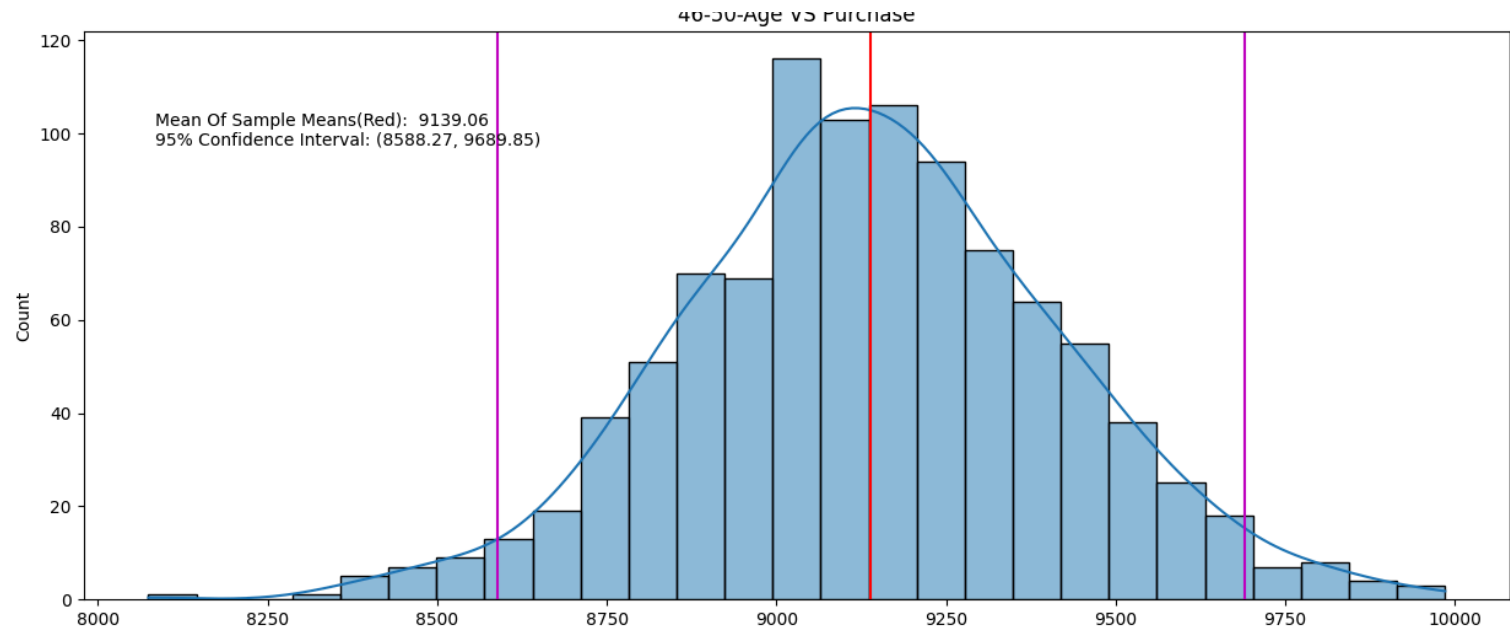
95% Confidence Interval For The Average Amount Spent By 51-55 For Sample Size 300: (8867.228641586049, 9988.32336508062)

95% Confidence Interval For The Average Amount Spent By 55+ For Sample Size 300: (8658.969529949723, 9759.238990050277)

```
1 color= ['b', 'g', '#1f77b4', 'c', 'm', 'y', 'k']
2 age_se_list= list(age_se.keys())
3 for i in age_cat:
4     plt.figure(figsize=(15,6))
5     plt.title(f'{i}-Age VS Purchase')
6     sns.histplot(age_sample_means[f'{i}_sample_mean_{sample_size}'], kde= True, fill= True)
7     plt.axvline(x= age_means[f'{i}_mean_{sample_size}'], color='r')
8     temp_mean= age_means[f'{i}_mean_{sample_size}']
9     plt.axvline(x= age_ci[f'{i}_lower_bound_{sample_size}'], color=color[age_se_list.index(f'{i}_se_{sample_size}']))
10    temp_low= age_ci[f'{i}_lower_bound_{sample_size}']
11    plt.axvline(x= age_ci[f'{i}_upper_bound_{sample_size}'], color=color[age_se_list.index(f'{i}_se_{sample_size}']))
12    temp_up= age_ci[f'{i}_upper_bound_{sample_size}']
13    plt.annotate(f'Mean Of Sample Means(Red): {np.mean(temp_mean) : .2f} \n95% Confidence Interval: {(np.round(temp_low, 2),
14                                                                                                     np.round(temp_up,2))}', xy=(0.05, 0.80), xycoords='axes fraction', fontsize=10)
15    plt.show()
```

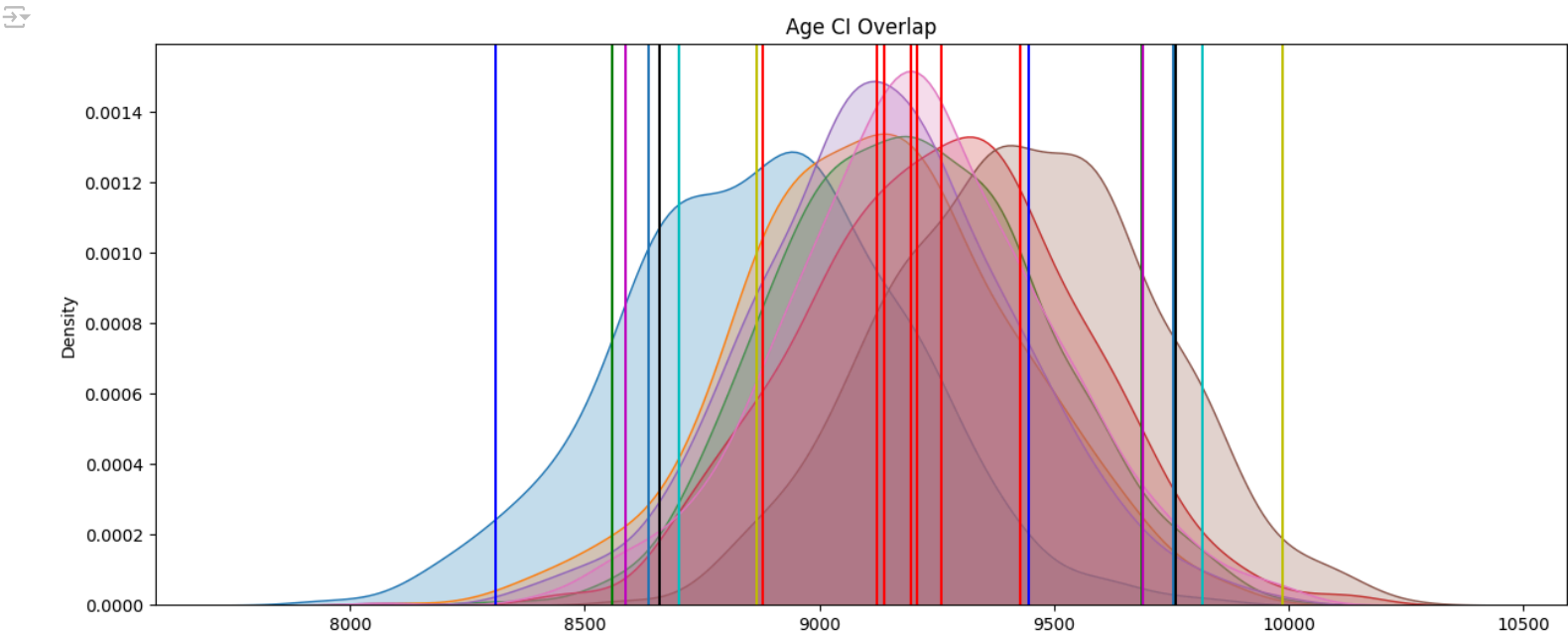


46-50-Age VS Purchase



Overlapping ?

```
1 color= ['b', 'g', '#1f77b4', 'c', 'm', 'y', 'k']
2 age_se_list= list(age_se.keys())
3 a, b, c, d, e, f, j = age_cat
4 age_cat_list= [a, b, c, d, e, f, j]
5 plt.figure(figsize=(15,6))
6 plt.title('Age CI Overlap')
7 for i in age_cat_list:
8     sns.kdeplot(age_sample_means[f'{i}_sample_mean_{sample_size}'], fill= True)
9     plt.axvline(x= age_means[f'{i}_mean_{sample_size}'], color='r')
10    plt.axvline(x= age_ci[f'{i}_lower_bound_{sample_size}'], color=color[age_se_list.index(f'{i}_se_{sample_size}']])
11    plt.axvline(x= age_ci[f'{i}_upper_bound_{sample_size}'], color=color[age_se_list.index(f'{i}_se_{sample_size}']])
12 plt.show()
```



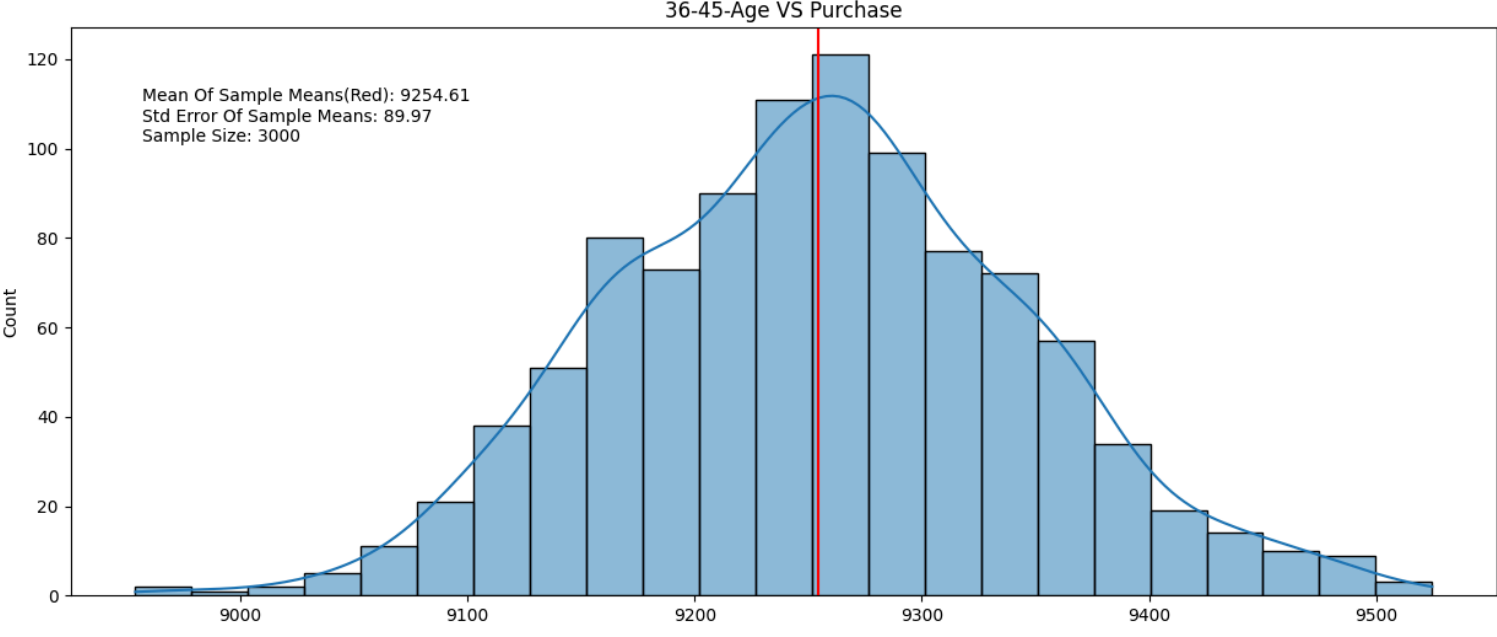
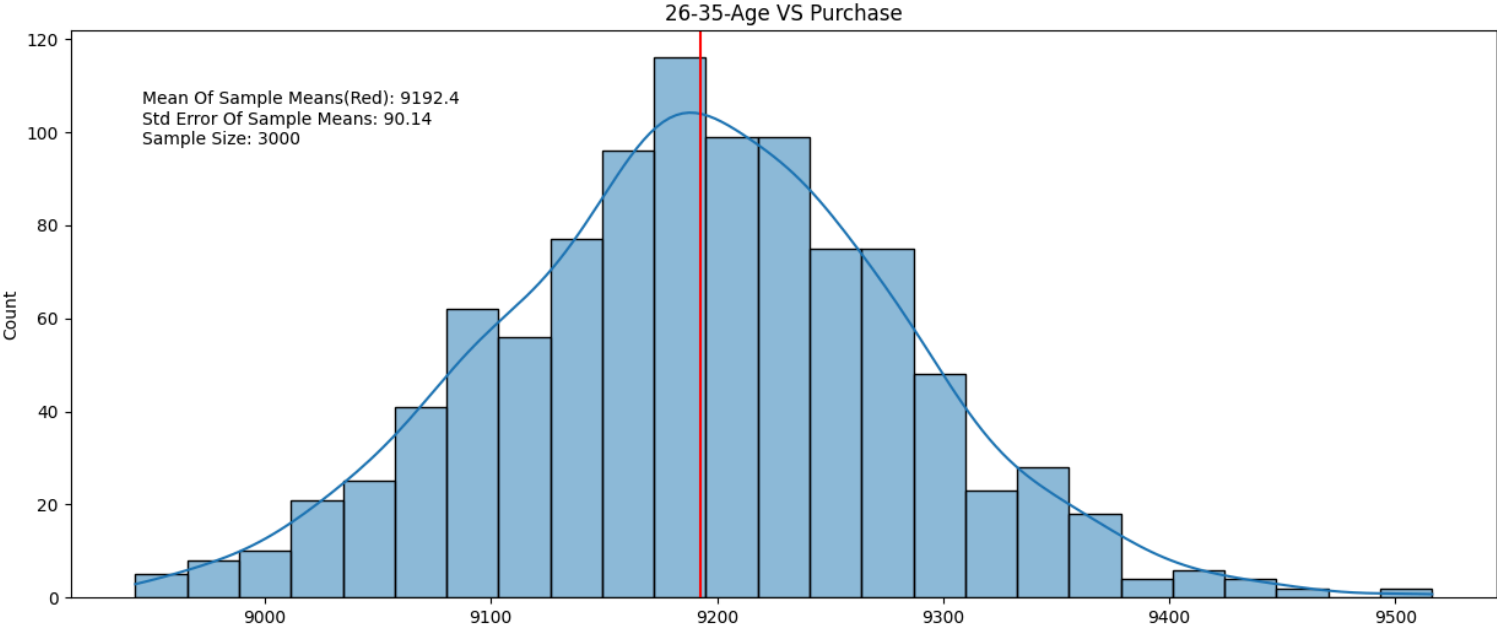
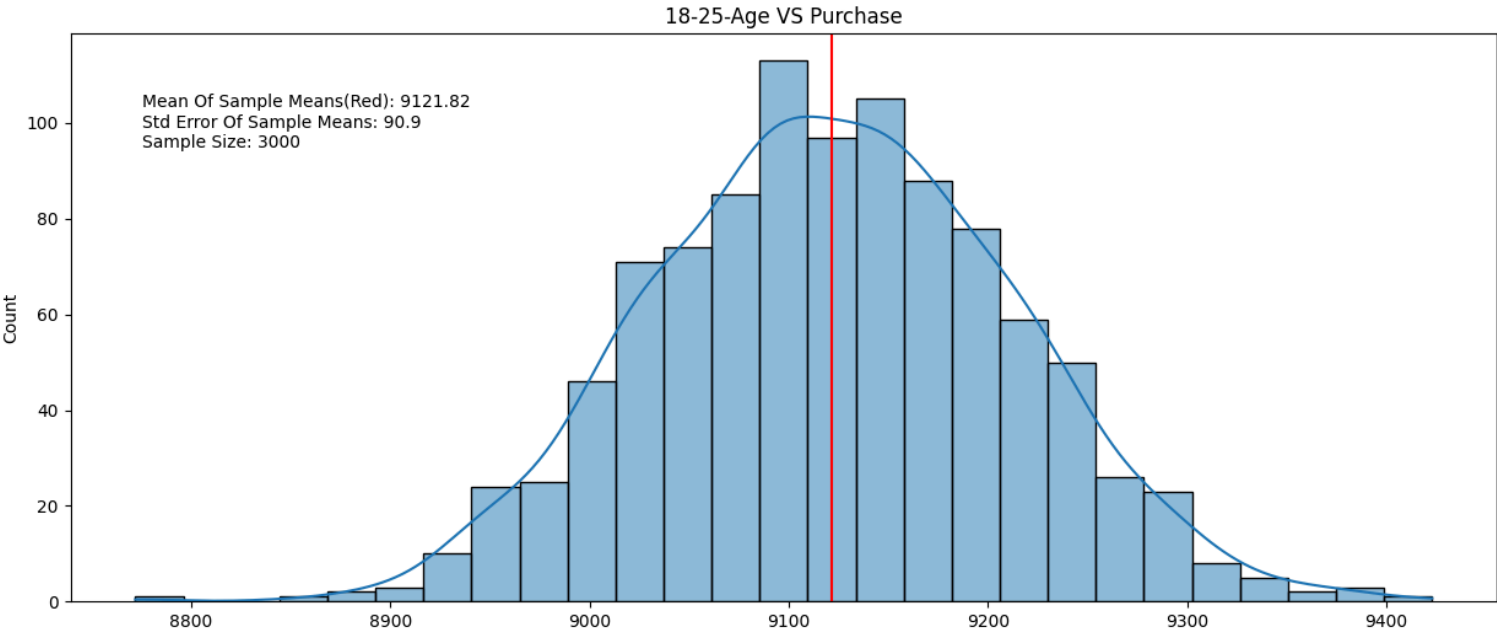
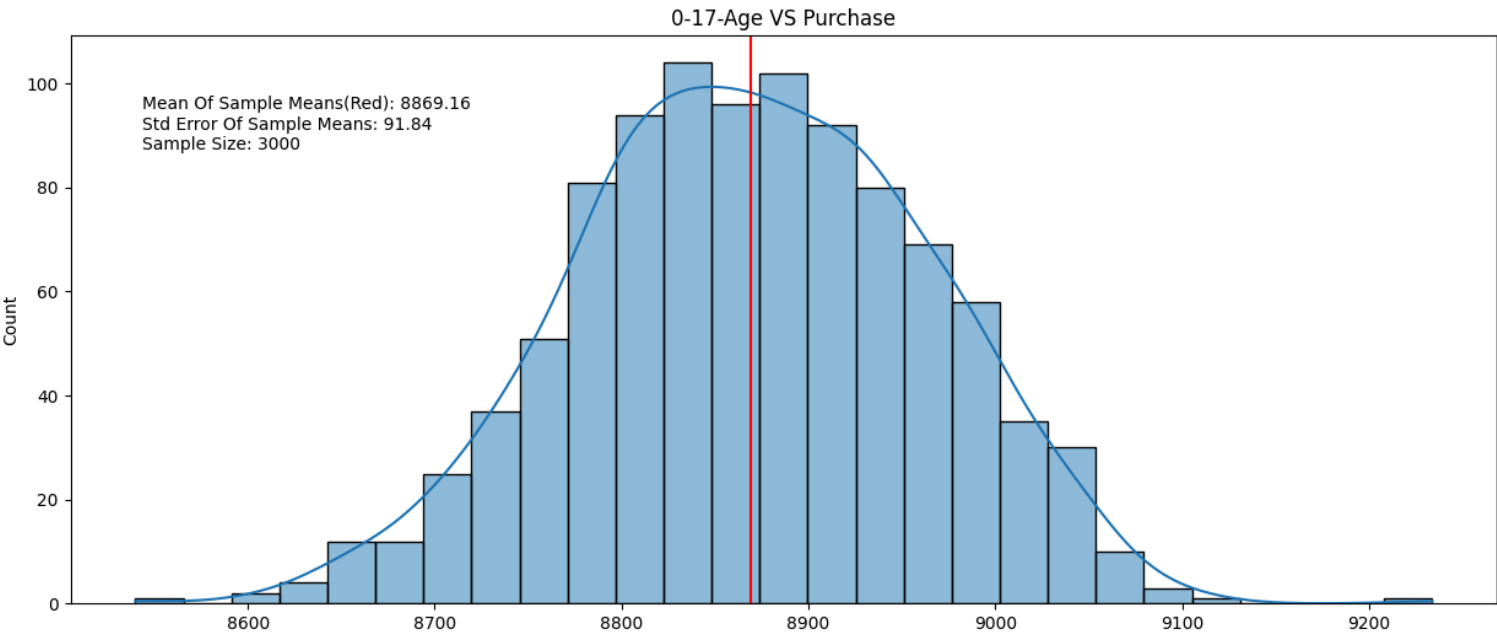
95% Confidence Interval For The Average Amount Spent By Age Category..

Sample Size: 3000

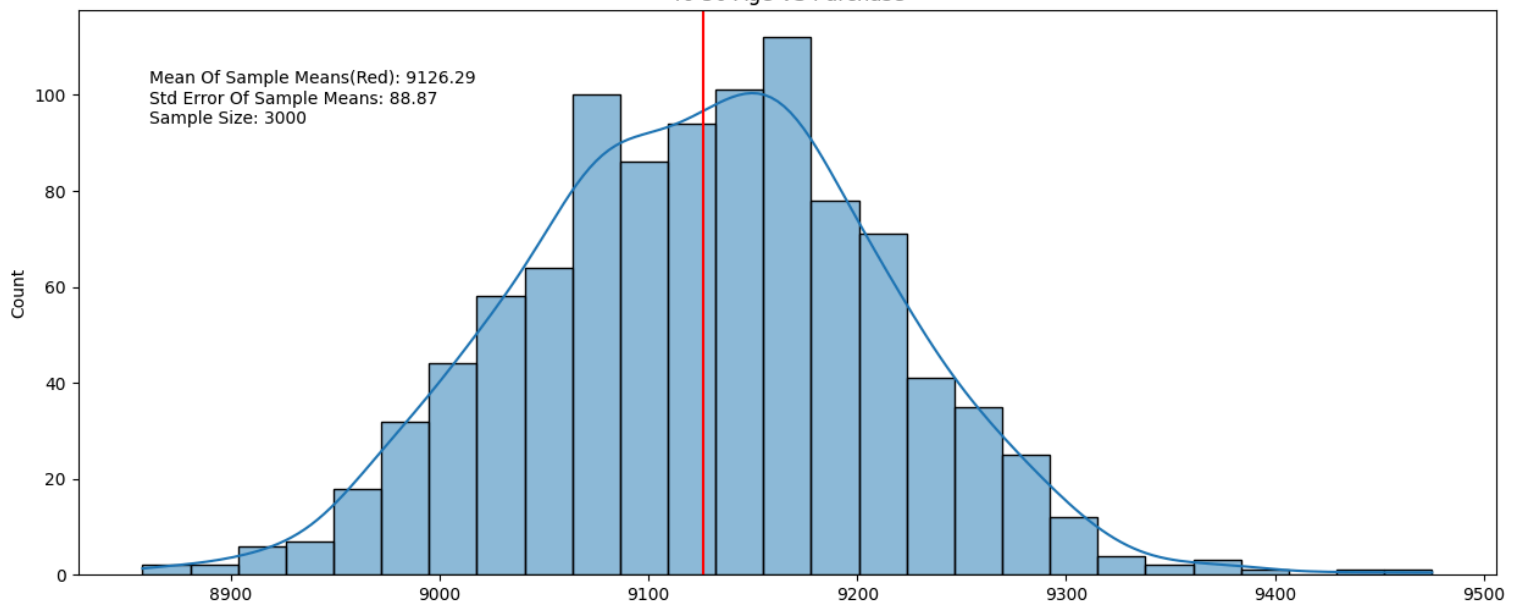
```
1 sample_size= 3000
2 age_cat= ['0-17', '18-25', '26-35', '36-45', '46-50', '51-55', '55+']
3 age_sample_means={}
4 age_means={}
5 age_se={}
6 for i in age_cat:
7     age_sample_means[f'{i}_sample_mean_{sample_size}'] = [np.mean(data.loc[data['Age']== i, 'Purchase'].sample(sample_size, replace= True))
8                                                         for _ in range(1000)]
9     age_means[f'{i}_mean_{sample_size}'] = np.mean(age_sample_means[f'{i}_sample_mean_{sample_size}'])
10    age_se[f'{i}_se_{sample_size}']= data.loc[data['Age']== i, 'Purchase'].std()/np.sqrt(sample_size)
11    print(f'{i} Purchase Amount- Mean Of Sample Means:', age_means[f'{i}_mean_{sample_size}'])
12    print(f'{i} Purchase Amount- Std Error Of Sample Means:', age_se[f'{i}_se_{sample_size}'])
13    print('-'*100)
```

```
0-17 Purchase Amount- Mean Of Sample Means: 8869.156598
0-17 Purchase Amount- Std Error Of Sample Means: 91.83578030511637
-----
18-25 Purchase Amount- Mean Of Sample Means: 9121.818608
18-25 Purchase Amount- Std Error Of Sample Means: 90.90060275354716
-----
26-35 Purchase Amount- Mean Of Sample Means: 9192.401153333334
26-35 Purchase Amount- Std Error Of Sample Means: 90.144377543831
-----
36-45 Purchase Amount- Mean Of Sample Means: 9254.608859333333
36-45 Purchase Amount- Std Error Of Sample Means: 89.96789279064534
-----
46-50 Purchase Amount- Mean Of Sample Means: 9126.293803333334
46-50 Purchase Amount- Std Error Of Sample Means: 88.86641392552428
-----
51-55 Purchase Amount- Mean Of Sample Means: 9424.881874333334
51-55 Purchase Amount- Std Error Of Sample Means: 90.44076388657436
-----
55+ Purchase Amount- Mean Of Sample Means: 9211.020649666667
55+ Purchase Amount- Std Error Of Sample Means: 88.76075176090565
-----
```

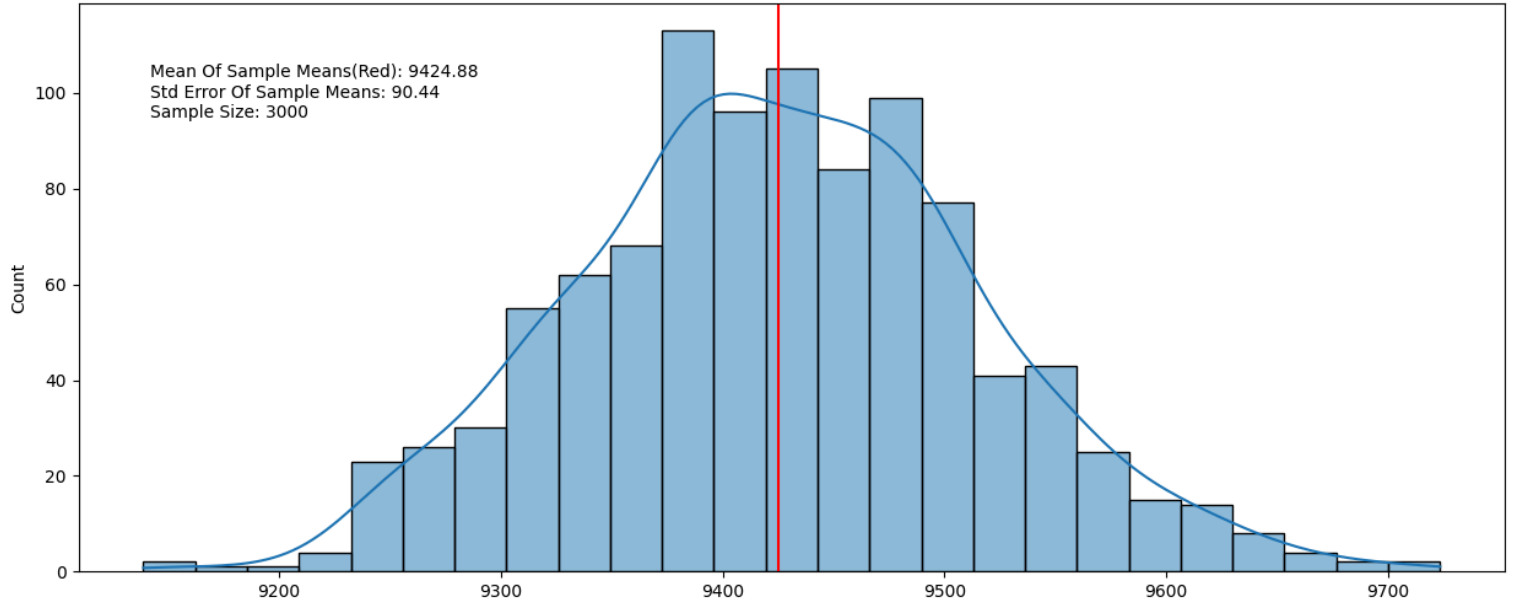
```
1 for i in age_cat:
2     plt.figure(figsize=(15,6))
3     plt.title(f'{i}-Age VS Purchase')
4     sns.histplot(age_sample_means[f'{i}_sample_mean_{sample_size}'], kde= True, fill= True)
5     plt.axvline(x= age_means[f'{i}_mean_{sample_size}'], color='r')
6     temp_mean= age_means[f'{i}_mean_{sample_size}']
7     temp_se= age_se[f'{i}_se_{sample_size}']
8     plt.annotate(f'Mean Of Sample Means(Red): {np.round(temp_mean, 2)} \nStd Error Of Sample Means: {np.round(temp_se, 2)}
9                 \nSample Size: {sample_size}', xy=(0.05, 0.80), xycoords='axes fraction', fontsize=10)
10    plt.show()
```



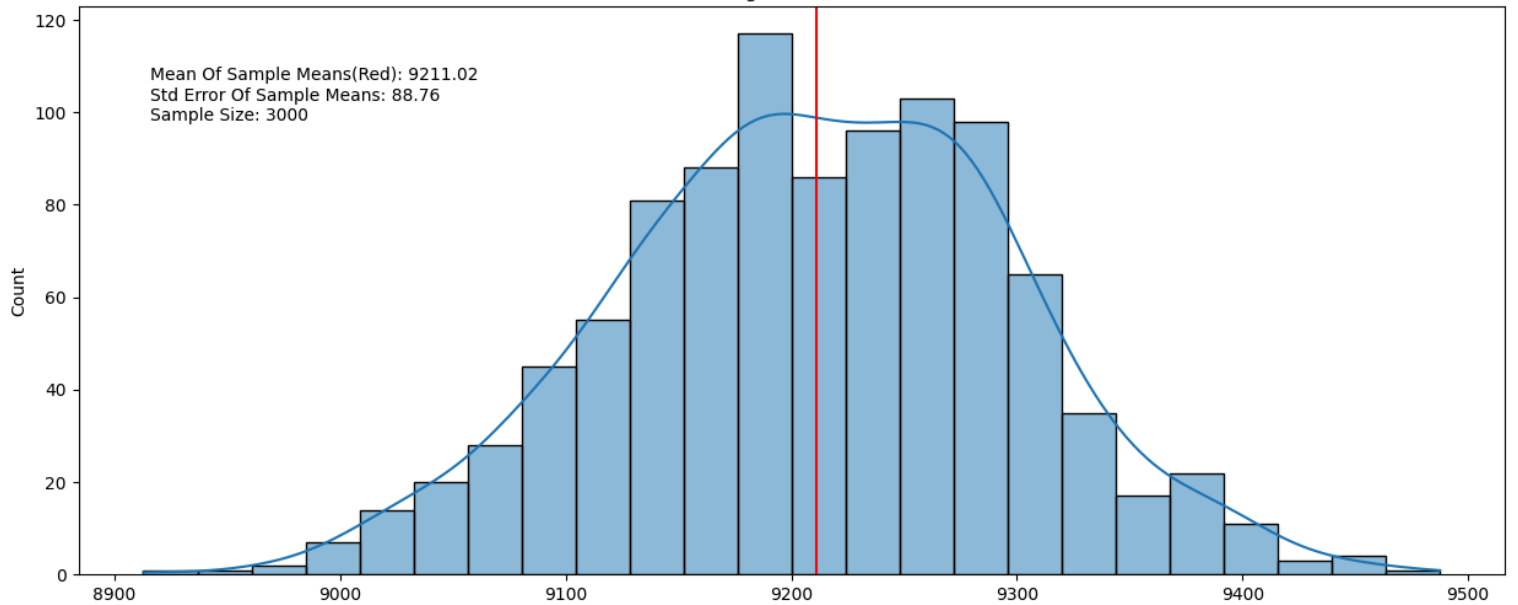
46-50-Age VS Purchase



51-55-Age VS Purchase



55+-Age VS Purchase



```
1 age_ci={}
2 for i in age_cat:
3     age_ci[f'{i}_lower_bound_{sample_size}'] = age_means[f'{i}_mean_{sample_size}'] + (norm.ppf(0.025) * age_se[f'{i}_se_{sample_size}'])
4     age_ci[f'{i}_upper_bound_{sample_size}'] = age_means[f'{i}_mean_{sample_size}'] + (norm.ppf(0.975) * age_se[f'{i}_se_{sample_size}'])
5     print(f'95% Confidence Interval For The Average Amount Spent By {i} For Sample Size {sample_size}:',
6           (age_ci[f'{i}_lower_bound_{sample_size}'], age_ci[f'{i}_upper_bound_{sample_size}']))
7     print('-'*100)
```

➡ 95% Confidence Interval For The Average Amount Spent By 0-17 For Sample Size 3000: (8689.161776109839, 9049.15141989016)

95% Confidence Interval For The Average Amount Spent By 18-25 For Sample Size 3000: (8943.656700430065, 9299.980515569934)

95% Confidence Interval For The Average Amount Spent By 26-35 For Sample Size 3000: (9015.721419938644, 9369.080886728025)

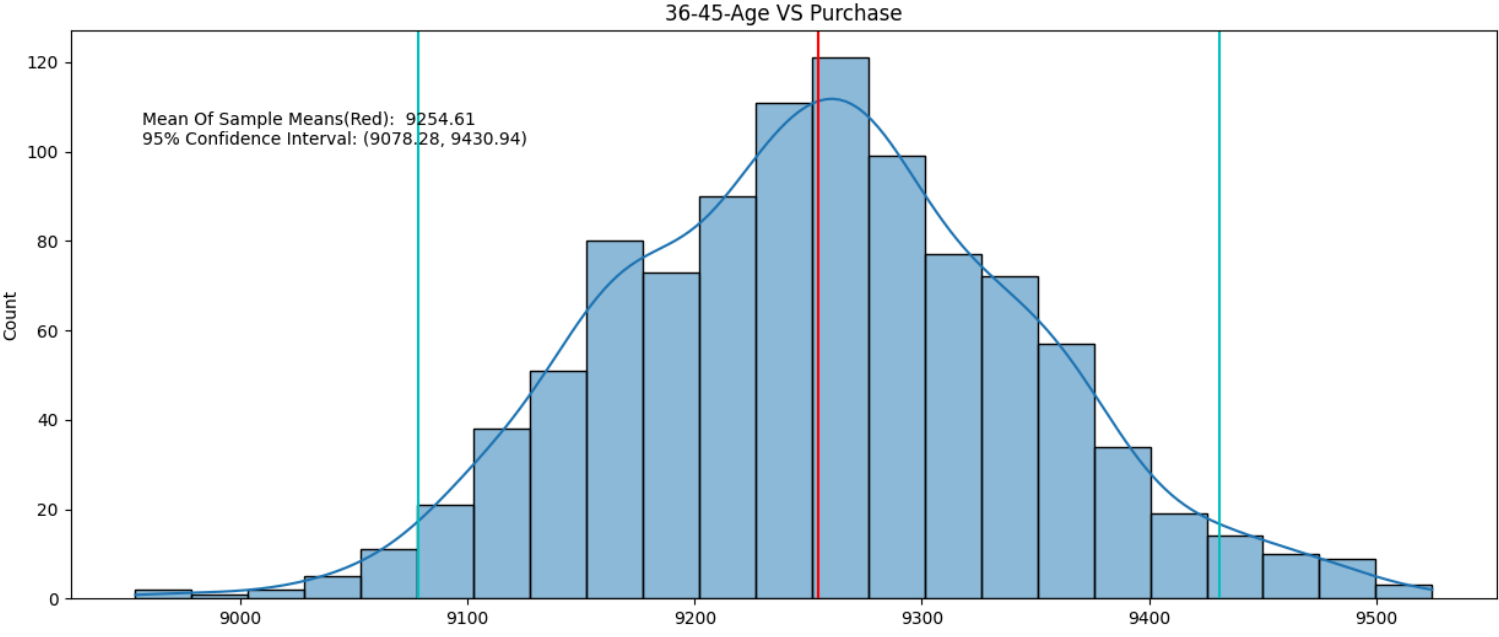
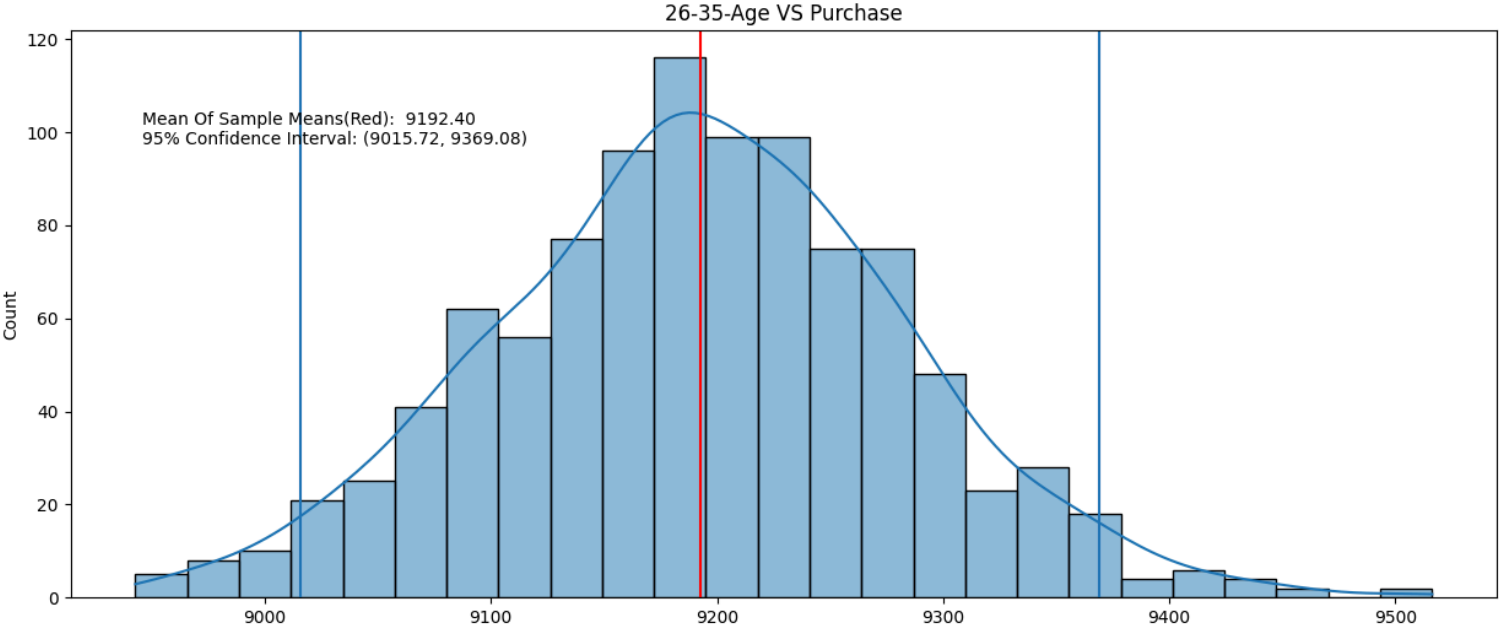
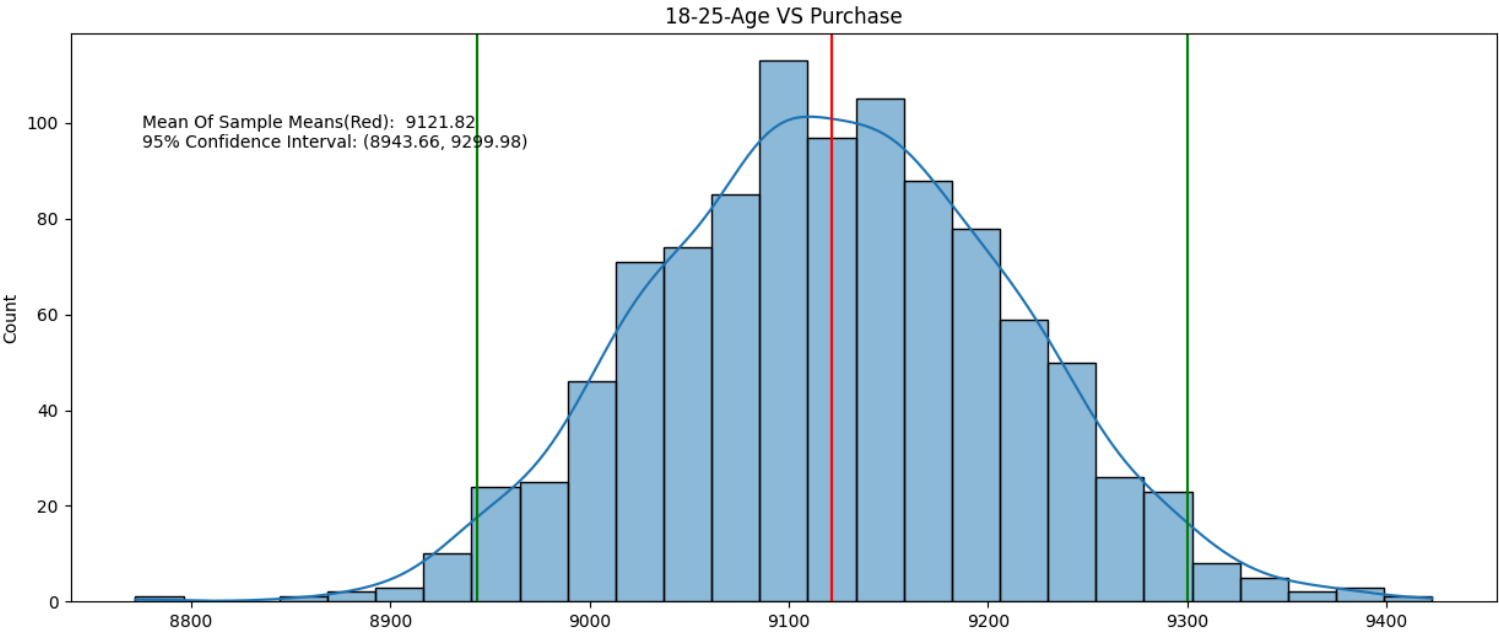
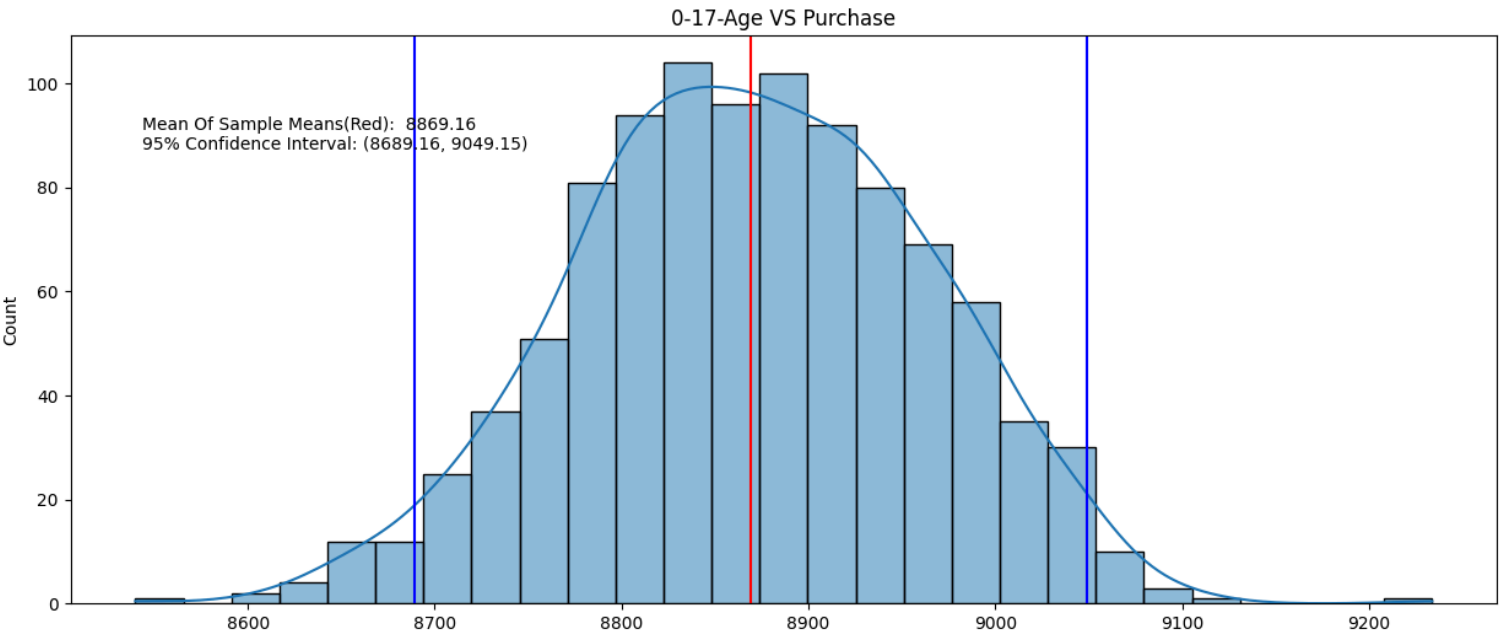
95% Confidence Interval For The Average Amount Spent By 36-45 For Sample Size 3000: (9078.275029698707, 9430.94268896796)

95% Confidence Interval For The Average Amount Spent By 46-50 For Sample Size 3000: (8952.118832604077, 9300.46877406259)

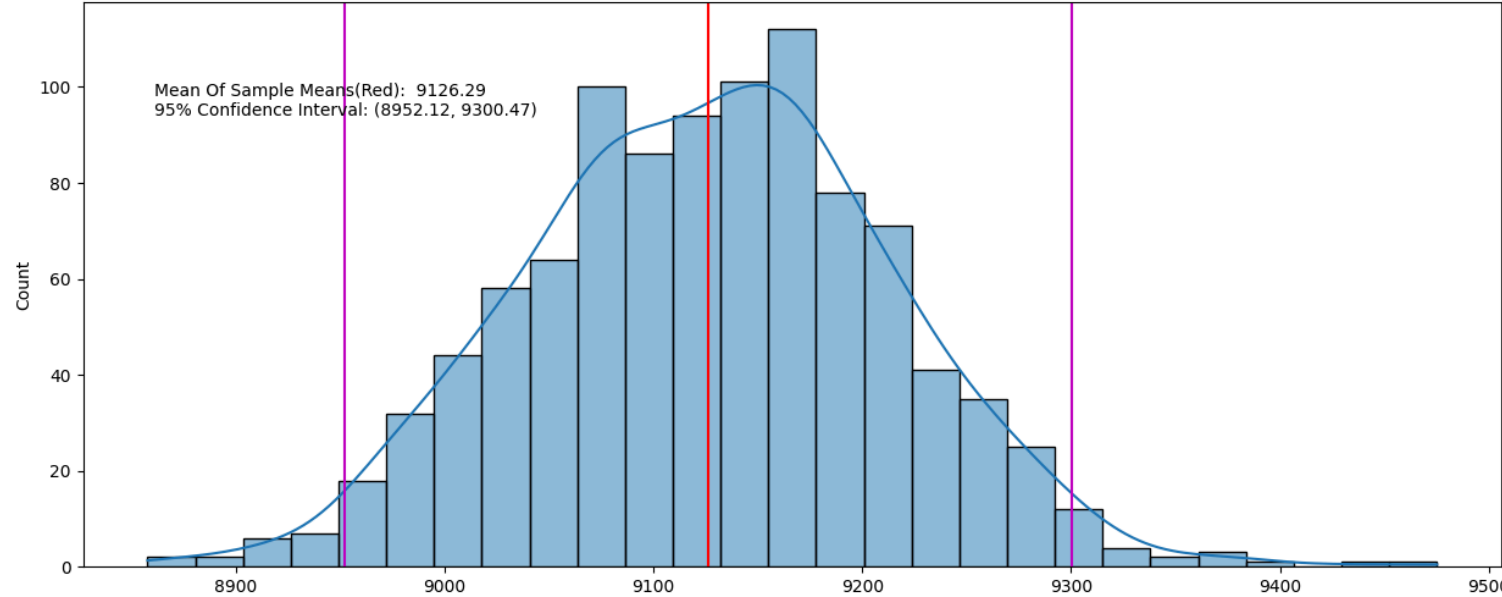
95% Confidence Interval For The Average Amount Spent By 51-55 For Sample Size 3000: (9247.621234381357, 9602.142514285311)

95% Confidence Interval For The Average Amount Spent By 55+ For Sample Size 3000: (9037.052772974592, 9384.988526358742)

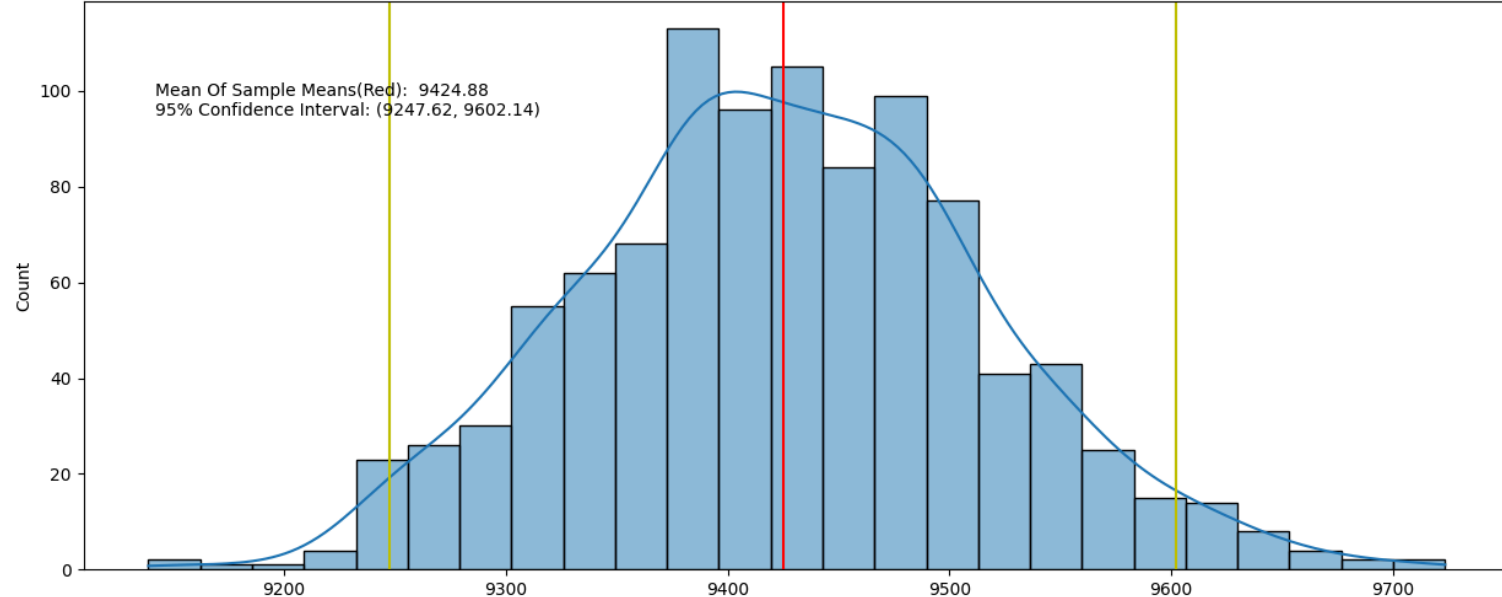
```
1 color= ['b', 'g', '#1f77b4', 'c', 'm', 'y', 'k']
2 age_se_list= list(age_se.keys())
3 for i in age_cat:
4     plt.figure(figsize=(15,6))
5     plt.title(f'{i}-Age VS Purchase')
6     sns.histplot(age_sample_means[f'{i}_sample_mean_{sample_size}'], kde= True, fill= True)
7     plt.axvline(x= age_means[f'{i}_mean_{sample_size}'], color='r')
8     temp_mean= age_means[f'{i}_mean_{sample_size}']
9     plt.axvline(x= age_ci[f'{i}_lower_bound_{sample_size}'], color=color[age_se_list.index(f'{i}_se_{sample_size}']))
10    temp_low= age_ci[f'{i}_lower_bound_{sample_size}']
11    plt.axvline(x= age_ci[f'{i}_upper_bound_{sample_size}'], color=color[age_se_list.index(f'{i}_se_{sample_size}']))
12    temp_up= age_ci[f'{i}_upper_bound_{sample_size}']
13    plt.annotate(f'Mean Of Sample Means(Red): {np.mean(temp_mean) : .2f} \n95% Confidence Interval:
14                {(np.round(temp_low, 2), np.round(temp_up,2))}', xy=(0.05, 0.80), xycoords='axes fraction', fontsize=10)
15    plt.show()
```



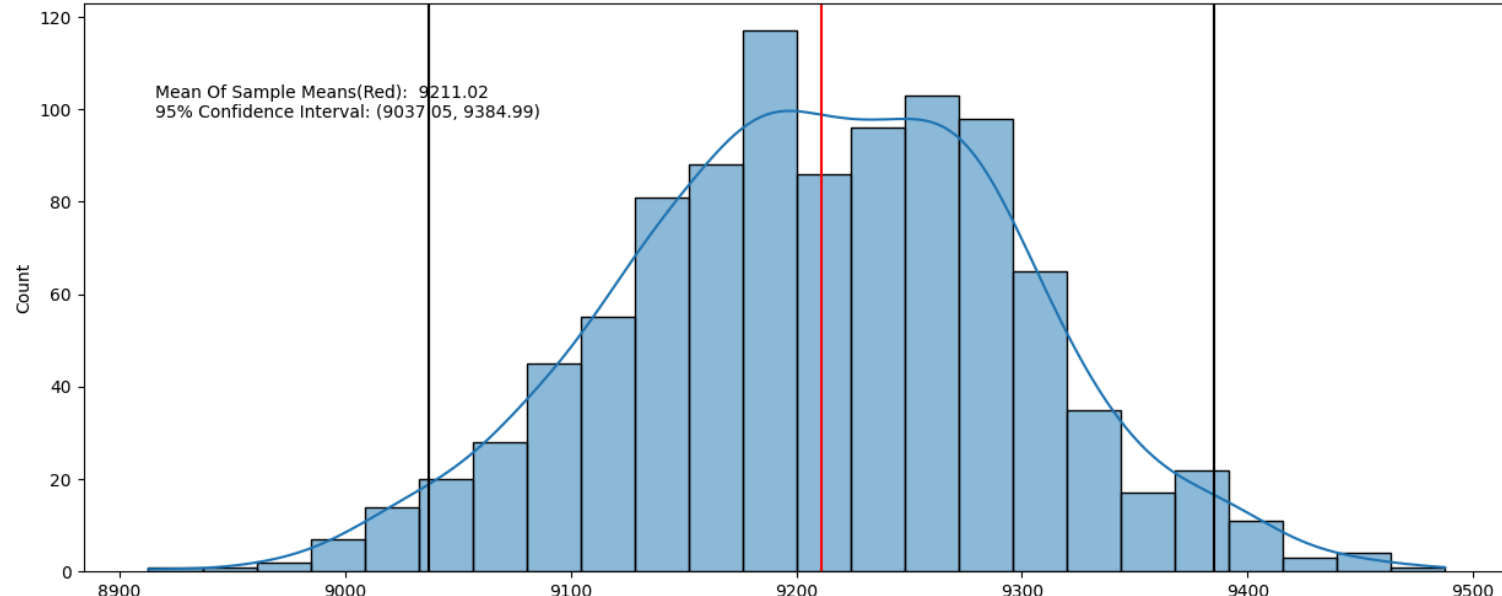
46-50-Age VS Purchase



51-55-Age VS Purchase

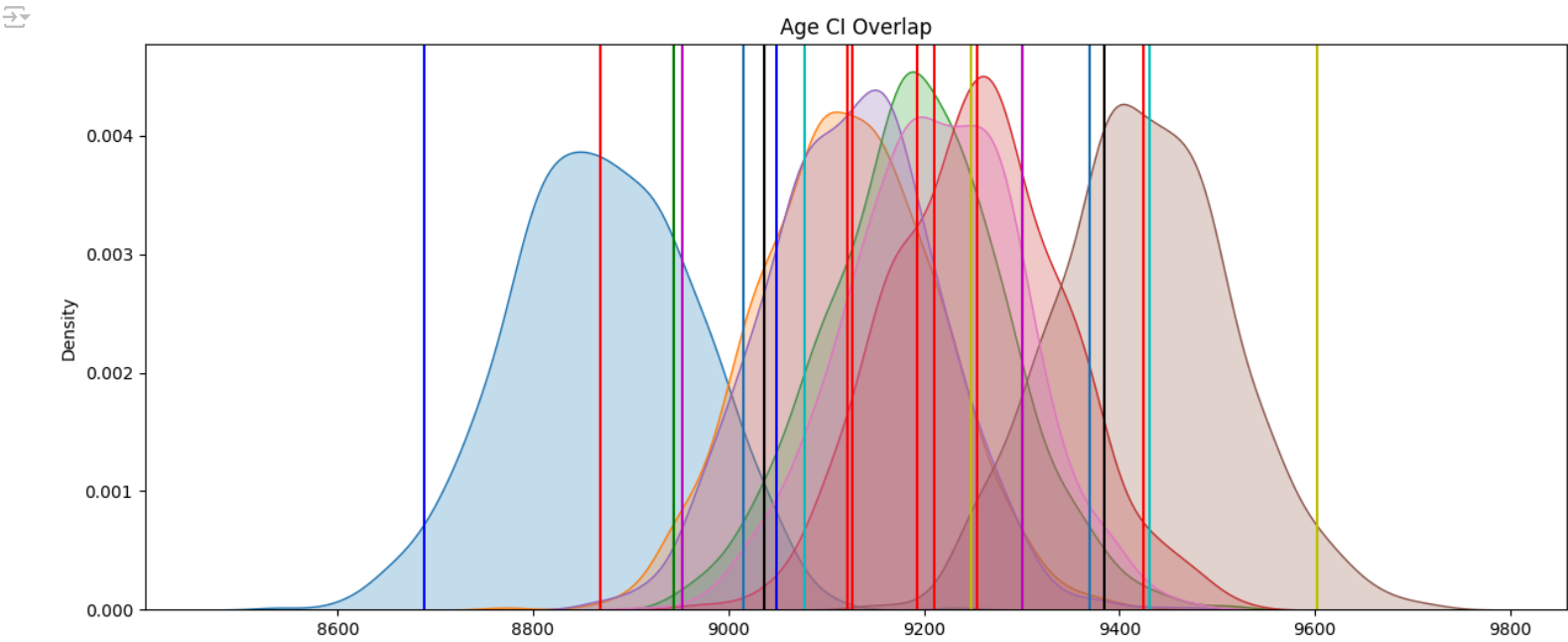


55+-Age VS Purchase



Overlapping ?

```
1 color= ['b', 'g', '#1f77b4', 'c', 'm', 'y', 'k']
2 age_se_list= list(age_se.keys())
3 a, b, c, d, e, f, j = age_cat
4 age_cat_list= [a, b, c, d, e, f, j]
5 plt.figure(figsize=(15,6))
6 plt.title('Age CI Overlap')
7 for i in age_cat_list:
8     sns.kdeplot(age_sample_means[f'{i}_sample_mean_{sample_size}'], fill= True)
9     plt.axvline(x= age_means[f'{i}_mean_{sample_size}'], color='r')
10    plt.axvline(x= age_ci[f'{i}_lower_bound_{sample_size}'], color=color[age_se_list.index(f'{i}_se_{sample_size}']])
11    plt.axvline(x= age_ci[f'{i}_upper_bound_{sample_size}'], color=color[age_se_list.index(f'{i}_se_{sample_size}']])
12 plt.show()
```



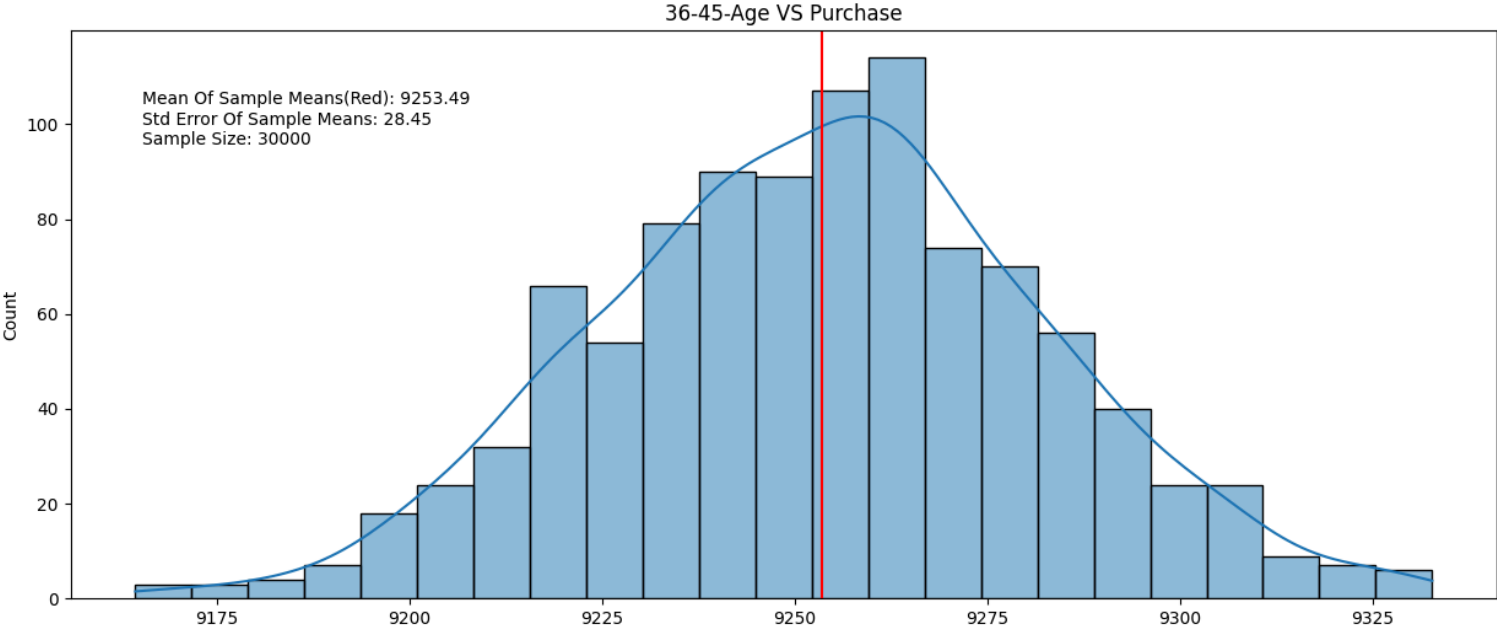
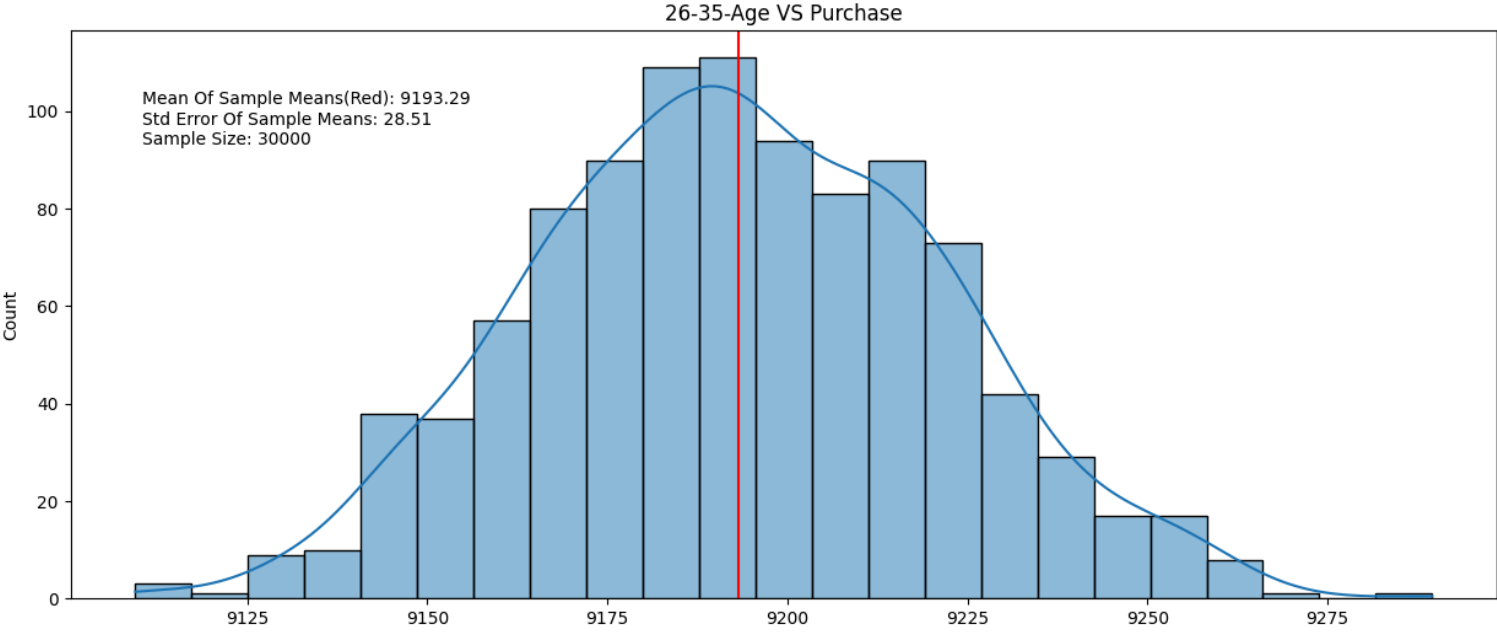
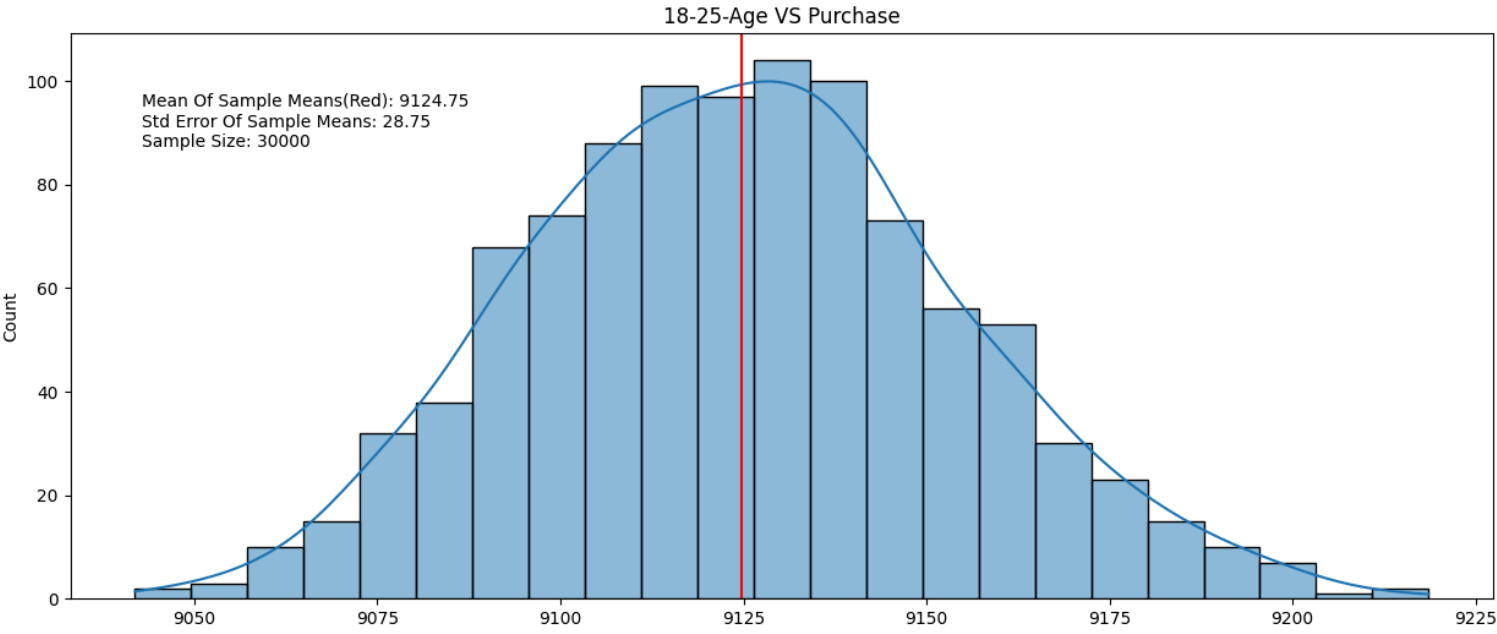
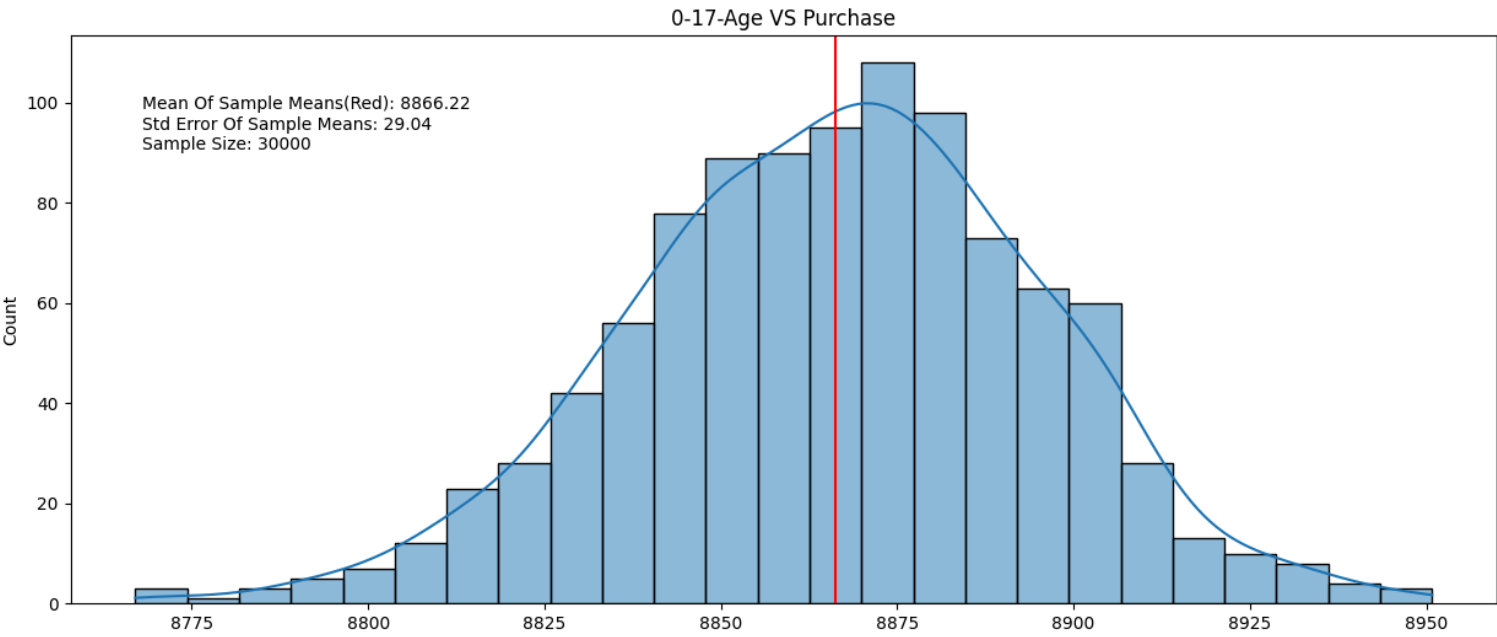
95% Confidence Interval For The Average Amount Spent By Age Category..

Sample Size: 30000

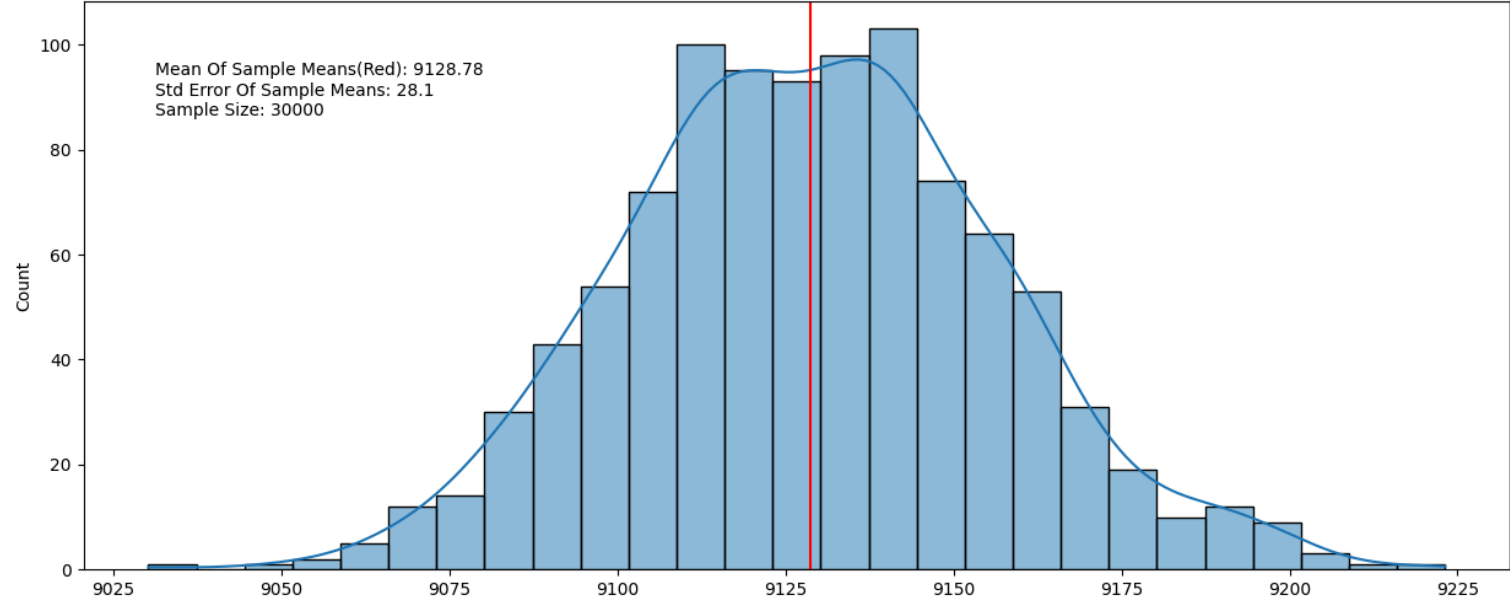
```
1 sample_size= 30000
2 age_cat= ['0-17', '18-25', '26-35', '36-45', '46-50', '51-55', '55+']
3 age_sample_means={}
4 age_means={}
5 age_se={}
6 for i in age_cat:
7     age_sample_means[f'{i}_sample_mean_{sample_size}'] = [np.mean(data.loc[data['Age']== i, 'Purchase'].sample(sample_size, replace= True))
8                                                         for _ in range(1000)]
9     age_means[f'{i}_mean_{sample_size}'] = np.mean(age_sample_means[f'{i}_sample_mean_{sample_size}'])
10    age_se[f'{i}_se_{sample_size}']= data.loc[data['Age']== i, 'Purchase'].std()/np.sqrt(sample_size)
11    print(f'{i} Purchase Amount- Mean Of Sample Means:', age_means[f'{i}_mean_{sample_size}'])
12    print(f'{i} Purchase Amount- Std Error Of Sample Means:', age_se[f'{i}_se_{sample_size}'])
13    print('-'*100)
```

```
0-17 Purchase Amount- Mean Of Sample Means: 8866.216896966665
0-17 Purchase Amount- Std Error Of Sample Means: 29.041023646300076
-----
18-25 Purchase Amount- Mean Of Sample Means: 9124.749422933333
18-25 Purchase Amount- Std Error Of Sample Means: 28.74529453833825
-----
26-35 Purchase Amount- Mean Of Sample Means: 9193.2854856
26-35 Purchase Amount- Std Error Of Sample Means: 28.50615512966409
-----
36-45 Purchase Amount- Mean Of Sample Means: 9253.48639
36-45 Purchase Amount- Std Error Of Sample Means: 28.450345750428156
-----
46-50 Purchase Amount- Mean Of Sample Means: 9128.776499866666
46-50 Purchase Amount- Std Error Of Sample Means: 28.102027549596162
-----
51-55 Purchase Amount- Mean Of Sample Means: 9423.686769400001
51-55 Purchase Amount- Std Error Of Sample Means: 28.599880720707723
-----
55+ Purchase Amount- Mean Of Sample Means: 9216.803052333333
55+ Purchase Amount- Std Error Of Sample Means: 28.068614239326312
-----
```

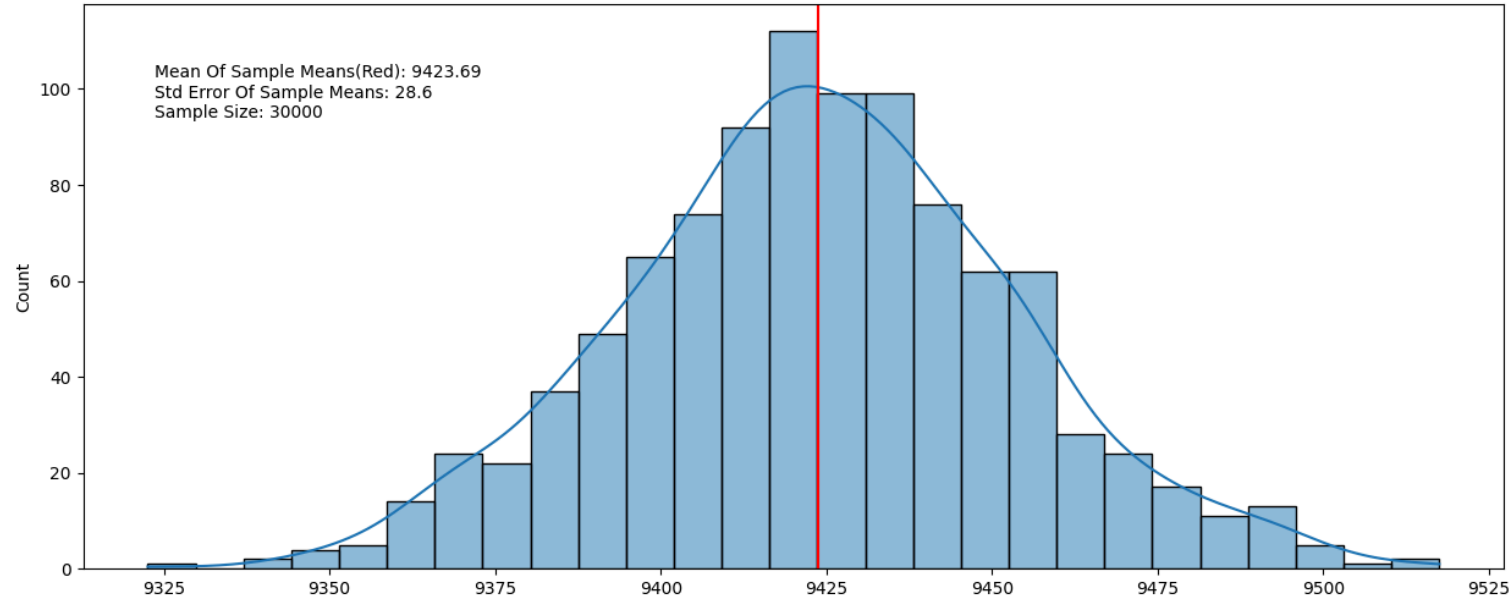
```
1 for i in age_cat:
2     plt.figure(figsize=(15,6))
3     plt.title(f'{i}-Age VS Purchase')
4     sns.histplot(age_sample_means[f'{i}_sample_mean_{sample_size}'], kde= True, fill= True)
5     plt.axvline(x= age_means[f'{i}_mean_{sample_size}'], color='r')
6     temp_mean= age_means[f'{i}_mean_{sample_size}']
7     temp_se= age_se[f'{i}_se_{sample_size}']
8     plt.annotate(f'Mean Of Sample Means(Red): {np.round(temp_mean, 2)} \nStd Error Of Sample Means: {np.round(temp_se, 2)}
9                 \nSample Size: {sample_size}', xy=(0.05, 0.80), xycoords='axes fraction', fontsize=10)
10    plt.show()
```



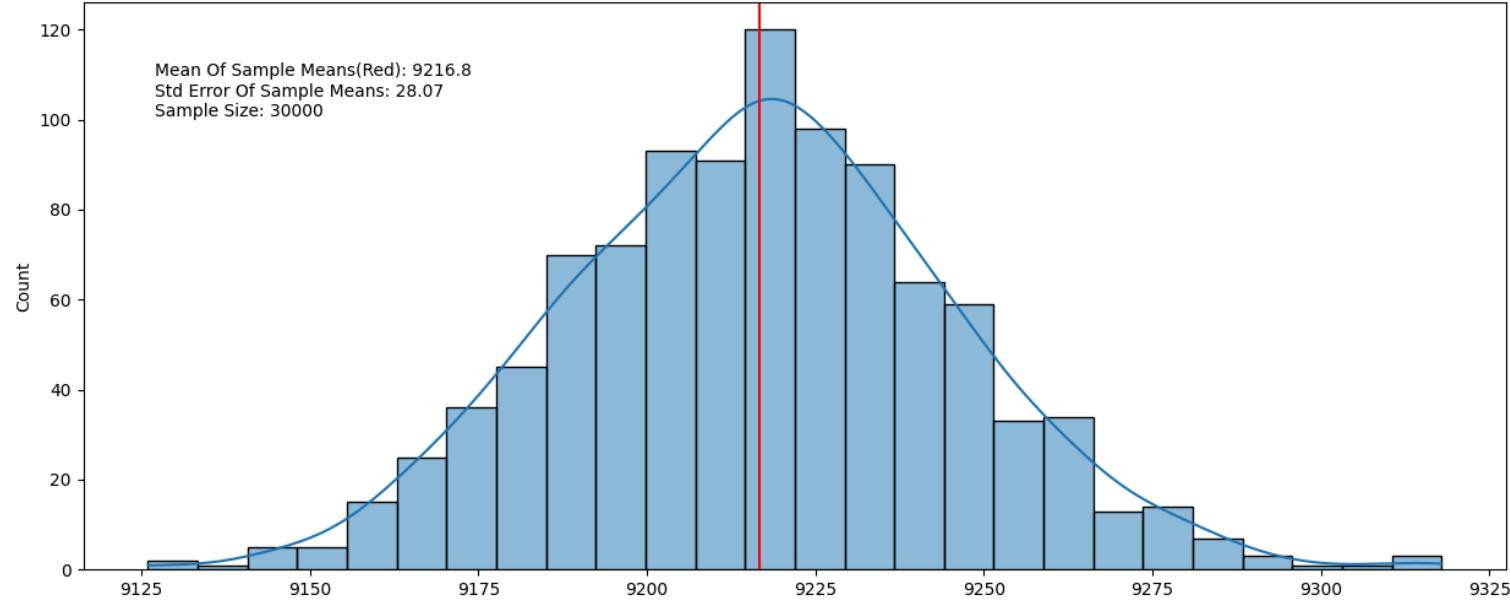
46-50-Age VS Purchase



51-55-Age VS Purchase



55+-Age VS Purchase



```
1 age_ci={}
2 for i in age_cat:
3     age_ci[f'{i}_lower_bound_{sample_size}'] = age_means[f'{i}_mean_{sample_size}'] + (norm.ppf(0.025) * age_se[f'{i}_se_{sample_size}'])
4     age_ci[f'{i}_upper_bound_{sample_size}'] = age_means[f'{i}_mean_{sample_size}'] + (norm.ppf(0.975) * age_se[f'{i}_se_{sample_size}'])
5     print(f'95% Confidence Interval For The Average Amount Spent By {i} For Sample Size {sample_size}:',
6           (age_ci[f'{i}_lower_bound_{sample_size}'], age_ci[f'{i}_upper_bound_{sample_size}']))
7     print('-'*100)
```

➡ 95% Confidence Interval For The Average Amount Spent By 0-17 For Sample Size 30000: (8809.29753654574, 8923.13625738759)

95% Confidence Interval For The Average Amount Spent By 18-25 For Sample Size 30000: (9068.409680913195, 9181.089164953471)

95% Confidence Interval For The Average Amount Spent By 26-35 For Sample Size 30000: (9137.414448208146, 9249.156522991852)

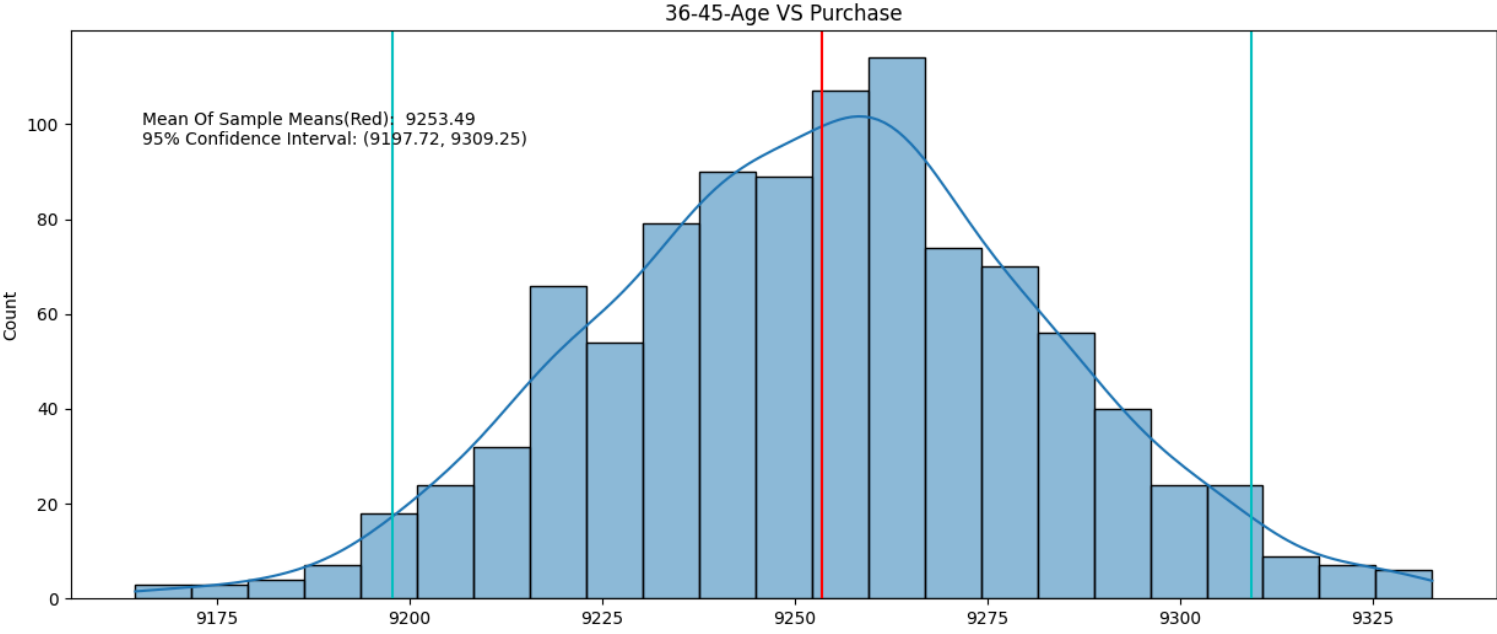
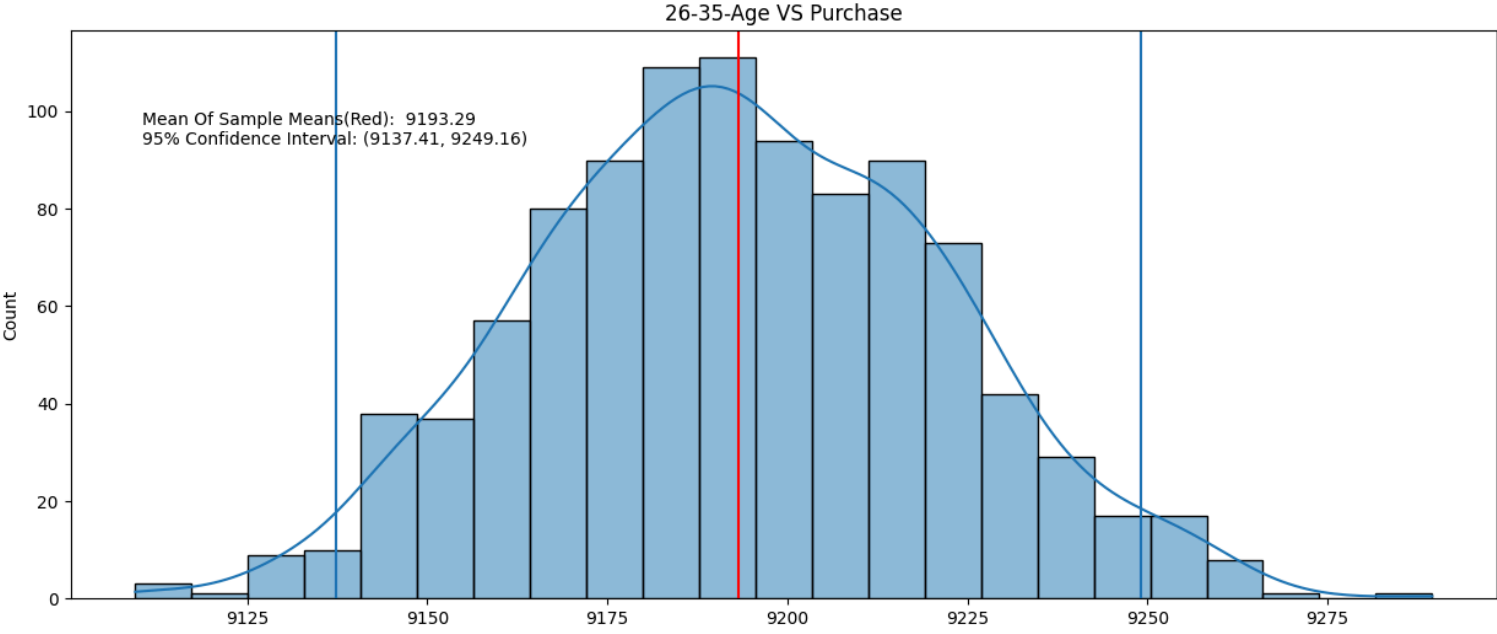
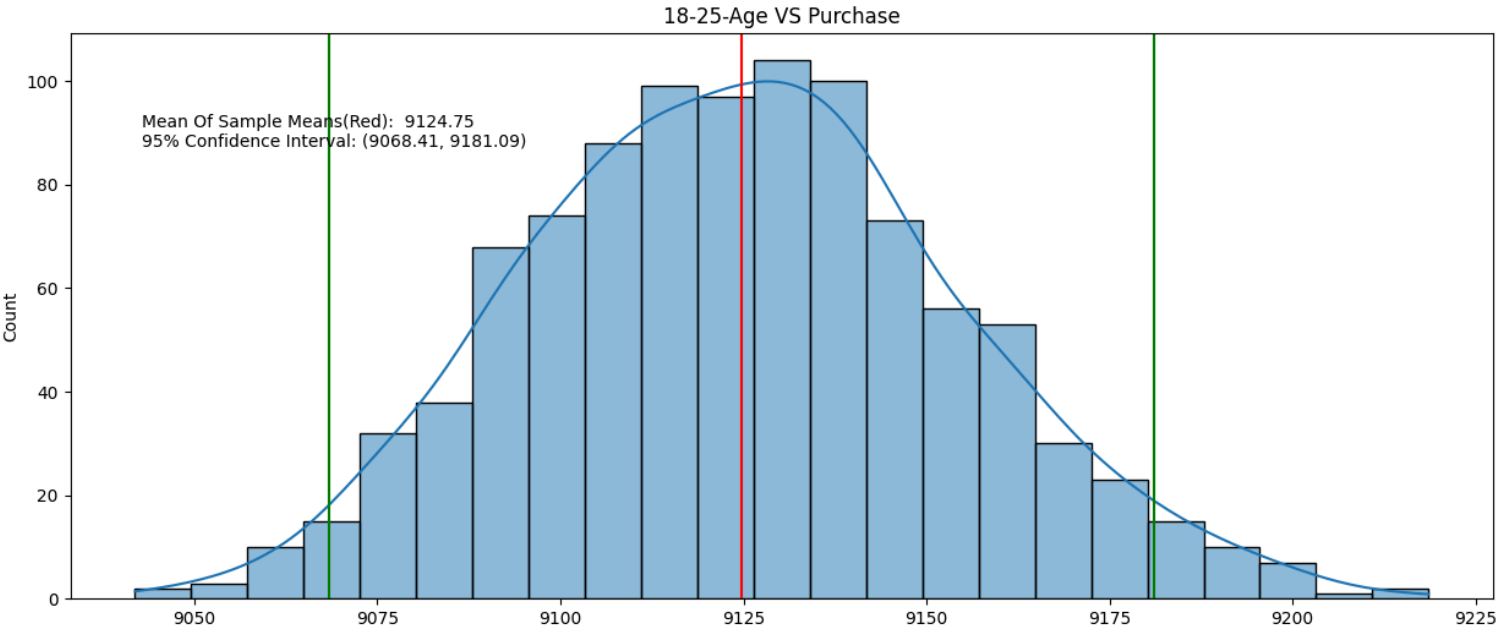
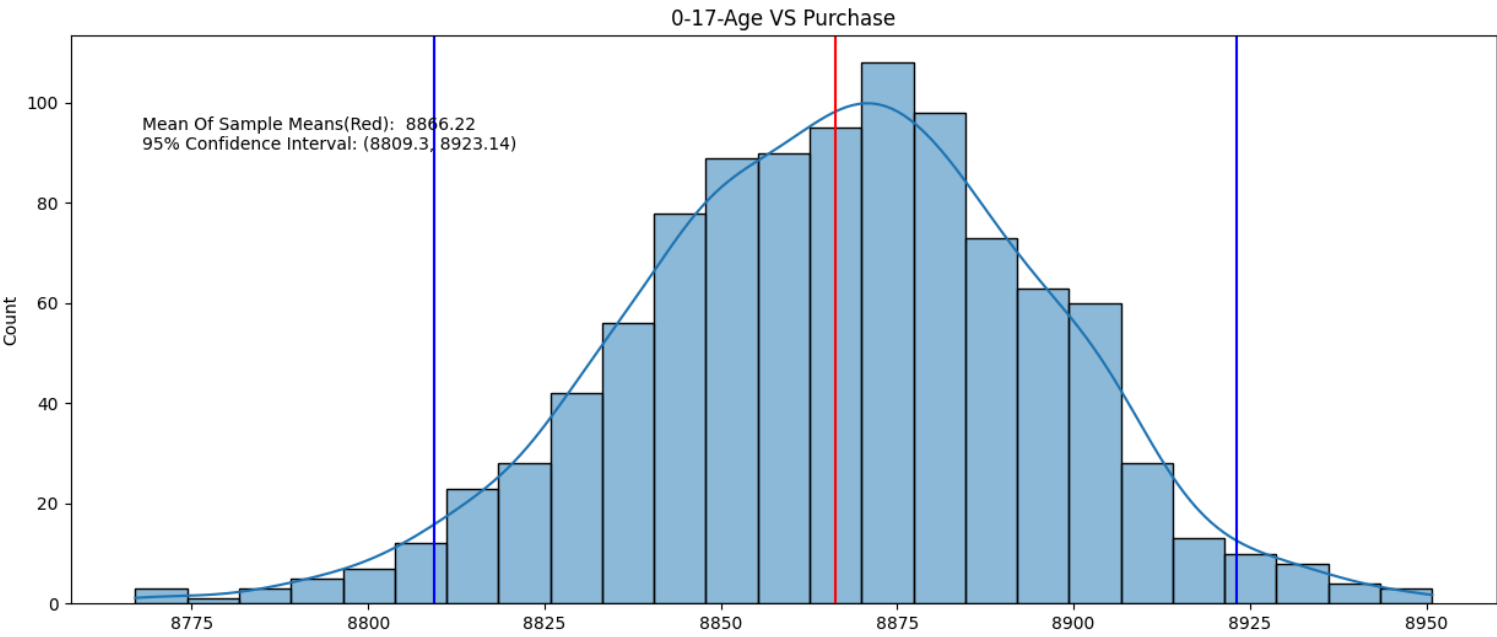
95% Confidence Interval For The Average Amount Spent By 36-45 For Sample Size 30000: (9197.724736981449, 9309.248043018551)

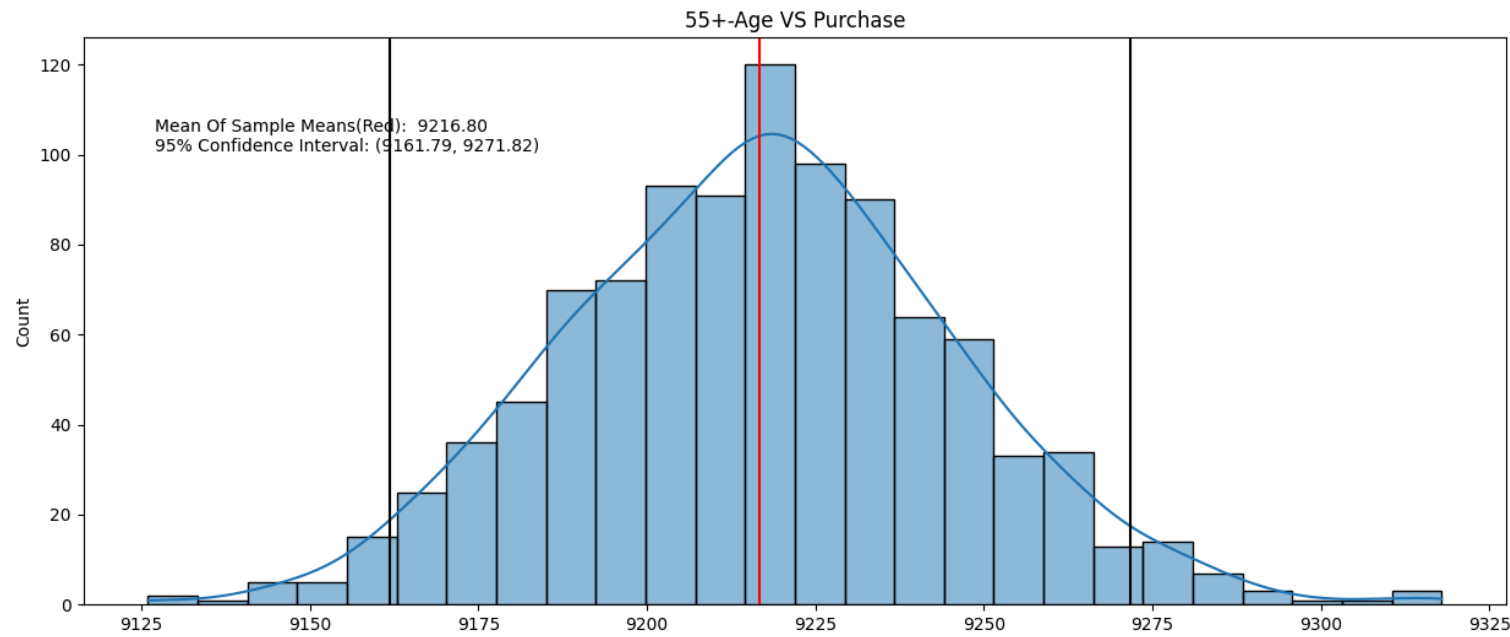
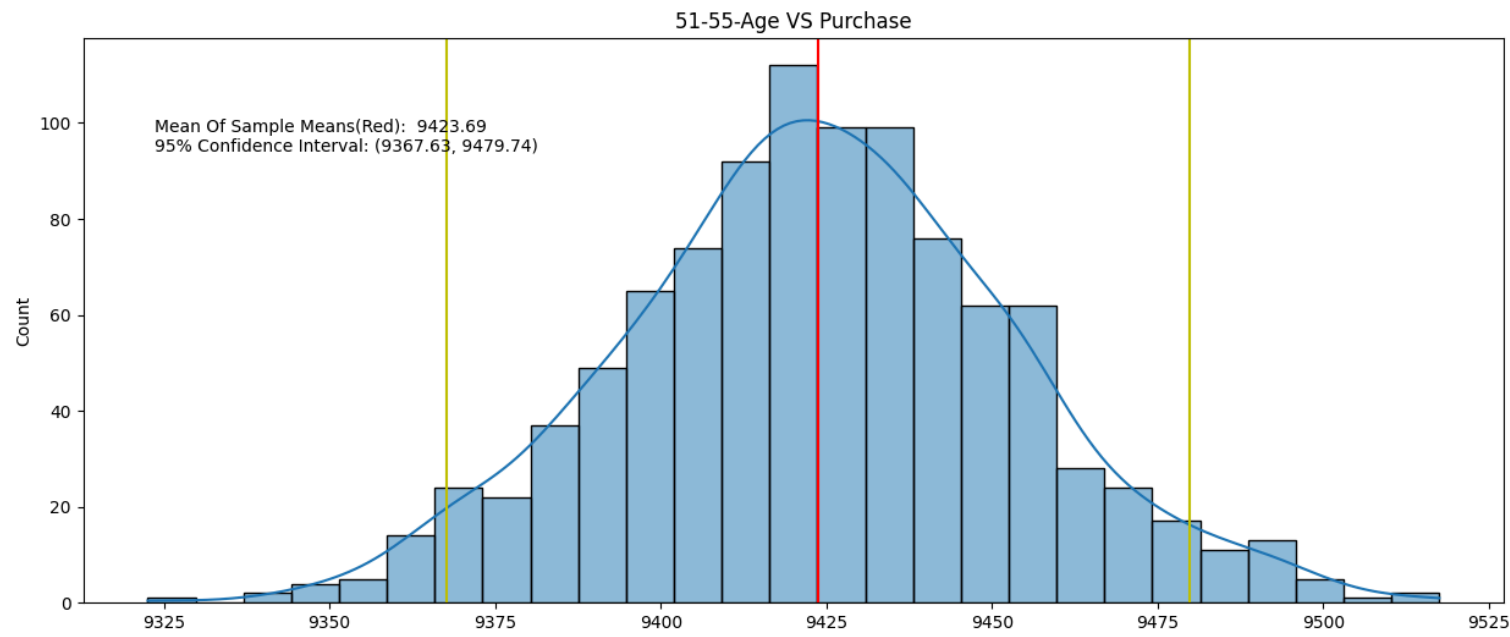
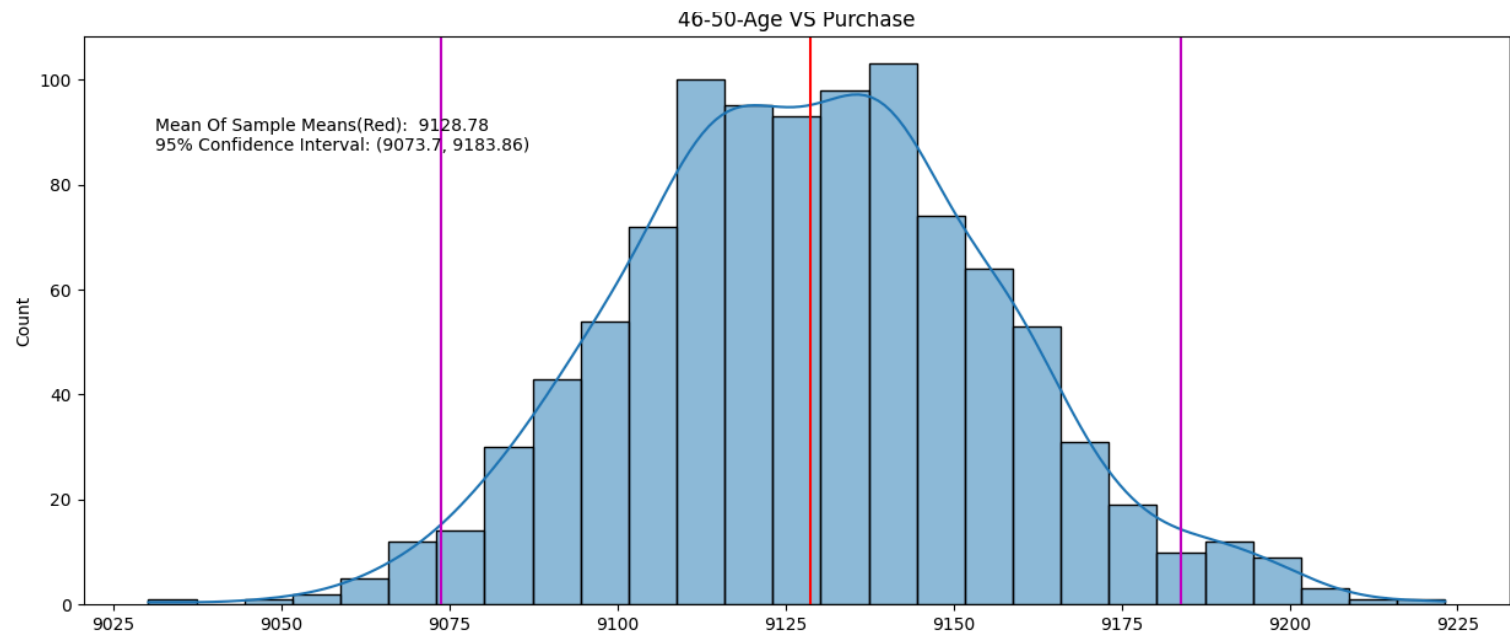
95% Confidence Interval For The Average Amount Spent By 46-50 For Sample Size 30000: (9073.697537976905, 9183.855461756428)

95% Confidence Interval For The Average Amount Spent By 51-55 For Sample Size 30000: (9367.632033225273, 9479.74150557473)

95% Confidence Interval For The Average Amount Spent By 55+ For Sample Size 30000: (9161.789579328304, 9271.816525338361)

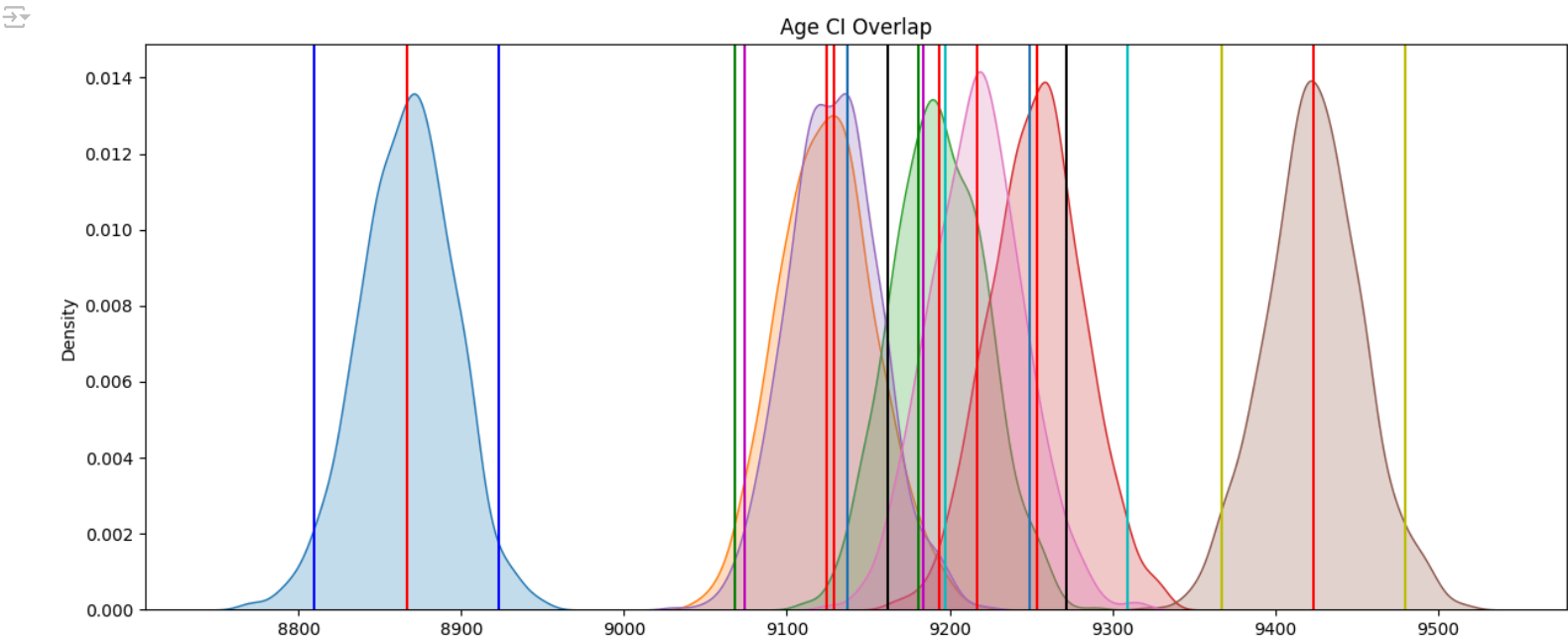
```
1 color= ['b', 'g', '#1f77b4', 'c', 'm', 'y', 'k']
2 age_se_list= list(age_se.keys())
3 for i in age_cat:
4     plt.figure(figsize=(15,6))
5     plt.title(f'{i}-Age VS Purchase')
6     sns.histplot(age_sample_means[f'{i}_sample_mean_{sample_size}'], kde= True, fill= True)
7     plt.axvline(x= age_means[f'{i}_mean_{sample_size}'], color='r')
8     temp_mean= age_means[f'{i}_mean_{sample_size}']
9     plt.axvline(x= age_ci[f'{i}_lower_bound_{sample_size}'], color=color[age_se_list.index(f'{i}_se_{sample_size}']))
10    temp_low= age_ci[f'{i}_lower_bound_{sample_size}']
11    plt.axvline(x= age_ci[f'{i}_upper_bound_{sample_size}'], color=color[age_se_list.index(f'{i}_se_{sample_size}']))
12    temp_up= age_ci[f'{i}_upper_bound_{sample_size}']
13    plt.annotate(f'Mean Of Sample Means(Red): {np.mean(temp_mean) : .2f} \n95% Confidence Interval:
14                {(np.round(temp_low, 2), np.round(temp_up,2))}', xy=(0.05, 0.80), xycoords='axes fraction', fontsize=10)
15    plt.show()
```





Overlapping ?

```
1 color= ['b', 'g', '#1f77b4', 'c', 'm', 'y', 'k']
2 age_se_list= list(age_se.keys())
3 a, b, c, d, e, f, j = age_cat
4 age_cat_list= [a, b, c, d, e, f, j]
5 plt.figure(figsize=(15,6))
6 plt.title('Age CI Overlap')
7 for i in age_cat_list:
8     sns.kdeplot(age_sample_means[f'{i}_sample_mean_{sample_size}'], fill= True)
9     plt.axvline(x= age_means[f'{i}_mean_{sample_size}'], color='r')
10    plt.axvline(x= age_ci[f'{i}_lower_bound_{sample_size}'], color=color[age_se_list.index(f'{i}_se_{sample_size}']])
11    plt.axvline(x= age_ci[f'{i}_upper_bound_{sample_size}'], color=color[age_se_list.index(f'{i}_se_{sample_size}']])
12 plt.show()
```



Conclusion:

Small Samples (n=300): High variability and broad intervals mask meaningful differences across age groups.

Larger Samples (n=3000): Differences begin to emerge, though overlaps in spending between adjacent age groups remain.

Largest Sample (n=30,000): Overlaps reduce significantly, revealing a clear spending hierarchy: 51-55 > 36-45 > 55+ > 26-35 > 18-25 > 46-50 > 0-17.

7. Insights

- >*Gender Distribution: 75.31% Male vs. 24.69% Female customers. Male customers dominate purchases on Black Friday.*
- >*Marital Status: 59.03% Unmarried vs. 40.97% Married customers. Unmarried individuals contribute more to purchases.*
- >*City Category: Purchases are distributed across cities as: A: 26.85%, B: 42.03%, C: 31.12%. City "B" sees the highest activity.*
- >*Popular Product Categories: Categories 1, 5, and 8 are the most purchased. Category 5 leads in popularity.*
- >*Age Group Insights: Customers aged 26–30 are the most active buyers, particularly in categories 1, 5, 8, and 11.*

▽How does Gender affect the amount spent?▽

▽ *Initial Observations (Sample Size = 300):*

Males: Mean = ₹9370.91, 95% CI = (₹8804.08, ₹9937.75)

Females: Mean = ₹8666.57, 95% CI = (₹8137.10, ₹9196.05)

Observation: Confidence intervals overlap, so we cannot conclusively state that males spend more than females.

▽ *Increased Sample Size (3000):*

Males: Mean = ₹9365.12, 95% CI = (₹9185.87, ₹9544.37)

Females: Mean = ₹8671.77, 95% CI = (₹8504.33, ₹8839.20)

Observation: CI no longer overlap, providing evidence that males spend more than females.

▽ *Further Increased Sample Size (30000):*

Males: Mean = ₹9366.82, 95% CI = (₹9310.14, ₹9423.51)

Females: Mean = ₹8671.26, 95% CI = (₹8618.31, ₹8724.20)

Observation: Narrower CIs and non-overlapping intervals confirm with high confidence that males spend more than females.

▽How does Marital_Status affect the amount spent?▽

▽ *Initial Observations (Sample Size = 300):*

Unmarried: Mean = ₹9201.28, 95% CI = (₹8641.33, ₹9761.22)

Married: Mean = ₹9197.15, 95% CI = (₹8639.82, ₹9754.48)

Observation: Overlapping confidence intervals indicate no significant difference in spending.

▽ *Increased Sample Size (3000):*

Unmarried: Mean = ₹9199.19, 95% CI = (₹9022.12, ₹9376.26)

Married: Mean = ₹9189.66, 95% CI = (₹9013.42, ₹9365.90)

Observation: Narrower CIs, but still overlapping, confirming similar spending patterns.

▽ Further Increased Sample Size (30,000):

Unmarried: Mean = ₹9201.11, 95% CI = (₹9145.12, ₹9257.11)

Married: Mean = ₹9186.31, 95% CI = (₹9130.58, ₹9242.05)

Observation: Overlap persists even with a large sample, indicating that spending by marital status has similar patterns.

▽How does Age affect the amount spent?▽

▽ >1. Sample Size = 300: Confidence intervals overlap significantly across age groups, making it difficult to confidently state differences in spending between age categories. Example: 0-17 (₹8300.44, ₹9438.82) overlaps with 18-25 (₹8546.02, ₹9672.81) and several other age groups. Indication: With a small sample size, variability is high, resulting in broad confidence intervals and overlapping ranges.

▽ >2. Sample Size = 3000: Increasing the sample size narrows the confidence intervals, but overlaps still exist between most age groups. Example: 18-25 (₹8944.08, ₹9300.40) overlaps with 26-35 (₹9013.19, ₹9366.55) and 36-45 (₹9084.03, ₹9436.70). Indication: Though differences become clearer for some groups, overlaps suggest spending differences among age categories are still not strongly distinguishable.

▽ >3. Sample Size = 30,000: Confidence intervals are much narrower, with minimal overlap between age groups. Example: 0-17 (₹8808.93, ₹8922.77) does not overlap with 18-25 (₹9066.65, ₹9179.33) or higher age groups. 51-55 (₹9366.47, ₹9478.58) stands out with the highest spending, and its interval does not overlap with younger groups. Indication: Larger sample sizes reveal clearer distinctions in spending patterns, showing 51-55 age group spends the most.

✓ 8. Recommendations:

>Target Females with special Black Friday campaigns to increase their participation. Highlight offers on products that traditionally appeal more to women.

>Promote bundles or discounts aimed at single individuals, such as gadgets or lifestyle products. Use themes like "Treat Yourself" to attract this group.

>Enhance advertising in City B to capitalize on its high engagement. Boost presence in City A and C with location-specific offers to balance sales distribution.

>Stock and promote Categories 1, 5, and 8 prominently in stores and online. Offer loyalty points or discounts on repeat purchases in these categories.

>Develop targeted campaigns for the 26-30 age group, focusing on popular categories like 1, 5, 8, and 11. Use digital channels like social media and influencers that resonate with this demographic.

>Bundle popular categories (e.g., Category 5 with 8) to increase cart size. Provide personalized recommendations based on purchase history.

>Introduce exclusive, high-value offers tailored to male preferences, such as electronics or outdoor equipment.

>Launch promotions specifically for females to close the spending gap, focusing on popular categories like apparel or home goods.

>Design bundles that appeal to both genders, promoting complementary products to boost cart sizes.

>Use detailed analytics to refine offers based on spending patterns, ensuring maximum engagement from high-spending male demographics.

>Design campaigns that appeal equally to married and unmarried customers, focusing on shared preferences like popular product categories.

>Introduce special discounts on family bundles or lifestyle products to attract married customers, leveraging their potential for larger household purchases.

>Highlight offers for personal-use products or experiences (like gadgets or subscriptions) to attract unmarried customers.

>Since marital status does not significantly impact spending, prioritize strategies based on factors like gender, age, or city category for more targeted growth.

>Target Age Group 51-55: Promote premium products and loyalty programs to capitalize on this high-spending demographic. Highlight offers on categories like electronics, furniture, and appliances, which appeal to middle-aged consumers.

>Focus on Younger Groups: Develop budget-friendly bundles and discounts for the 0-17 and 18-25 age groups to boost spending. Use targeted social media campaigns for younger demographics.

>Age-Specific Marketing: Tailor advertisements and product placements based on age group preferences. For example, promote practical products to the 36-45 age group and indulgent ones to 51-55.

>Bundle products for families, especially targeting the 36-55 age groups who likely have dependents and households.

✓ THANK YOU

-Walmart Case Study By: Nitish Vishwanath Hatapaki